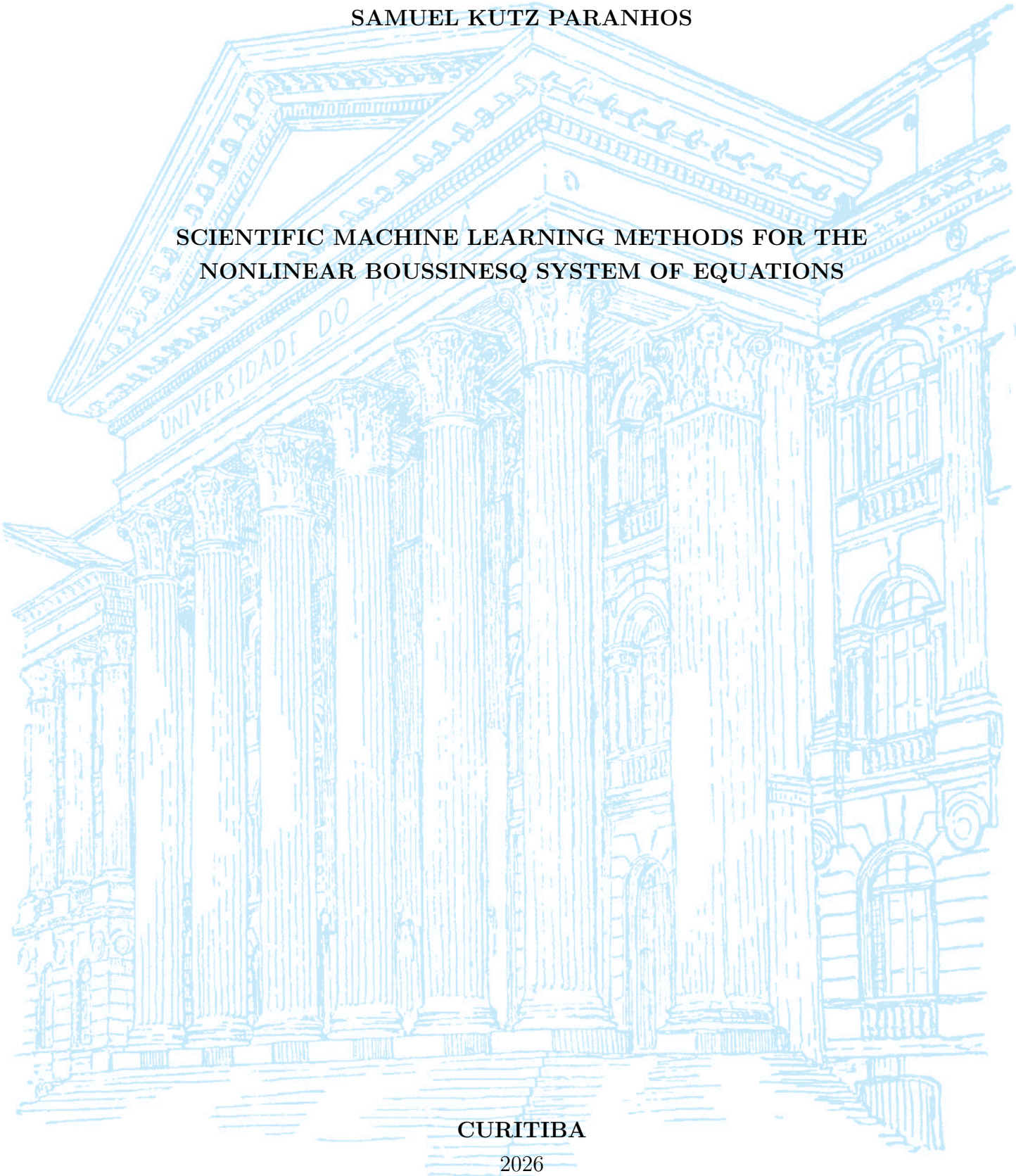
UNIVERSIDADE FEDERAL DO PARANÁ

SAMUEL KUTZ PARANHOS

SCIENTIFIC MACHINE LEARNING METHODS FOR THE NONLINEAR BOUSSINESQ SYSTEM OF EQUATIONS



CURITIBA

2026

SAMUEL KUTZ PARANHOS

TRABALHO DE CONCLUSÃO DE CURSO

SCIENTIFIC MACHINE LEARNING METHODS FOR THE NONLINEAR BOUSSINESQ SYSTEM OF EQUATIONS

Trabalho apresentado como requisito parcial à obtenção do grau de Bacharel em Matemática Industrial no Departamento de Matemática da Universidade Federal do Paraná.

Área de concentração: Matemática Industrial.

Orientador: Prof. Roberto Ribeiro.

Título do Projeto: Scientific Machine Learning Methods for the Nonlinear Boussinesq System of Equations.

CURITIBA

2026

RESUMO

Um dos modelos clássicos da hidrodinâmica é o sistema de Boussinesq, o qual consiste em um sistema de equações diferenciais parciais não lineares que tem como uma das incógnitas o perfil da onda de superfície. Este modelo já foi bastante explorado teoricamente e numericamente via métodos clássicos, como o método pseudoespectral. Contudo, até onde sabemos, nenhum trabalho tratou do sistema de Boussinesq via métodos de Aprendizado de Máquina Científico (*Scientific Machine Learning*, SciML). Este trabalho visa preencher esta lacuna na literatura por meio do estudo de seis métodos de SciML. Uma solução numérica construída por meio do método pseudoespectral serve como solução de referência e conjunto de dados para os métodos estudados. O objetivo principal deste trabalho é verificar a acurácia dos seis métodos: um perceptron multicamadas; uma Rede Neural Informada pela Física, com e sem dados; um Operador Neural de Fourier; e um Operador Neural Informado pela Física, com e sem dados. Por "dados", entende-se que, para o treinamento, é fornecido um conjunto de dados de referência. Além da comparação destes métodos com a solução de referência, este trabalho também tem como objetivo investigar o viés espectral via um perceptron multicamadas o mais simples possível, e realizar um estudo comparativo com previsões teóricas obtidas por análise do Núcleo Tangente Neural (NTK). Os ensaios numéricos realizados indicam que o viés espectral aparece em todos os métodos, porém, em alguns deles, de maneira mais atenuada. De maneira geral, conclui-se que a combinação de dados supervisionados e resíduo da EDP apresentou, de longe, o melhor desempenho. Além disso, as estimativas de decaimento da função objetivo ao longo das iterações, previstas pela teoria do NTK, verificaram-se na prática.

Palavras-chave: Scientific Machine Learning; Equações Diferenciais Parciais; Sistema de Boussinesq; Redes Neurais Informadas pela Física; Viés Espectral.

ABSTRACT

One of the classical models of hydrodynamics is the Boussinesq system, which consists of a system of nonlinear partial differential equations having the surface wave profile as one of its unknowns. This model has already been extensively explored theoretically and numerically via classical methods, such as the pseudospectral method. However, to the best of our knowledge, no work has addressed the Boussinesq system via Scientific Machine Learning (SciML) methods. This work aims to fill this gap in the literature through the study of six SciML methods. A numerical solution constructed via the pseudospectral method serves as the reference solution and dataset for the methods studied. The main objective of this work is to verify the accuracy of the six methods: a multilayer perceptron; a Physics-Informed Neural Network, with and without data; a Fourier Neural Operator; and a Physics-Informed Neural Operator, with and without data. By "data", it is understood that, for training, a reference dataset is provided. In addition to comparing these methods with the reference solution, this work also aims to investigate spectral bias via the simplest possible multilayer perceptron, and to carry out a comparative study with theoretical predictions obtained through Neural Tangent Kernel (NTK) analysis. The numerical experiments performed indicate that spectral bias appears in all methods, although in some of them it is more attenuated. Overall, it is concluded that the combination of supervised data and PDE residual presented, by far, the best performance. Furthermore, the objective function decay estimates over iterations, predicted by NTK theory, were verified in practice.

Keywords: Scientific Machine Learning; Partial Differential Equations; Boussinesq System; Physics-Informed Neural Networks; Spectral Bias.

ACKNOWLEDGEMENTS

First, I need to express my deepest gratitude to the professors at Federal University of Paraná, Prof. Roberto Ribeiro and Prof. Thiago Quinelato. In the fifth semester, they revitalized my love for mathematics after a burnout in the fourth semester. I did not like numerical analysis, nor anything relating to optimization; suddenly the career I was chasing escaped me and nothing made sense. They brought it back, through their pedagogical excellence and their patience in explaining each concept. I am grateful that they can now grade this work that finishes one of the most important moments of my life.

I need to also express my love and gratitude to my mother, Angela Kutz. To all of her three sons, she knew that the only education could improve things. Indeed, now I understand and I look forward to returning all the effort and faith she put in me.

I chose to study applied mathematics very early in my life, around sixteen years old. I did not know a thing about it, just thought the name was cool to show around. Now, I know this is what I love. My biggest hobby. I cannot stay far from mathematics anymore and I know for a fact that for the rest of my life I will be trying to learn new mathematical content every day. It now brings food to my table, and it will continue to be a bedrock skill I developed that makes me survive in this world.

Also, this course brought me some of my best friends that I will most certainly take to life. To all my dear friends, thank you.

Last, but not least, I would like to thank P4E (Pró-Reitoria de Pertencimento e Políticas de Permanência Estudantil), which enabled me to have the financial support to begin this course. Without it, I would not be here. I had nothing. Federal University of Paraná (UFPR) showed me a path I could pursue. Also, I thank the CNPq for financial aid in my undergraduate research, the LabFluid laboratory for providing the best learning environment that I could have, with daily debates and lessons, and UFPR for their support, amazing professors, and for providing democratized knowledge for us, the wicked.

This work was supported by CNPq and developed with the collaboration of the LabFluid research group at UFPR.

"We are part of the universe that has developed a remarkable ability: we can hold an image of the world in our minds. We are matter contemplating itself."

CARROLL, *The Particle at the End of the Universe*, 2012.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

SciML	Scientific Machine Learning
ML	Machine Learning
PDE	Partial Differential Equation
ODE	Ordinary Differential Equation
UDE	Universal Differential Equation
SINDy	Sparse Identification of Nonlinear Dynamics
ANN	Artificial Neural Network
NN	Neural Network
MLP	Multilayer Perceptron
PINN	Physics-Informed Neural Network
NO	Neural Operator
FNO	Fourier Neural Operator
PINO	Physics-Informed Neural Operator
NTK	Neural Tangent Kernel
DFT	Discrete Fourier Transform
IDFT	Inverse Discrete Fourier Transform
FFT	Fast Fourier Transform
RK4	Runge–Kutta 4th-order method
GD	Gradient Descent
MSE	Mean Squared Error
AD	Automatic Differentiation
GPU	Graphics Processing Unit

LIST OF SYMBOLS

Domain and general operators

x	spatial coordinate
t	temporal coordinate
$u(x,t)$	PDE solution (generic placeholder; not the Boussinesq velocity field below)
$\mathcal{D}$	differential operator of the PDE
$\mathcal{L}$	linear differential operator; also a loss functional
∇	gradient (nabla) operator
Ω	spatial domain, $\Omega = [-L, L]$
T	final time of the horizon $[0, T]$
N_x	number of spatial grid points
N_t	number of time steps
Δx, Δt	spatial and temporal grid spacings
$\mathcal{S}$	parametric dataset of reference solutions
M	number of samples in $\mathcal{S}$

Boussinesq system and spectral quantities

$\eta(x,t)$	surface elevation field
$u(x,t)$	depth-averaged horizontal velocity field
α	nonlinearity parameter of the Boussinesq system
β	dispersion parameter of the Boussinesq system
A	initial wave amplitude
L	domain half-length, $\Omega = [-L, L]$
k	wavenumber (integer mode index)
ξ_k	wavenumber $\pi k/L$ on the domain $[-L, L]$
$\hat{f}(k)$	Fourier coefficient of f at mode k
$\hat{f}(k,t)$	time-evolving Fourier mode of f at wavenumber k
ω	primitive root of unity $e^{-2\pi \mathrm{i}/N}$ of the DFT matrix
N_k	number of retained real-DFT modes
B_{low}, B_{mid}, B_{high}	low, mid and high wavenumber bands

Error metrics

$E(t_n)$	time-resolved relative L^2 error at time level t_n
$E_{\text{spec}}(k, t_n)$	absolute spectral error at mode k and time t_n
$E_{\text{band}}(B)$	band-averaged absolute spectral error over band B

Neural networks and Neural Tangent Kernel

θ	trainable parameter vector of a neural network
u_θ	neural network approximation of the PDE solution
$h^{(l)}$	hidden-layer activation vector at layer l
σ	activation function
$W^{(l)}$, $b^{(l)}$	weight matrix and bias vector at layer l
N_{MLP}	number of neurons in a hidden layer
L_{MLP}	number of layers in the neural network
N_θ	number of trainable parameters
μ	learning rate / gradient-descent step size
J, J_0	Jacobian of the network output, and its value at initialization
$K(\theta)$, K_0	empirical Neural Tangent Kernel, $K = JJ^\mathsf{T}$, and its value at initialization
V, Λ	eigenvectors and eigenvalues of K_0, $K_0 = V\Lambda V^\mathsf{T}$
λ_k	NTK eigenvalue associated with mode k
$\lambda_{\max}$	largest eigenvalue of K_0
v_k	k-th eigenvector of K_0
e, $e^{(n)}$	training-error vector on the grid, and its value at iteration n
c_k	error coefficient along the eigendirection v_k
τ_k	decay time constant of mode k
n_k	iterations for mode k to reach the precision ε
ε	precision threshold on the modal error
$\hat{e}(k, t)$	Fourier coefficient of the training error at mode k

Neural operators

$G^\dagger$, G_θ	target solution operator and its neural approximation
$\mathcal{K}_\ell$	integral operator of the ℓ-th FNO spectral layer
κ_ℓ	kernel function of the FNO integral operator
P	lifting operator of the FNO
Q	projection operator of the FNO
d_c	channel (width) dimension of the FNO
$R_{\ell,k}$	FNO spectral weight (discretized kernel) at layer ℓ, mode k
$k_{\max}$	FNO mode-truncation threshold

1 INTRODUCTION

Since the start of scientific thinking, mathematics has been one of the key languages in which we can understand the world, and Partial Differential Equations (PDEs) are one of the main dialects of the mathematical language we use to model and to simplify nature's complexity.

When modeling these equations, we cannot always afford them to have simple closed-form solutions (that is, when they have a solution at all). This exposes the need to obtain approximate solutions coming from numerical methods for these PDEs, also giving the option to circumvent analytical complexity and allowing the extraction of predictions, comparisons with data, the building of simulations, and so on. These classical numerical methods, widely studied for all types of differential equations, usually rely on discretizing the PDE's domain into grids and performing temporal steps.

With the recent ascension of Machine Learning (ML) methods and a whole ecosystem of tools built around it, a natural path is to bridge these two areas and let ML augment the numerical solution of PDEs, taking advantage of the many software frameworks developed. This is one facet of Scientific Machine Learning (SciML), the broader effort to fuse physics-based, mechanistic models with data-driven methods.

Within SciML, this work focuses on the numerical solution of the Boussinesq System. Given the vast literature around the application of SciML on PDEs, we restrict our literature review to works on hydrodynamics equations.

1.1 Literature Overview on Scientific Machine Learning for Hydrodynamics PDEs

The neural network building blocks underlying all these SciML approaches trace a line beginning with the formal neuron of McCulloch et al. (1943) and Rosenblatt's perceptron (ROSENBLATT, 1958), the first trainable single-layer architecture. The key algorithmic advance that unlocked multi-layer training was the backpropagation algorithm (RUMELHART et al., 1986), making gradient-based optimization practical for deep architectures. Shortly after, the universal approximation theorem (CYBENKO, 1989; HORNIK et al., 1989) provided the theoretical guarantee for a multilayer percep-

tron (MLP) with a single hidden layer of sufficient width to be able to approximate any continuous function on a compact domain to arbitrary precision. This combination of practical trainability and universal approximation established the MLP as the backbone of modern deep learning. The term neural network itself reflects the biological metaphor at the heart of these architectures, in which McCulloch et al. (1943) drew an explicit analogy to the firing of biological neurons, and this vocabulary propagated through the field, causing MLPs to be routinely and interchangeably referred to as artificial neural networks (ANNs) or feedforward neural networks. The idea of "deep" in deep learning, popularized following the breakthrough results of LeCun et al. (2015), simply denotes an MLP with many hidden layers, a regime in which representational capacity grows substantially and where the backpropagation-based training paradigm proves to be exceptionally efficient.

These ML foundations become the basis of a scientific methodology once the methods are coupled to the physical laws that govern a problem of interest. This is the premise of Scientific Machine Learning, a term consolidated by Baker et al. (2019) to name the discipline at the intersection of scientific computing and machine learning, and surveyed in breadth by Karniadakis et al. (2021). Its unifying goal is to combine the interpretability, generalization, and inductive biases of physics-based models with the flexibility and data-adaptivity of ML, rather than treating the network as a pure black box. The methods span several families. Some recover governing equations directly from data, as in the Sparse Identification of Nonlinear Dynamics (SINDy) (BRUNTON et al., 2016). Others embed differential-equation structure into the network itself, as in Neural Ordinary Differential Equations (neural ODEs) (CHEN, R. T. Q. et al., 2018) and Universal Differential Equations (UDEs), in which a trainable network stands in for the unknown terms of an otherwise mechanistic model (RACKAUCKAS et al., 2020). These tools have been carried across a wide range of domains, from global weather and climate forecasting (BONEV et al., 2023) to molecular dynamics, materials design, and turbulence modeling (KARNIADAKIS et al., 2021). Among all these directions, the one that concerns us here is the use of SciML to obtain numerical solutions of hydrodynamics equations, and in particular of the Boussinesq system presented in Section 1.2.

SciML for PDEs was reshaped by one idea in particular, the Physics-Informed Neural Networks (PINNs) (RAISSI et al., 2019), that is, the embedding of the governing PDE residual directly into the training loss of a Neural Network, working as a regularization term for not only fitting data, but also respecting the underlying physics. In hydrodynamics, a growing body of work has applied PINNs to water flow (DE PINA et al., 2023), shallow-water dynamics on the sphere (BIHLO et al., 2022), and inverse problems in strongly nonlinear wave systems (JAGTAP et al., 2022). The consistent result that we can address from reviews (CUOMO et al., 2023) is that PINNs are most effective when

observational data are sparse and the task is parameter identification, namely inverse problems, which are usually not well-posed. But, for a reliable precise forward simulation at scale, they do not yet rival classical solvers. In our vision, these approaches serve best to complement them.

As a natural extension from neural networks, there is a recent development in learning maps between infinite-dimensional function spaces, in which the theoretical foundation was popularized by T. Chen et al. (1995), who extended the universal approximation theorem to nonlinear operators, in other words, a shallow network can approximate any continuous operator between Banach spaces to arbitrary precision. Bringing this idea into practical terms, Lu et al. (2021) introduced DeepONet, the first deep architecture expressly designed for learning operators coming from differential equations, demonstrating it across a range of ODEs and PDEs. Building on this line of work, operator learning puts aside the single-instance limitation of PINNs by training an operator to approximate the full mapping from parameters and initial data to the solution, so that at inference time a new solution costs only a forward pass for the network. Parallel to DeepONet, the Fourier Neural Operator (FNO) (LI et al., 2021) is one of the most widely adopted instances of this idea: it parametrizes an integral kernel in Fourier space, achieving speed-ups over other neural operators, and it is discretization-independent, the so-called zero-shot superresolution. Subsequent works explored alternative bases like wavelets (GUPTA et al., 2021), but FNO was validated on dispersive equations and nonlinear equations such as nonlinear Schrödinger (NLS) equation, Hirota equation and showed to be able to capture different types of soliton solutions (e.g. bright solitons, breathers, peakons, rogons, and periodic waves) that appear in these nonlinear wave equations. FNO was also scaled to geophysical problems spanning regional ocean modeling, shallow-water emulation, and global weather forecasting (BONEV et al., 2023; LKHAGVASUREN et al., 2024; UCHIDA et al., 2025; PATEL et al., 2025; YU et al., 2025).

Following the idea for PINNs, the Physics-Informed Neural Operator (PINO) (LI et al., 2023a) combines operator learning with PDE-residual constraints, cutting the labelled-data requirement while preserving physical consistency. Applications to shallow-water systems (ROSOFSKY et al., 2023) confirmed that this hybrid regime inherits the generalization of FNO without its data hunger. Broad surveys of the field (SHARMA et al., 2023) identify these physics-data approaches working together as a promising SciML direction, while naming turbulence modeling, scalability, and noisy-data robustness as open challenges.

A known limitation that cuts across all SciML architectures is the so-called spectral bias, the tendency of gradient-based optimizers to learn the low-frequency components of a solution faster than the high-frequency ones (RAHAMAN et al., 2019; XU et al.,

2020). The theoretical account of this behavior came with the Neural Tangent Kernel (NTK) (JACOT et al., 2018), which showed that a sufficiently wide network trains, to leading order, as a linear model governed by a kernel that stays essentially fixed during optimization, the so-called lazy-training regime. In this picture each mode of the solution is learned at a rate set by its associated NTK eigenvalue, so the wide disparity between the largest and smallest eigenvalues translates directly into the frequency imbalance seen during training (ARORA et al., 2019; CAO et al., 2021; BORDELON et al., 2020). Wang, Yu, et al. (2022) carried this analysis to PINNs, showing that the eigenvalue disparity between the PDE-residual and initial-condition loss components is what drives their spectral bias. This pathology is especially consequential for dispersive wave problems, where the solution energy is broadly distributed across wavenumbers. A recent systematic study (KHODAKARAMI et al., 2026) showed that second-order optimizers combined with spectrum-aware loss formulations substantially mitigate it.

As far as this review reveals, SciML methods have been applied to a broad class of shallow-water and dispersive wave equations. To the best of our knowledge, however, no study has systematically evaluated FNO, PINO, or PINNs on the parametric 1D Boussinesq system, where both the nonlinearity coefficient α and the dispersion coefficient β vary simultaneously. This gap motivates our effort to implement and evaluate these SciML methods on that system, documenting the details a reader would need to apply them to a dispersive wave problem of their own.

1.2 The Boussinesq System of Equations

For this work, we choose to implement SciML methods for a specific PDE originating from J. Boussinesq's 1872 effort to give a theoretical account of the solitary wave of translation observed by J. Scott Russell in a canal in 1834 (BOUSSINESQ, 1872). Russell had noticed that a hump of water, set in motion by a stopping barge, could travel for miles without changing shape, a phenomenon the linear wave theory of the time could not explain. Boussinesq showed that the stability of such a wave is the result of a precise balance between nonlinear steepening and frequency dispersion, two effects that are separately destabilizing but mutually compensating in the shallow-water, long-wave regime. The mathematical structure of this balance, and of dispersive wave theory in general, are treated in depth by Whitham (1974) and Debnath (2012).

A modern two-field form of the system, for the free-surface elevation $\eta(x,t)$ and the depth-averaged horizontal velocity $u(x,t)$, was systematically derived and classified by Bona et al. (2002), who identified a four-parameter family of equivalent Boussinesq systems obtainable from the incompressible, irrotational Euler equations by asymptotic simplifications. Earlier, Peregrine (1967) had already written down the standard form

of these equations for water of variable depth, establishing the notation and scaling that remain in widespread use. What makes the Boussinesq system particularly suitable as a parametric testbed for this work is that changing the nonlinearity and dispersion coefficients continuously deforms the solution from a wave-packet to a solitary wave-like, giving rise to a more interesting range of behaviors within a single mathematical framework.

The specific one-dimensional form of the system used in this work, with its parameterization of the nonlinearity coefficient α and the dispersion coefficient β, was adopted from Martins (2023) and Martins et al. (2024), which is:

$$\begin{cases} \eta_t + u_x + \alpha\,(\eta u)_x = 0, \\ u_t - \dfrac{\beta}{3}\,u_{xxt} + \eta_x + \alpha\,u u_x = 0. \end{cases} \tag{1.1}$$

For this work, the parameters α and β in (1.1) are held equal, defining a one-parameter family of problems whose difficulty grows with the parameter. Namely, for very small $\alpha = \beta$ the wave dynamics can be approximated by the linear second order wave equation, while for large $\alpha = \beta$ the strong coupling between nonlinearity and dispersion produces richer dynamics that are fundamentally harder to learn.

Classical numerical methods for the Boussinesq system rely on pseudospectral and finite-difference discretizations that exploit the periodic or nearly periodic spatial structure of the waves. Pseudospectral (TREFETHEN, 2000) schemes achieve spectral accuracy on the spatial derivatives and are therefore well suited to the dispersive character of the equations (STEINMOELLER et al., 2012); this is the approach used to generate the reference dataset in this work. On the application side, operational coastal-engineering codes such as FUNWAVE (NASCIMENTO et al., 2010) implement Boussinesq-type solvers for harbor and nearshore wave propagation, demonstrating the practical relevance of the model beyond the academic setting.

SciML methods have recently been applied only to the Boussinesq equation, not to the specific Boussinesq system. Liu et al. (2023) applied parameter-integrated PINNs with phase-domain decomposition to reconstruct two-dimensional Boussinesq dynamics including phase transitions and rogue-wave formation, identifying spectral bias as a key obstacle at high nonlinearity. Wang et al. (2024) trained a physics-informed network to infer velocity and pressure fields from surface-elevation data alone on a Boussinesq model, demonstrating the inverse-problem capabilities of the approach. Complementary studies on related nonlinear wave equations (ZHANG et al., 2024; KUMAR et al., 2025) have confirmed that purely data-driven and purely physics-driven networks each display characteristic failure modes at strong nonlinearity. Taken together, these results suggest that a systematic, parametric evaluation of FNO/PINO/PINN on the 1D Boussinesq sys-

tem, which is the focus of this work, can be fruitful to understand better the interactions between the methods' ease of training and nonlinearities and dispersiveness.

1.3 Research Goals

This work implements and evaluates four SciML architectures on the parametric 1D Boussinesq system, arranged into two families of three trained methods each. The neural networks family represents a single parameter: a plain multilayer perceptron (MLP) trained on data alone, and a Physics-Informed Neural Network (PINN) trained both with and without supervised data. The operator family learns the whole parameter range in a single training and answers a new parameter with one forward pass: a Fourier Neural Operator (FNO) trained on data alone, and a Physics-Informed Neural Operator (PINO) trained both with and without supervised data. Crossing the two families with the three training regimes, pure data, data and physics, and pure physics, gives the six methods compared throughout. Beyond the comparison among these six methods mentioned previously in Section 1.1, we also investigate the spectral bias for the simplest possible MLP. This is done by fitting a single Boussinesq wave elevation profile at a fixed time, across a range of network widths, whose Neural Tangent Kernel (NTK) is measured directly to isolate the spectral bias mechanism. All results are measured against a pseudospectral reference solver, which sets a high-accuracy baseline. The study is guided by the following questions.

- Accuracy relative to the pseudospectral baseline: how close can the two methods with and without supervised data reproduce the reference solutions as nonlinearity and dispersion grow? Where does each method start to fail, and how does its error scale with the difficulty of the regime? Now, for only operator methods (FNO and PINO), does the discretization invariance their spectral layers promise survive a query off the training grid?

- Data-driven, physics-informed, and hybrid training: how does the presence or absence of supervised reference data affect the accuracy and training behavior of each method? Does adding a physics residual term improve generalization, or does it introduce new difficulties?

- Spectral bias across training regimes: how does spectral bias manifest in each architecture when the error spectrum is tracked band by band during training, and to what extent does the inclusion of data or physics constraints alter the frequency at which errors concentrate?

- The mechanism behind spectral bias: does Neural Tangent Kernel (NTK) theory

genuinely predict the frequency imbalance seen on the Boussinesq system, or only describe it after the fact? We put the theory to two quantitative tests on the Spectral Bias MLP, fitting a single Boussinesq elevation profile at a fixed time, across a range of network widths. The first targets the initial-kernel (lazy-training) assumption the analysis depends on: we measure how far the network parameters and the empirical kernel drift over training, and whether the kernel's eigenvalue spectrum is essentially unchanged from initialization to final iteration. The second targets the predicted driver of the bias: modes carrying the high-frequency detail are learned far more slowly than low-frequency ones. Testing both, where they hold and where they break, ties the frequency-band errors seen during training to a concrete mechanism instead of a loose analogy.

Together, these questions locate where each family and each training regime holds up on dispersive, nonlinear wave problems and where it fails, and what that implies for choosing among them.

1.4 Work Structure

This work is written with the goal of introducing SciML to senior undergraduates in mathematics, physics, or engineering who aim to apply these methods in their future research, presenting its pros and cons. We assume the reader has good knowledge of scientific computing and linear algebra. The remaining chapters are organized as follows.

- Chapter 2 develops the mathematical background. It covers the Boussinesq system and its analytical properties, the discrete Fourier transform and the pseudospectral solver built on it, and then the theoretical foundations of the two families, the MLP and the PINN on one side and the FNO and the PINO on the other. It closes with the Neural Tangent Kernel and a derivation of the spectral bias mechanism that the later chapters put to the test.

- Chapter 3 describes the experimental methodology. It details the computational setup, the pseudospectral dataset that serves as both training target and ground truth, the error metrics that score every method on one scale, the six trained methods on the two-family by three-regime grid, and the minimal experiment that isolates the spectral bias mechanism.

- Chapter 4 presents the numerical results. It accounts for what each method costs to train, then takes them along one axis at a time: the nonlinearity tests, the multi-resolution tests, and the spectrum compared mode by mode and closes with the

NTK measurements and the frequency-band tracking that tie the observed errors to spectral bias for all methods.

- Chapter 5 assesses each family against the questions posed above, states what the study cannot claim, and outlines directions for future work.

For the sake of methodological transparency and alignment with modern research practices, it is hereby noted that Artificial Intelligence-based language models, specifically Claude, by Anthropic, were employed throughout the development of this work. Such tools acted exclusively as technological aids, being used in the review and refinement of the academic text, in the optimization and debugging of computational programming routines, and in the adaptation of images, with authorship, analytical rigor, and scientific direction remaining strictly under the responsibility of the author

2 MATHEMATICAL BACKGROUND

This chapter introduces the mathematical foundations of the methods studied in this work. Section 2.1 presents the Boussinesq system and the limiting wave-equation regime. Section 2.2 develops the pseudospectral numerical method used to generate reference solutions. Sections 2.3 and 2.4 introduce, respectively, the neural network and neural operator architectures studied in this work.

2.1 The Boussinesq System

The Boussinesq system is a weakly nonlinear, weakly dispersive model governing the coupled evolution of two scalar fields in shallow water: the free surface elevation $\eta(x,t)$ and the depth-averaged horizontal velocity $u(x,t)$, both depending on position x and time t. In the one-dimensional, spatially periodic setting studied in this work, the system reads

$$\eta_t + u_x + \alpha\,(\eta u)_x = 0, \tag{2.1a}$$

$$u_t - \frac{\beta}{3}\,u_{xxt} + \eta_x + \alpha\,u u_x = 0, \tag{2.1b}$$

on the spatial domain $x \in [-L, L]$, where $\alpha \geq 0$ is the nonlinearity parameter, $\beta > 0$ is the dispersion parameter, and $L > 0$ is the domain half-length. The two coefficients weigh the competing effects the model balances: α scales the nonlinear terms $(\eta u)_x$ and uu_x, which steepen the wave, while β scales the dispersive term u_{xxt}, which spreads it. This nondimensional form follows equation (3.1) of Martins (2023) (see also Martins et al. (2024)).

The system (2.1) is solved throughout this work from a single solitary profile at rest,

$$\eta(x,0) = A\,\operatorname{sech}^2(x), \qquad u(x,0) = 0, \tag{2.2}$$

where $A > 0$ is the initial amplitude and $\operatorname{sech}(z) = 2/(e^z + e^{-z})$ is the hyperbolic secant. This places one crest at the center $x = 0$ of an otherwise flat surface, with zero initial velocity, as drawn in Figure 2.1.

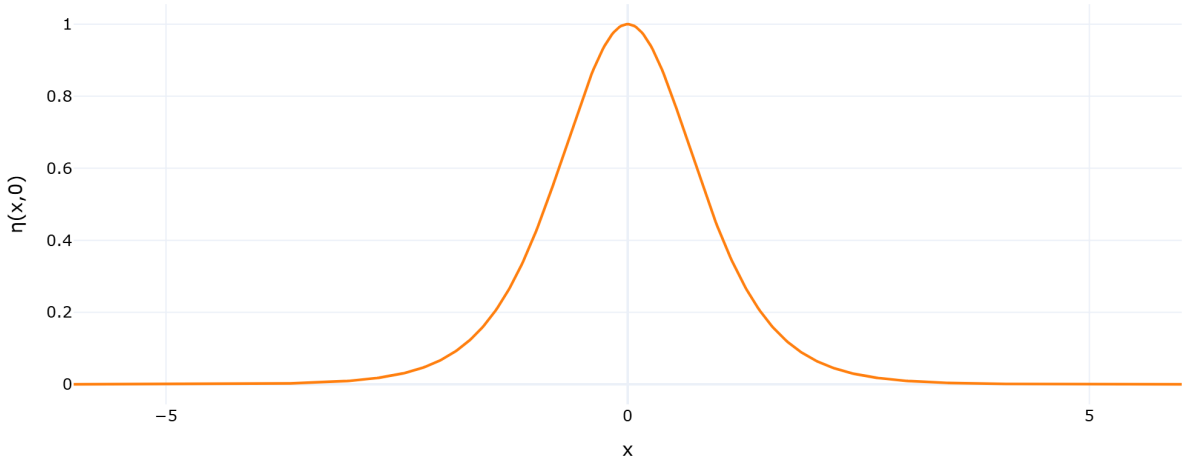


Figure 2.1 – Initial condition $\eta(x,0) = A\,\mathrm{sech}^2(x)$ for $A = 1$.

Source: The author (2026).

2.1.1 Recovering the Wave Equation

When both coefficients vanish, $\alpha = \beta = 0$, the nonlinear and the dispersive terms disappear together and the Boussinesq system reduces to:

$$\eta_t + u_x = 0, \tag{2.3}$$

$$u_t + \eta_x = 0. \tag{2.4}$$

The two fields decouple into a single scalar equation. Differentiating (2.3) with respect to t gives

$$\eta_{tt} + u_{xt} = 0, \tag{2.5}$$

and differentiating (2.4) with respect to x gives

$$u_{tx} + \eta_{xx} = 0. \tag{2.6}$$

Since $u_{xt} = u_{tx}$, subtracting (2.6) from (2.5) cancels the mixed derivative and leaves

$$\eta_{tt} - \eta_{xx} = 0,$$

that is, the wave equation for η,

$$\eta_{tt} = \eta_{xx}. \tag{2.7}$$

In an analogous manner, differentiating (2.4) with respect to t and (2.3) with respect to x produces the same cancellation, leaving $u_{tt} - u_{xx} = 0$, so u obeys the same equation.

D'Alembert's formula (DEBNATH, 2012) provides the general solution to (2.7).

Given initial data $\eta(x,0) = f(x)$ and $\eta_t(x,0) = g(x)$, the solution is

$$\eta(x,t) = \frac{f(x+t) + f(x-t)}{2} + \frac{1}{2}\int_{x-t}^{x+t} g(s)\,ds. \tag{2.8}$$

We now apply this to the initial condition (2.2). From (2.3), $\eta_t(x,0) = -u_x(x,0) = 0$, so $f(x) = A\,\mathrm{sech}^2(x)$ and $g \equiv 0$. The integral in (2.8) vanishes and the surface elevation evolves as

$$\eta(x,t) = \frac{A}{2}\big[\mathrm{sech}^2(x+t) + \mathrm{sech}^2(x-t)\big]. \tag{2.9}$$

The initial bump of amplitude A therefore splits into two counter-propagating pulses of amplitude $A/2$, traveling at speed ± 1 without changing shape.

For u, from (2.4) we have $u_t(x,0) = -\eta_x(x,0)$, so $f \equiv 0$ and $g(x) = -\eta_x(x,0)$. The f term vanishes in (2.8), leaving

$$u(x,t) = -\frac{1}{2}\int_{x-t}^{x+t} \eta_x(s,0)\,ds = -\frac{1}{2}\big[\eta(x+t,0) - \eta(x-t,0)\big],$$

and substituting $\eta(x,0) = A\,\mathrm{sech}^2(x)$,

$$u(x,t) = \frac{A}{2}\big[\mathrm{sech}^2(x-t) - \mathrm{sech}^2(x+t)\big]. \tag{2.10}$$

Any deviation from this wave-equation baseline in the full system (2.1) is attributable either to dispersion ($\beta > 0$) or to nonlinearity ($\alpha > 0$).

2.2 Pseudospectral Method

The pseudospectral method is applied here to the Boussinesq system (2.1) on the periodic spatial domain $\Omega = [-L, L]$. It transforms the two fields in space alone and leaves time as a separate evolution direction, which is the treatment an evolution equation asks for: taking the DFT along x turns each spatial derivative into an algebraic multiplication, so the PDE collapses into a system of ordinary differential equations in time, one for every spatial frequency.

2.2.1 Spatial Discretization and the Discrete Fourier Transform

The spatial domain $\Omega = [-L, L]$ of total length $2L$ is discretized into N_x equispaced points:

$$x_j = -L + j\,\Delta x, \qquad \Delta x = \frac{2L}{N_x}, \qquad j = 0, \ldots, N_x - 1.$$

The discrete Fourier transform (DFT) of a function f sampled at these points is (TREFETHEN, 2000):

$$\hat{f}(k) = \sum_{j=0}^{N_x-1} f(x_j)\, e^{-2\pi i k j/N_x}, \qquad k = -\tfrac{N_x}{2}, \ldots, \tfrac{N_x}{2} - 1, \tag{2.11}$$

with its inverse, the Inverse Discrete Fourier Transform (IDFT), recovering the physical values:

$$f(x_j) = \frac{1}{N_x} \sum_{k} \hat{f}(k)\, e^{2\pi i k j/N_x}. \tag{2.12}$$

The fundamental identity of the DFT with respect to differentiation is that, for a periodic function, the spatial derivative transforms into a pointwise multiplication (TREFETHEN, 2000; STEINMOELLER et al., 2012):

$$\widehat{(f_x)}(k) = i\frac{\pi k}{L}\,\hat{f}(k), \quad \widehat{(f_{xx})}(k) = -\frac{\pi^2 k^2}{L^2}\,\hat{f}(k).$$

This is in contrast to finite-difference methods, where the derivative carries a truncation error that shrinks with the grid spacing.

We define the wavenumber of mode k on $[-L, L]$ as:

$$\xi_k \;:=\; \frac{\pi k}{L}. \tag{2.13}$$

With this definition, the derivative identities read:

$$\widehat{(f_x)}(k) = i\,\xi_k\,\hat{f}(k), \qquad \widehat{(f_{xx})}(k) = -\xi_k^2\,\hat{f}(k). \tag{2.14}$$

2.2.2 Spectral Formulation of the Boussinesq System

To bring the Boussinesq system (2.1) into the Fourier domain, following the spectral formulation of Martins (2023), we apply the DFT to each equation and use the derivative identities (2.14). Equation (2.1a), treating $\widehat{(\eta u)}(k,t)$ as a given nonlinear term, becomes:

$$\hat{\eta}_t(k,t) = -i\,\xi_k\,\hat{u}(k,t) - \alpha\, i\,\xi_k\,\widehat{(\eta u)}(k,t). \tag{2.15}$$

Equation (2.1b) requires more care because of the u_{xxt} term. Since spatial derivatives commute with the DFT:

$$\widehat{(u_{xxt})}(k,t) = -\xi_k^2\,\hat{u}_t(k,t).$$

Substituting into the DFT of (2.1b), with its nonlinear term written in the equivalent form $uu_x = \frac{1}{2}(u^2)_x$:

$$\hat{u}_t(k,t) + \frac{\beta}{3}\,\xi_k^2\,\hat{u}_t(k,t) + i\,\xi_k\,\hat{\eta}(k,t) + \alpha\,i\,\xi_k\widehat{\left(\tfrac{1}{2}u^2\right)}(k,t) = 0.$$

Collecting the $\hat{u}_t(k,t)$ terms on the left and solving algebraically:

$$\hat{u}_t(k,t) = \frac{-i\,\xi_k\,\hat{\eta}(k,t) - \alpha\,i\,\xi_k\widehat{\left(\tfrac{1}{2}u^2\right)}(k,t)}{1 + \dfrac{\beta\,\xi_k^2}{3}}. \tag{2.16}$$

The division in (2.16) is where the transform pays off. Isolating $\hat{u}_t$ means inverting the operator $1 - \frac{\beta}{3}\frac{\partial^2}{\partial x^2}$, which on the grid couples neighboring nodes; under the DFT it is diagonal, acting on mode k as multiplication by $1 + \frac{\beta\,\xi_k^2}{3}$, so the inversion reduces to one division per mode.

Equations (2.15) and (2.16) can be written jointly as a first-order ODE system in time for each fixed wavenumber $k \in \mathbb{Z}$:

$$\begin{cases} \hat{\eta}_t(k,t) = -i\,\xi_k\,\hat{u}(k,t) - \alpha\,i\,\xi_k\,\widehat{(\eta u)}(k,t), \\[2ex] \hat{u}_t(k,t) = \dfrac{-i\,\xi_k\,\hat{\eta}(k,t) - \alpha\,i\,\xi_k\widehat{\left(\tfrac{1}{2}u^2\right)}(k,t)}{1 + \dfrac{\beta\,\xi_k^2}{3}}. \end{cases} \tag{2.17}$$

The PDE system (2.1), two equations in (x,t), is thus equivalent to an infinite family of two-dimensional ODE systems (2.17), one for each $k \in \mathbb{Z}$, coupled through the nonlinear terms.

The nonlinear terms $\widehat{(\eta u)}(k,t)$ and $\widehat{(\frac{1}{2}u^2)}(k,t)$ cannot be computed directly in Fourier space without a convolution sum, which costs $O(N_x^2)$. The pseudospectral approach avoids this by computing products in physical space. For $\widehat{(\eta u)}(k,t)$, the procedure is:

1. recover $\eta(x_j)$ and $u(x_j)$ by applying the inverse DFT (2.12);
2. compute the product $(\eta u)_j = \eta(x_j) \cdot u(x_j)$ pointwise on the grid;
3. apply the DFT (2.11) to obtain $\widehat{(\eta u)}(k,t)$.

Similarly, $\widehat{(\frac{1}{2}u^2)}(k,t)$ is obtained by the same three-step procedure with $u^2(x_j) = u(x_j)^2$, where $u(x_j) = \mathrm{IDFT}(\hat{u})(x_j)$; and $(\eta u)(x_j) = \mathrm{IDFT}(\hat{\eta})(x_j) \cdot \mathrm{IDFT}(\hat{u})(x_j)$. Since each DFT/IDFT costs $O(N_x \log N_x)$ via the Fast Fourier Transform (FFT), this pseudospectral strategy assembles each nonlinear term in $O(N_x \log N_x)$ instead of $O(N_x^2)$.

The nonlinear term of (2.1b) fits this procedure only in the form it was given in (2.16). Since $uu_x = \frac{1}{2}(u^2)_x$, the two forms are equivalent, but only the second is a product of a field with itself: recovering u on the grid, squaring it pointwise and transforming back costs one inverse and one forward transform, whereas uu_x would require recovering u and u_x separately before multiplying them. Both forms appear in this work. The solver uses $\frac{1}{2}(u^2)_x$ for exactly this reason, while the physics-informed network of Section 2.3.5 differentiates uu_x directly by automatic differentiation, where the products are never assembled on a grid and the distinction carries no cost.

2.2.3 Time Evolution via Runge-Kutta 4

Collecting the two spectral fields into a single state vector at each wavenumber k,

$$\hat{Y}(k,t) = \big(\hat{\eta}(k,t),\, \hat{u}(k,t)\big),$$

the system (2.17) takes the standard form of an initial value problem for a system of ordinary differential equations,

$$\hat{Y}_t(k,t) = F\big(\hat{Y}(k,t)\big), \tag{2.18}$$

where the pseudospectral right-hand side operator F is the map

$$F\Big(\hat{Y}(k,t)\Big) = F(\hat{\eta},\hat{u})\,(k,t) \;=\; \begin{pmatrix} -i\,\xi_k\,\hat{u}(k,t) - \alpha\, i\,\xi_k\,\widehat{(\eta u)}(k,t) \\ \dfrac{-i\,\xi_k\,\hat{\eta}(k,t) - \alpha\, i\,\xi_k\big(\widehat{\frac{1}{2}u^2}\big)(k,t)}{1+\dfrac{\beta\,\xi_k^2}{3}} \end{pmatrix},$$

whose two components are $\hat{\eta}_t$ and $\hat{u}_t$ as given by equations (2.15)–(2.16), with the nonlinear terms assembled by the pseudospectral procedure above.

In the form (2.18) the problem is what any explicit one-step integrator expects, and time evolution uses the classical fourth-order Runge-Kutta (RK4) scheme (SÜLI et al., 2003; BURDEN et al., 2011). Discretizing (2.18) in time, write $\hat{Y}_n = (\hat{\eta}^n, \hat{u}^n)$ for the spectral state at time t_n and fixed wavenumber k. The four stage evaluations $\kappa_1, \kappa_2, \kappa_3, \kappa_4$

and the update to $t_{n+1} = t_n + \Delta t$ are:

$$\begin{aligned}
\kappa_1 &= F(\hat{Y}_n), \\
\kappa_2 &= F\left(\hat{Y}_n + \tfrac{\Delta t}{2}\,\kappa_1\right), \\
\kappa_3 &= F\left(\hat{Y}_n + \tfrac{\Delta t}{2}\,\kappa_2\right), \\
\kappa_4 &= F\left(\hat{Y}_n + \Delta t\,\kappa_3\right), \\
\hat{Y}_{n+1} &= \hat{Y}_n + \frac{\Delta t}{6}\,(\kappa_1 + 2\,\kappa_2 + 2\,\kappa_3 + \kappa_4).
\end{aligned}$$

At each stage, the intermediate state is decoded to physical space to assemble the nonlinear products, then encoded back to Fourier space to evaluate the spectral derivatives. This combination of physical-space products, Fourier-space derivatives, and time evolution by a separate marching scheme is what characterizes a pseudospectral method.

2.2.4 Training Dataset

The numerical experiments in this work vary only the PDE parameters (α, β) while keeping the initial condition fixed at (2.2). The methods are therefore trained and evaluated on the dataset

$$\mathcal{S} = \left\{\big((\alpha_i, \beta_i),\, (\eta_i, u_i)\big)\right\}_{i=1}^{M}, \tag{2.19}$$

where (α_i, β_i) are the nonlinearity and dispersion parameters for the i-th sample, and (η_i, u_i) is the corresponding space-time solution obtained from the pseudospectral solver of Section 2.2 with those parameters and the shared initial condition (2.2).

2.3 Neural Networks

This section introduces the neural network architectures used in this work, together with the Neural Tangent Kernel theory that describes how such networks train. We begin with the Multilayer Perceptron, the building block underlying every architecture studied here. We then develop the Neural Tangent Kernel, the tool we use to analyze the spectral bias a network produces when it is fitted to data sampled on a regular grid. Finally we turn to Physics-Informed Neural Networks, where the same kernel analysis carries over and spectral bias is a well-documented obstacle.

2.3.1 Multilayer Perceptrons

In this work, we focus on Multilayer Perceptrons (MLPs), popularized by Rumelhart et al. (1986). These networks are compositions of affine transformations parametrized

by tunable weights and biases, separated by nonlinear activations, where an optimization process fits the network to a dataset.

Given a labeled dataset $\{(x_i, u_i)\}_{i=1}^{N}$, the objective is to fit the function $u_\theta(x)$ by minimizing a loss function, here the mean squared error (MSE). With $\|\cdot\|$ denoting the Euclidean norm, the unconstrained optimization problem is:

$$\mathcal{L}_{\text{MLP}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \|u_\theta(x_i) - u_i\|^2. \tag{2.20}$$

The MLP $u_\theta : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$ is defined by the composition:

$$\begin{aligned} h^{(0)} &= x \in \mathbb{R}^{d_{\text{in}}}, \\ h^{(l)} &= \sigma\left(W^{(l)} h^{(l-1)} + b^{(l)}\right) \in \mathbb{R}^{N_{\text{MLP}}}, \quad l = 1, \ldots, L_{\text{MLP}} - 1, \\ u_\theta(x) &= W^{(L_{\text{MLP}})} h^{(L_{\text{MLP}}-1)} + b^{(L_{\text{MLP}})} \in \mathbb{R}^{d_{\text{out}}}. \end{aligned}$$

where:

- $L_{\text{MLP}} \in \mathbb{N}$ is the number of layers and $N_{\text{MLP}} \in \mathbb{N}$ the number of neurons per hidden layer;
- $W^{(l)} \in \mathbb{R}^{N_{\text{MLP}} \times N_{\text{MLP}}}$ and $b^{(l)} \in \mathbb{R}^{N_{\text{MLP}}}$ are the weight matrix and bias vector at layer l, with $W^{(1)} \in \mathbb{R}^{d_{\text{in}} \times N_{\text{MLP}}}$ and $W^{(L_{\text{MLP}})} \in \mathbb{R}^{N_{\text{MLP}} \times d_{\text{out}}}$ adapted at the boundaries;
- $\sigma : \mathbb{R} \to \mathbb{R}$ is a sigmoidal activation function applied elementwise, satisfying $\sigma(t) \to 1$ as $t \to +\infty$ and $\sigma(t) \to 0$ as $t \to -\infty$;
- $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^{L_{\text{MLP}}}$ denotes all trainable parameters.

If the activation function is sigmoidal, MLPs are universal approximators: they can approximate any continuous function on a compact domain to arbitrary precision (CYBENKO, 1989).

In Figure 2.2, we show a common depiction of the neural network. Here the input vector is the pair $(x, t) \in \mathbb{R}^2$ and the output is $(\eta_\theta(x, t), u_\theta(x, t)) \in \mathbb{R}^2$.

2.3.2 Gradient Flow

The analysis of the following sections is carried out on the continuous-time limit of gradient descent, the form in which the neural-tangent-kernel results this work relies on are stated (JACOT et al., 2018; WANG; YU, et al., 2022). This subsection derives that limit.

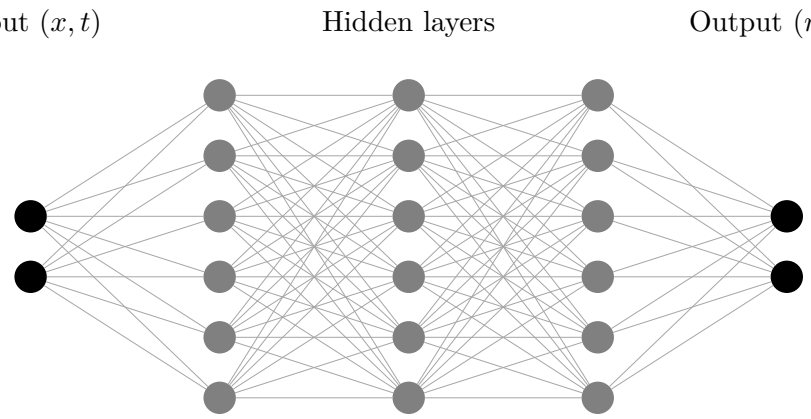


Figure 2.2 – Example of MLP architecture of input (x, t), with $L_{\text{MLP}} = 3$ hidden layers and $N_{\text{MLP}} = 6$ neurons per layer.

Source: The author (2026).

To understand how an MLP minimizes its data-fitting loss (2.20), consider gradient descent with learning rate $\mu > 0$. Starting from an initialization $\theta^{(0)}$, each iteration updates the parameters by

$$\theta^{(k+1)} = \theta^{(k)} - \mu \nabla_\theta \mathcal{L}(\theta^{(k)}), \qquad k = 0, 1, 2, \ldots, \tag{2.21}$$

where $\theta^{(k)}$ denotes the parameter vector after k steps. Rearranging (2.21) and dividing by the step size μ isolates the per-step increment as a difference quotient:

$$\frac{\theta^{(k+1)} - \theta^{(k)}}{\mu} = -\nabla_\theta \mathcal{L}(\theta^{(k)}). \tag{2.22}$$

The left-hand side of (2.22) already has the shape of a derivative: a difference of two parameter vectors divided by a spacing. It is not one yet. The index k counts iterations and carries no unit of time, so there is nothing for the difference to be taken with respect to. Reading (2.22) as the discretization of an ordinary differential equation therefore requires a new, continuous parameter along which the iterates can be laid out.

The idea is to regard the discrete sequence $\{\theta^{(k)}\}_{k=0}^{\infty}$ as samples of a smooth curve $\theta(\tau)$, read at a set of instants $\tau^{(k)}$,

$$\theta^{(k)} = \theta(\tau^{(k)}),$$

so that the recursion becomes a statement about $\theta(\tau)$ rather than about the index k. What remains is to fix the instants. The step size μ supplies their spacing: we assign to

the k-th iterate the training-time stamp

$$\tau^{(k)} = k\mu, \qquad \text{so that} \qquad \tau^{(k+1)} - \tau^{(k)} = \mu, \tag{2.23}$$

which places successive iterates μ apart along a time axis τ and gives $\theta^{(k+1)} = \theta(\tau^{(k)} + \mu)$. Substituting into the left-hand side of (2.22) recasts it as a forward finite-difference quotient of θ in τ:

$$\frac{\theta(\tau^{(k)} + \mu) - \theta(\tau^{(k)})}{\mu} = -\nabla_\theta \mathcal{L}\big(\theta(\tau^{(k)})\big). \tag{2.24}$$

As the step size $\mu \to 0$, the spacing in (2.23) shrinks to zero, the discrete stamps $\tau^{(k)}$ fill out a continuous variable τ, and the left-hand side of (2.24) converges to the derivative $d\theta/d\tau$ by definition of the derivative. The recursion thus becomes the gradient flow ODE:

$$\frac{d\theta}{d\tau} = -\nabla_\theta \mathcal{L}(\theta), \qquad \theta(0) = \theta^{(0)}. \tag{2.25}$$

The gradient flow (2.25) is a central object in optimization, and many standard first-order methods can be recovered as different time-discretization schemes of this same continuous trajectory. The discrete update (2.21) is precisely the forward (explicit) Euler discretization of (2.25) with step size μ (SÜLI et al., 2003; BURDEN et al., 2011), so gradient descent is just one numerical integrator for the flow; applying instead the backward (implicit) Euler scheme (SÜLI et al., 2003) recovers the proximal gradient method (PARIKH et al., 2013). Reading optimizers as discretizations of (2.25) thus organizes them into a single family, and it is the continuous flow that we analyze in what follows.

2.3.3 Neural Tangent Kernel

The kernel derived in this subsection was introduced by Jacot et al. (2018) and carried to the physics-informed setting by Wang, Yu, et al. (2022); the derivation below follows their treatment, specialized to the data-fitting loss (2.20). Consider the MLP fit to the dataset $\{(x_i, u_i)\}_{i=1}^N$ of Section 2.3.1. Group the pointwise fitting errors into a single error vector:

$$e(\theta) = \begin{bmatrix} u_\theta(x_1) - u_1 \\ \vdots \\ u_\theta(x_N) - u_N \end{bmatrix} \in \mathbb{R}^N. \tag{2.26}$$

Up to the constant factor $1/N$, the loss (2.20) is the squared norm of this vector, which we use in the form

$$\mathcal{L}(\theta) = \frac{1}{2}\,\|e(\theta)\|^2 = \frac{1}{2}\sum_{i=1}^{N} e_i(\theta)^2, \tag{2.27}$$

where $e_i(\theta) = u_\theta(x_i) - u_i$ is the i-th component of the error vector (2.26). Define the Jacobian of the error vector with respect to the parameters:

$$J(\theta) = \frac{\partial e}{\partial \theta} \in \mathbb{R}^{N \times N_\theta}, \qquad J_{ij} = \frac{\partial e_i}{\partial \theta_j}, \quad i = 1, \dots, N,\ j = 1, \dots, N_\theta,$$

where N_θ is the total number of trainable parameters. Each row $J_i = (J_{i1}, \dots, J_{iN_\theta}) \in \mathbb{R}^{N_\theta}$ is the parameter-gradient of the i-th residual. Since $\mathcal{L}$ is the quadratic form (2.27), the j-th component of the loss gradient is:

$$\frac{\partial \mathcal{L}}{\partial \theta_j} = \sum_{i=1}^{N} e_i(\theta)\, \frac{\partial e_i}{\partial \theta_j} = \sum_{i=1}^{N} J_{ij}\, e_i(\theta) = \big(J(\theta)^T e(\theta)\big)_j.$$

Collecting all $j = 1, \dots, N_\theta$ into a column vector:

$$\nabla_\theta \mathcal{L} = J(\theta)^T\, e(\theta) \in \mathbb{R}^{N_\theta}. \tag{2.28}$$

Substituting (2.28) into the gradient flow (2.25):

$$\frac{d\theta}{d\tau} = -J(\theta)^T\, e(\tau). \tag{2.29}$$

The error is written here as a function of τ rather than of θ. The two notations denote the same object: along the flow the parameters are themselves a function of training time, so $e(\tau) := e(\theta(\tau))$, and it is this composition whose evolution we now track. To find how the error vector itself evolves, differentiate each component $e_i(\theta(\tau))$ with respect to τ by the chain rule:

$$\frac{de_i}{d\tau} = \sum_{j=1}^{N_\theta} \frac{\partial e_i}{\partial \theta_j}\, \frac{d\theta_j}{d\tau} = J_i(\theta) \cdot \frac{d\theta}{d\tau}.$$

Stacking this identity for all $i = 1, \dots, N$ into a matrix equation:

$$\frac{de}{d\tau} = J(\theta)\, \frac{d\theta}{d\tau}.$$

Substituting (2.29):

$$\frac{de}{d\tau} = J(\theta)\big(-J(\theta)^T e(\tau)\big) = -\underbrace{J(\theta)\, J(\theta)^T}_{K(\theta) \in \mathbb{R}^{N \times N}}\, e(\tau). \tag{2.30}$$

The matrix $K(\theta) = J(\theta)J(\theta)^T$ is the empirical Neural Tangent Kernel (NTK) (JACOT et al., 2018). It is symmetric positive semi-definite by construction: for any $v \in \mathbb{R}^N$,

$$v^T K(\theta)\, v = v^T J(\theta)\, J(\theta)^T v = \left\| J(\theta)^T v \right\|^2 \geq 0.$$

The (i, j) entry of $K(\theta)$ follows directly from expanding the matrix product $J(\theta)J(\theta)^T$:

$$K_{ij}(\theta) = \sum_{l=1}^{N_\theta} J_{il}(\theta)\, J_{jl}(\theta) = \sum_{l=1}^{N_\theta} \frac{\partial e_i}{\partial \theta_l}(\theta)\, \frac{\partial e_j}{\partial \theta_l}(\theta) = \nabla_\theta e_i(\theta)^T\, \nabla_\theta e_j(\theta).$$

This is the inner product in $\mathbb{R}^{N_\theta}$ of the parameter gradients of the network output at the i-th and j-th training points, where $e_i = u_\theta(x_i) - u_i$. Hence $K(\theta) \in \mathbb{R}^{N \times N}$ is the Gram matrix of the network's sensitivities across the whole dataset, and equation (2.30) shows that it alone governs how the fitting error evolves under gradient flow.

2.3.4 Spectral Bias

The spectral bias (or frequency principle) is the empirical observation that neural networks trained by gradient-based methods reduce the low-frequency components of the error faster than the high-frequency ones (RAHAMAN et al., 2019). This section derives that phenomenon directly from the NTK dynamics established above. When the training points lie on a regular grid and the target function is periodic, the error admits a discrete Fourier representation, and one can ask how its frequency content is approximated along the training iterations. We first establish the decay in the eigenbasis of the NTK and then, by transforming to Fourier space, show that it is precisely the low frequencies that are resolved first.

For a network with a sufficiently large number of parameters, the Jacobian $J(\theta)$ changes negligibly during training, a regime known as lazy training (JACOT et al., 2018; CHIZAT et al., 2019; WANG; YU, et al., 2022). The motivation for it is a scaling argument. In the wide-network limit the output at any point is an aggregate of many hidden units, each contributing a vanishing share of it as the width grows. An $O(1)$ change in the function is then reached by many parameters each moving very little, instead of a few moving a lot. Since the Jacobian is itself a function of the parameters, it barely moves either, and in the infinite-width limit it stays frozen at its value at initialization (JACOT et al., 2018; CHIZAT et al., 2019). The networks trained here are finite, so this is an approximation, not an identity, and how far it holds is measured directly in Chapter 4. Writing $J_0 = J(\theta(0))$ and $K_0 = J_0 J_0^T$, the approximation $K(\theta(\tau)) \approx K_0$ turns

equation (2.30) into a linear ODE with constant coefficients:

$$\frac{de}{d\tau} = -K_0\, e(\tau), \qquad e(0) = e_0. \tag{2.31}$$

Written component-wise, the i-th row of this system is

$$\frac{de_i}{d\tau} = -\sum_{j=1}^{N} K_0^{ij}\, e_j(\tau), \qquad i = 1, \ldots, N,$$

where $K_0^{ij} = \nabla_{\theta_0} e_i^T \nabla_{\theta_0} e_j$ is the inner product, at initialization, of the parameter gradients of the network output at training points x_i and x_j. The rate of change of the error at point i is therefore a weighted sum of all current errors, with weights given by these pairwise kernel values. Through this coupling, the geometry of the sample grid and the architecture of the network jointly determine the convergence of every component.

Since $K_0 \in \mathbb{R}^{N\times N}$ is symmetric and positive semi-definite, the spectral theorem for symmetric matrices (STRANG, 2006; GOLUB et al., 2013) guarantees a decomposition

$$K_0 = V\, \Lambda\, V^T, \tag{2.32}$$

where $V = [v_1 \mid \cdots \mid v_N]$ is an orthogonal matrix ($V^T V = I$) and

$$\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_N), \qquad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_N \geq 0.$$

Introduce the change of coordinates

$$c(\tau) = V^T e(\tau), \tag{2.33}$$

so that, since V is orthogonal, $e(\tau) = V c(\tau)$. Differentiating (2.33) with respect to τ and substituting the ODE (2.31):

$$\frac{dc}{d\tau} = V^T \frac{de}{d\tau} = -V^T K_0\, e(\tau).$$

Inserting $K_0 = V\Lambda V^T$ and $e(\tau) = Vc(\tau)$:

$$\frac{dc}{d\tau} = -V^T\, (V\Lambda V^T)\, (Vc(\tau)) = -(V^T V)\, \Lambda\, (V^T V)\, c(\tau) = -\Lambda\, c(\tau),$$

using $V^T V = I$ at each step. Since Λ is diagonal, this decouples into N independent scalar equations:

$$\frac{dc_k}{d\tau} = -\lambda_k\, c_k(\tau), \qquad k = 1, \ldots, N. \tag{2.34}$$

Each equation in (2.34) is a first-order linear ODE with constant coefficient (BOYCE et al., 2012). Separating the variables and integrating both sides from the initial state at $\tau = 0$ up to time τ, with s and r as integration variables for the amplitude and the time,

$$\int_{c_k(0)}^{c_k(\tau)} \frac{ds}{s} = -\lambda_k \int_0^{\tau} dr \qquad \Longrightarrow \qquad \ln \frac{|c_k(\tau)|}{|c_k(0)|} = -\lambda_k \, \tau.$$

Exponentiating both sides yields the solution

$$c_k(\tau) = c_k(0)\, e^{-\lambda_k \tau}, \tag{2.35}$$

where $c_k(0) = v_k^T e_0$ is the projection of the initial error e_0 onto the k-th eigenvector of K_0. For $\lambda_k = 0$ the equation gives $c_k(\tau) = c_k(0)$ for all τ: that eigendirection never decays.

Returning to the original coordinates via $e(\tau) = Vc(\tau)$:

$$e(\tau) = \sum_{k=1}^{N} \big(v_k^T e_0\big)\, e^{-\lambda_k \tau}\, v_k. \tag{2.36}$$

The error decomposes into N independent eigendirections. Each eigenvector v_k carries initial amplitude $v_k^T e_0$ and decays exponentially at rate λ_k: eigenvectors with large λ_k are suppressed quickly, while those with small λ_k persist throughout training. This per-eigendirection decay, each at the rate set by its eigenvalue, is the rigorous statement of spectral bias established by Cao et al. (2021); the same spectral ordering governs generalization as the training set grows (BORDELON et al., 2020).

Equation (2.36) is the continuous flow; gradient descent takes finite steps, and its error evolution inherits the same eigenstructure in discrete form. The link between the two is the training-time stamp of (2.23): iterate n sits at continuous time $\tau^{(n)} = n\mu$, so the iteration count and the training time are read off one another through

$$\tau = n\mu, \qquad n = \tau/\mu.$$

Section 2.3.2 read the update (2.21) as forward Euler applied to the gradient flow in the parameters. The same statement is needed for the error, since that is what the analysis follows. There are two ways to obtain it: discretize the parameter update and then linearize, or linearize first, into the error equation (2.31), and discretize that. Both give the same law.

Take the first. Gradient descent on the loss is

$$\theta^{(n+1)} = \theta^{(n)} - \mu\, \nabla_\theta \mathcal{L}\big(\theta^{(n)}\big).$$

Under lazy training the Jacobian is frozen at J_0, so (2.28) gives $\nabla_\theta \mathcal{L}(\theta^{(n)}) = J_0^T e^{(n)}$ and the step is

$$\Delta\theta^{(n)} := \theta^{(n+1)} - \theta^{(n)} = -\mu\, J_0^T e^{(n)}.$$

Expanding the error to first order in that displacement,

$$e^{(n+1)} = e^{(n)} + J_0\, \Delta\theta^{(n)} + O\big(\|\Delta\theta^{(n)}\|^2\big),$$

and substituting the step,

$$e^{(n+1)} = e^{(n)} - \mu\, J_0 J_0^T e^{(n)}.$$

Now the second. Equation (2.31) is already linear in e. The forward, or explicit, Euler scheme (SÜLI et al., 2003; LEVEQUE, 2007) replaces its derivative at $\tau^{(n)}$ by a forward difference over one step of length μ,

$$\left.\frac{de}{d\tau}\right|_{\tau^{(n)}} \approx \frac{e^{(n+1)} - e^{(n)}}{\mu},$$

and evaluates the right-hand side at the current iterate,

$$\frac{e^{(n+1)} - e^{(n)}}{\mu} = -K_0\, e^{(n)}.$$

Multiplying through by μ,

$$e^{(n+1)} = e^{(n)} - \mu\, K_0\, e^{(n)}.$$

Since $K_0 = J_0 J_0^T$, the two results are the same equation. Collecting terms gives the single-step law

$$e^{(n+1)} = (I - \mu K_0)\, e^{(n)}. \tag{2.37}$$

One gradient-descent step of size μ is one forward-Euler step of length μ on the error. Iterating from $e^{(0)} = e_0$ and diagonalizing with $K_0 = V \Lambda V^T$,

$$e^{(n)} = (I - \mu K_0)^n e_0 = \sum_{k=1}^{N} \big(v_k^T e_0\big)\, (1 - \mu\lambda_k)^n\, v_k. \tag{2.38}$$

The continuous solution reaches the same expression, one mode at a time. Evaluate the decay factor of (2.36) at $\tau = n\mu$ and split it across the n steps,

$$e^{-\lambda_k \tau} = e^{-\lambda_k n \mu} = \big(e^{-\lambda_k \mu}\big)^n,$$

then expand the per-step factor and truncate at first order in $\mu\lambda_k$ (BURDEN et al., 2011),

$$e^{-\lambda_k \mu} = 1 - \mu\lambda_k + O\big((\mu\lambda_k)^2\big) \approx 1 - \mu\lambda_k. \tag{2.39}$$

Substituting term by term carries (2.36) into (2.38), so $(1 - \mu\lambda_k)^n$ is the forward-Euler counterpart of $e^{-\lambda_k \tau}$. The two agree as the step vanishes with $\tau = n\mu$ held fixed,

$$\lim_{\mu \to 0} \big(1 - \mu\lambda_k\big)^{\tau/\mu} = e^{-\lambda_k \tau}.$$

The same truncation constrains the step. Under the continuous flow every mode with $\lambda_k > 0$ decays for any $\mu > 0$. The discrete mode contracts only when its amplification factor is smaller than one in magnitude,

$$\big|1 - \mu\lambda_k\big| < 1 \qquad \Longleftrightarrow \qquad 0 < \mu\lambda_k < 2,$$

which is the absolute-stability condition of the explicit Euler method (LEVEQUE, 2007; SÜLI et al., 2003). The product $\mu\lambda_k$ alone fixes how the k-th mode behaves, and it does so in four regimes. Below one, the factor lies in $(0, 1)$ and the mode decays monotonically toward zero; at exactly one it vanishes and the mode is removed in a single step. Between one and two the factor is negative, so the mode still decays but flips sign at every step. Past two it exceeds one in magnitude and the mode grows without bound. A single step size therefore acts differently on every eigendirection of K_0.

Since $\lambda_1 = \lambda_{\max}$ is the largest eigenvalue, every mode stays stable only when the largest one does, which caps the learning rate at

$$\mu < \frac{2}{\lambda_{\max}}.$$

The minimal experiment of Section 3.5 sets its step from this limit: rather than a nominal value, μ is fixed at a constant fraction of $2/\lambda_{\max}$ at every width, small enough to keep every mode in the monotone regime $\mu\lambda_k < 1$ while staying comparable across widths whose $\lambda_{\max}$ differ.

The same argument gives the time each mode needs. From (2.35), the amplitude of the k-th eigendirection at time τ is

$$\big|c_k(\tau)\big| = \big|v_k^T e_0\big|\, e^{-\lambda_k \tau}.$$

Require it to fall below a target precision $\varepsilon > 0$,

$$\big|v_k^T e_0\big|\, e^{-\lambda_k \tau} < \varepsilon,$$

and divide by $\left|v_k^T e_0\right| > 0$,

$$e^{-\lambda_k \tau} < \frac{\varepsilon}{\left|v_k^T e_0\right|}.$$

Taking logarithms preserves the inequality,

$$-\lambda_k \, \tau < \ln \frac{\varepsilon}{\left|v_k^T e_0\right|},$$

and dividing by $-\lambda_k < 0$ reverses it,

$$\tau > \frac{1}{\lambda_k} \, \ln \frac{\left|v_k^T e_0\right|}{\varepsilon}.$$

The minimum training time to resolve the k-th eigendirection to precision ε is therefore

$$\tau_k = \frac{1}{\lambda_k} \, \ln \frac{\left|v_k^T e_0\right|}{\varepsilon}. \tag{2.40}$$

The projection enters (2.40) through a logarithm and the eigenvalue through $1/\lambda_k$, so λ_k sets the scale of τ_k while the projection only shifts it. Since $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_N \geq 0$, the times are ordered $\tau_1 \leq \tau_2 \leq \cdots \leq \tau_N$: higher-indexed eigendirections require more training time, in inverse proportion to their eigenvalue. Arora et al. (2019) reached similar conclusions from a discrete-time analysis of gradient descent, showing that the convergence speed of each eigenvector is governed by its eigenvalue and by the projection of the target onto that eigenvector.

The decomposition (2.36) is expressed in the eigenbasis $\{v_k\}_{k=1}^N$ of the NTK, which need not coincide with any physically meaningful basis. When the N training points lie on a regular grid and the target is periodic, however, the natural basis is the Fourier one, and recasting the dynamics there shows that the decay is genuinely organized by frequency. Let $F \in \mathbb{C}^{N \times N}$ be the Fourier matrix, that is, the discrete Fourier transform (DFT) matrix with entries

$$F_{jl} = \frac{1}{\sqrt{N}} \, \omega^{jl}, \qquad \omega = e^{-2\pi \mathrm{i}/N}, \qquad j, l = 0, \ldots, N-1.$$

With this normalization F is unitary, $F^* F = I$, so that $F^{-1} = F^*$, where F^* is the conjugate transpose (GOLUB et al., 2013; STRANG, 2006). Applying F to the error vector yields its spectrum, with inverse transform F^{-1}:

$$\hat{e}(\tau) = F \, e(\tau), \qquad e(\tau) = F^{-1} \hat{e}(\tau) = F^* \hat{e}(\tau). \tag{2.41}$$

In particular $\hat{e}_0 = F e_0$ is the spectrum of the initial error, the difference between the

spectra of the initial network output and the target. Since F is constant in τ, applying it to the linearized dynamics (2.31) carries the error evolution into the frequency domain:

$$\frac{d\hat{e}}{d\tau} = F\,\frac{de}{d\tau} = -FK_0\,e(\tau) = -FK_0F^{-1}\,\hat{e}(\tau).$$

Inserting the eigendecomposition $K_0 = V\Lambda V^T$ from (2.32),

$$FK_0F^{-1} = FV\,\Lambda\,V^TF^{-1} = (FV)\,\Lambda\,(FV)^* = (FV)\,\Lambda\,(FV)^{-1},$$

where $V^TF^{-1} = V^*F^* = (FV)^*$ uses that V is real orthogonal, and $(FV)^* = (FV)^{-1}$ uses that a product of unitary matrices is unitary. The columns of FV, namely the Fourier transforms $\{Fv_k\}_{k=1}^N$ of the NTK eigenvectors, therefore diagonalize the frequency-domain dynamics. Defining the modal amplitudes $\tilde{c}(\tau) = (FV)^{-1}\hat{e}(\tau)$ and repeating the steps (2.33)–(2.35) gives

$$\frac{d\tilde{c}_k}{d\tau} = -\lambda_k\,\tilde{c}_k(\tau) \qquad \Longrightarrow \qquad \tilde{c}_k(\tau) = \tilde{c}_k(0)\,e^{-\lambda_k\tau}. \tag{2.42}$$

These are the same amplitudes as before: $\tilde{c} = (FV)^{-1}Fe = V^{-1}e = V^Te = c$, the projection of the error onto the NTK eigenvectors, now read in Fourier space. Transforming back through (2.41) yields the evolution of the error spectrum,

$$\hat{e}(\tau) = \sum_{k=1}^{N}\big[(Fv_k)^*\hat{e}_0\big]\,e^{-\lambda_k\tau}\,(Fv_k). \tag{2.43}$$

Equation (2.43) is the spectral bias statement in the literal sense of the frequency spectrum: the error spectrum is a superposition of the transformed eigenvectors Fv_k, each damped at its own rate λ_k. The frequencies resolved first are those carried by the eigenvectors with the largest eigenvalues; empirically these eigenvectors are smooth and concentrated in the low Fourier modes (RAHAMAN et al., 2019; XU et al., 2020), so the low-frequency content of the error decays first while the high-frequency content persists.

To close, we specialize the initial condition. The weights of an MLP are drawn at random from a distribution centered at zero, so at small initialization scale $\theta_0 \approx 0$ and the network output u_{θ_0} is close to zero across the whole grid. Set against a target u of order one, it is negligible. This is a common simplification in spectral bias analyses, and

under it the initial error is the target with opposite sign, as is its spectrum:

$$\hat{e}_0 = F\big(u_{\theta_0} - u\big) = \hat{u}_{\theta_0} - \hat{u} \approx -\,\hat{u} = -\begin{bmatrix} \hat{u}(0) \\ \vdots \\ \hat{u}(N-1) \end{bmatrix}. \tag{2.44}$$

Substituting (2.44) into (2.43) casts the dynamics as a reconstruction of the target frequency by frequency: each Fourier mode of u is acquired at a rate set by the NTK eigenvalues, the smooth low-frequency content emerging first and the fine high-frequency detail last. This is the regime anticipated at the start of the section, a periodic target on a regular grid, in which the quality of the approximation can be tracked mode by mode along the training iterations. It is also the setting we implement: the minimal experiment of Chapter 3 fits a single Boussinesq elevation profile on exactly such a grid, so that (2.44) holds by construction and the mode-by-mode reconstruction can be measured against the prediction.

This last observation can be made quantitative. Taking the modulus of the modal amplitude (2.42) isolates the magnitude of the k-th component along training,

$$|\tilde{c}_k(\tau)| = |\tilde{c}_k(0)|\, e^{-\lambda_k \tau}, \qquad |\tilde{c}_k(0)| = |(Fv_k)^* \hat{e}_0| \approx |(Fv_k)^* \hat{u}|, \tag{2.45}$$

where the last estimate uses the small-scale initialization (2.44), so that $|(Fv_k)^*\hat{u}|$ measures how strongly the target overlaps with the k-th eigenmode. Training, however, advances in discrete steps: by the construction of the gradient flow (2.23), the continuous time and the iteration count n are related by $\tau = n\mu$, with μ the learning rate. Substituting $\tau = n\mu$ into (2.45) and requiring the amplitude to fall below a target precision $\varepsilon > 0$,

$$|(Fv_k)^*\hat{u}|\, e^{-\lambda_k n\mu} < \varepsilon \implies n > \frac{1}{\mu\,\lambda_k}\,\ln\frac{|(Fv_k)^*\hat{u}|}{\varepsilon},$$

gives an estimate of the number of gradient-descent iterations needed to resolve the k-th frequency:

$$n_k = \left\lceil \frac{1}{\mu\,\lambda_k}\,\ln\frac{|(Fv_k)^*\hat{u}|}{\varepsilon} \right\rceil. \tag{2.46}$$

The count follows just as directly from the discrete recursion, without passing through the continuous flow. The discrete counterpart of (2.45) is the modal amplitude read off (2.38), $|\tilde{c}_k^{(n)}| = |\tilde{c}_k(0)|\,|1-\mu\lambda_k|^n$, and requiring it below ε gives

$$|\tilde{c}_k(0)|\,|1-\mu\lambda_k|^n < \varepsilon \implies n > \frac{1}{-\ln|1-\mu\lambda_k|}\,\ln\frac{|\tilde{c}_k(0)|}{\varepsilon}, \tag{2.47}$$

the inequality flipping because in the stable regime $|1-\mu\lambda_k| < 1$, so $\ln|1-\mu\lambda_k| < 0$.

This exact discrete count reduces to the continuous estimate (2.46): by the first-order truncation (2.39), $-\ln|1-\mu\lambda_k| = \mu\lambda_k + O\big((\mu\lambda_k)^2\big)$, so the denominator collapses to $\mu\lambda_k$ and, with $|\tilde{c}_k(0)| \approx |(Fv_k)^*\hat{u}|$, equation (2.47) becomes (2.46). Substituting $\tau = n\mu$ into the continuous decay and counting steps directly on the discrete recursion therefore agree to leading order in the step, exactly as the forward-Euler link between (2.36) and (2.38) requires.

This estimate is tested numerically in this work. The minimal experiment of Chapter 3 records, for each eigendirection, the iteration at which the measured amplitude first falls below ε, and sets it against the count predicted by (2.47). The discrete form is the one put to the test, since it rests on the same geometric law $|1-\mu\lambda_k|^n$ as the measured decay; the continuous estimate (2.46) would add the truncation of the exponential to whatever discrepancy the measurement shows. Because (2.47) carries no reference to the width of the network, it predicts that networks of different widths fall on a single curve, which Chapter 4 checks at three widths.

2.3.5 Physics-Informed Neural Networks

Introduced by Raissi et al. (2019), Physics-Informed Neural Networks (PINNs) insert the PDE directly into the optimization process, forcing the neural network to satisfy the model equations and approximating the PDE solution.

Consider the general PDE initial value problem:

$$\begin{cases} \mathcal{D}[u](x,t) = 0, \quad \forall (x,t) \in \Omega \times (0,T], \\ \quad u(x,0) = u_0(x), \quad x \in \Omega, \end{cases} \tag{2.48}$$

where $\mathcal{D}$ is a differential operator on u, $\Omega \times (0,T]$ is the spatio-temporal domain, and u_0 is the initial condition.

The PDE residual loss penalizes violation of the governing equation at collocation points $\{(x_i,t_i)\}_{i=1}^{N_\mathcal{D}} \subset \Omega \times (0,T]$:

$$\mathcal{L}_{\text{pde}}(\theta) = \frac{1}{N_\mathcal{D}} \sum_{i=1}^{N_\mathcal{D}} \left\| \mathcal{D}[u_\theta](x_i,t_i) \right\|^2 .$$

The initial condition is enforced at collocation points $\{x_i\}_{i=1}^{N_0} \subset \Omega$:

$$\mathcal{L}_{\text{ic}}(\theta) = \frac{1}{N_0} \sum_{i=1}^{N_0} \left\| u_\theta(x_i,0) - u_0(x_i) \right\|^2 .$$

The combined PINN loss is:

$$\mathcal{L}_{\text{PINN}}(\theta) = \mathcal{L}_{\text{pde}}(\theta) + \mathcal{L}_{\text{ic}}(\theta). \tag{2.49}$$

Due to the compositional structure of MLPs, PINN implementations evaluate PDE residuals via Automatic Differentiation (AD) (LINNAINMAA, 1970; PASZKE et al., 2019), using Adam (KINGMA et al., 2014) or L-BFGS (NOCEDAL, 1980) as the optimizer.

Application to the Boussinesq Dataset

For the Boussinesq system, a separate PINN is trained for each sample $(\alpha_i, \beta_i) \in \mathcal{S}$ (2.19). The differential operator $\mathcal{D}$ in the generic problem (2.48) is instantiated as the pair of Boussinesq residuals. Each acts on both predicted fields and carries the coefficients of the sample, so we write them with all four arguments displayed:

$$\begin{aligned}
\mathcal{D}_1[\eta_\theta, u_\theta, \alpha_i, \beta_i] &= (\eta_\theta)_t + (u_\theta)_x + \alpha_i\,(\eta_\theta\, u_\theta)_x, \\
\mathcal{D}_2[\eta_\theta, u_\theta, \alpha_i, \beta_i] &= (u_\theta)_t - \tfrac{\beta_i}{3}(u_\theta)_{xxt} + (\eta_\theta)_x + \alpha_i\, u_\theta (u_\theta)_x.
\end{aligned}$$

The combined loss (2.49) for sample i then reads:

$$\begin{aligned}
\mathcal{L}^i_{\text{PINN}}(\theta) = \frac{1}{N_{\mathcal{D}}} \sum_{l=1}^{N_{\mathcal{D}}} \Big[\mathcal{D}_1[\eta_\theta, u_\theta, \alpha_i, \beta_i](x_l, t_l)^2 + \mathcal{D}_2[\eta_\theta, u_\theta, \alpha_i, \beta_i](x_l, t_l)^2 \Big] \\
+ \frac{1}{N_0} \sum_{l=1}^{N_0} \Big[(\eta_\theta(x_l, 0) - \eta_0(x_l))^2 + (u_\theta(x_l, 0) - u_0(x_l))^2 \Big],
\end{aligned} \tag{2.50}$$

where η_0 and u_0 are given by the shared initial condition (2.2). Because the parameters (α_i, β_i) enter the residuals explicitly, each PINN encodes knowledge of a single sample and must be retrained for every new parameter pair.

2.3.6 Spectral Bias in Physics-Informed Neural Networks

The Neural Tangent Kernel analysis of Sections 2.3.3–2.3.4 was carried out for an MLP fitting data on a uniform grid, the setting in which the kernel is approximately circulant and its eigenvectors are genuine Fourier modes. A PINN does not fit data directly: its loss (2.49) combines an initial-condition term with a PDE-residual term. The same machinery nevertheless applies once both constraints are stacked into a single error vector, now indexed over the N_r residual points in place of the N data points of the

MLP,

$$e(\theta) = \begin{bmatrix} u_\theta(x_1, 0) - u_0(x_1) \\ \vdots \\ u_\theta(x_{N_0}, 0) - u_0(x_{N_0}) \\ \mathcal{D}[u_\theta](x_1, t_1) \\ \vdots \\ \mathcal{D}[u_\theta](x_{N_\mathcal{D}}, t_{N_\mathcal{D}}) \end{bmatrix} \in \mathbb{R}^{N_r}, \qquad N_r = N_0 + N_\mathcal{D}, \tag{2.51}$$

so that, up to normalization, $\mathcal{L}_{\text{PINN}}$ reduces to the same quadratic form $\frac{1}{2}\|e\|^2$ analyzed for the MLP. With this definition the gradient-flow derivation leading to (2.30) carries over verbatim, with the Jacobian now also containing the parameter gradients of the PDE residuals, and the error still obeys the linear dynamics $\frac{de}{d\tau} = -K_0\, e$, with convergence of each eigenvector governed by its eigenvalue exactly as in (2.40) (WANG; YU, et al., 2022).

This prediction is borne out empirically. Spectral bias is a frequently reported obstacle in PINN training: networks fit smooth, low-frequency solutions readily but stall on sharp gradients and high-frequency content (RAHAMAN et al., 2019; XU et al., 2020; WANG; TENG, et al., 2021), and the same pathology underlies several documented PINN failure modes (KRISHNAPRIYAN et al., 2021). Wang, Yu, et al. (2022) traced the effect directly to the NTK, showing that its spectrum is concentrated at low eigenvalues, so that high-frequency PDE residuals are the last to be minimized. For nonlinear wave equations of Boussinesq type, Wang et al. (2024) report the same difficulty in resolving the finer wave structure; the two-field Boussinesq system studied here has not been examined in this respect, which is part of what Chapter 4 sets out to do. A range of remedies targets this imbalance directly, among them Fourier feature mappings that recondition the kernel spectrum (TANCIK et al., 2020), adaptive loss weighting (WANG; TENG, et al., 2021), and second-order or spectrum-aware optimizers (KHODAKARAMI et al., 2026). Pursuing these mitigations is beyond the scope of this work: our aim is to apply the SciML methods to the Boussinesq system and to expose and understand the spectral bias that emerges there, not to remove it. Understanding the effect nonetheless remains essential background for the neural-operator methods studied in the remainder of this work, whose ability to capture the high-frequency content of the Boussinesq solutions is examined directly in Chapter 4.

2.4 Neural Operators

Neural operators generalize neural networks from learning finite-dimensional mappings to learning operators between infinite-dimensional function spaces. This section introduces the general framework, then the Fourier Neural Operator (FNO) as the principal

architecture, and finally the Physics-Informed Neural Operator (PINO) as the physics-constrained extension.

2.4.1 General Definition

Let $D \subset \mathbb{R}^d$ be a bounded open domain and let $\mathcal{A}(D;\mathbb{R}^{d_a})$ and $\mathcal{U}(D;\mathbb{R}^{d_u})$ be Banach spaces of functions (KREYSZIG, 1978). The target operator is a (potentially nonlinear) map

$$G^\dagger : \mathcal{A} \to \mathcal{U}, \qquad a \mapsto u = G^\dagger(a),$$

where both the input a and the output u are functions defined on D.

In this work, $G^\dagger$ is the solution operator of the Boussinesq system: given an input a encoding the PDE coefficients (α, β) together with the initial condition (2.2), the operator returns the full space-time solution (η, u). Because the initial condition is the same for every sample of the dataset (2.19), it carries nothing that distinguishes one sample from another, and the input reduces to the pair (α_i, β_i), which is how the operator is written from here on.

A neural operator G_θ approximates $G^\dagger$ by alternating linear layers with pointwise nonlinearities, in the composition

$$G_\theta = Q \circ \sigma\Big(W_L + \mathcal{K}_L\Big) \circ \cdots \circ \sigma\Big(W_1 + \mathcal{K}_1\Big) \circ P, \tag{2.52}$$

where:

- $P : \mathbb{R}^{d_a} \to \mathbb{R}^{d_c}$ is the lifting operator, mapping the input $a(x)$ pointwise to a higher-dimensional channel representation with $d_c \gg d_a$;
- $\sigma\Big(W_\ell + \mathcal{K}_\ell\Big)$, for $\ell = 1, \ldots, L$, are the operator layers, each combining a local linear map W_ℓ with a nonlocal integral operator $\mathcal{K}_\ell$, followed by a pointwise activation σ;
- $Q : \mathbb{R}^{d_c} \to \mathbb{R}^{d_u}$ is the projection operator mapping the final channel representation back to the output dimension.

What makes (2.52) an operator, rather than a map between finite-dimensional vectors, is the nonlocal term $\mathcal{K}_\ell$ in each layer. It is an integral operator,

$$(\mathcal{K}_\ell v)(x) = \int_D \kappa_\ell(x, y)\, v(y)\, dy,$$

where $\kappa_\ell : D \times D \to \mathbb{R}^{d_c \times d_c}$ is a learned kernel. Each output point therefore draws on the whole input function, and not only on its value at x, which is what the local map

W_ℓ alone would give. Different choices of kernel parametrization for each $\mathcal{K}_\ell$ give rise to different neural operator architectures.

2.4.2 Fourier Neural Operators

Presented by Li et al. (2021), the Fourier Neural Operator (FNO) is the neural operator (2.52) in which each $\mathcal{K}_\ell$ uses a shift-invariant kernel: $\kappa_\ell(x, y) = \kappa_\ell(x - y)$. This assumption turns each integral operator into a convolution, which by the convolution theorem (DEBNATH, 2012) can be evaluated efficiently in the frequency domain.

Each spectral layer updates the channel representation via:

$$v_{\ell+1}(x) = \sigma\Big((W_\ell\, v_\ell)(x) + (\mathcal{K}_\ell v_\ell)(x)\Big). \tag{2.53}$$

The operator $\mathcal{K}_\ell$ acts via convolution with the shift-invariant kernel κ_ℓ:

$$(\mathcal{K}_\ell v_\ell)(x) = \int_D \kappa_\ell(x - y)\, v_\ell(y)\, dy.$$

By the convolution theorem (DEBNATH, 2012), this equals:

$$(\mathcal{K}_\ell v_\ell)(x) = \mathcal{F}^{-1}\Big(\mathcal{F}(\kappa_\ell) \cdot \mathcal{F}(v_\ell)\Big)(x). \tag{2.54}$$

Property (2.54) is what makes the layer (2.53) affordable: the nonlocal integral never has to be formed in physical space, since a forward transform, a pointwise multiplication and an inverse transform deliver it. To see how this is parametrized, recall that for a periodic function f on $[0, 2\pi]$:

$$f(x) = \sum_{k\in\mathbb{Z}} \hat{f}(k)\, e^{ikx}, \qquad \hat{f}(k) = \frac{1}{2\pi}\int_0^{2\pi} f(x) e^{-ikx}\, dx.$$

The convolution $(\kappa * v)$ simplifies to:

$$(\kappa * v)(x) = \int_0^{2\pi} \kappa(x - y)\, v(y)\, dy = \sum_{k\in\mathbb{Z}} \big(2\pi\, \hat{\kappa}(k)\, \hat{v}(k)\big)\, e^{ikx}.$$

Instead of learning κ_ℓ as a function in physical space, the FNO directly parametrizes its Fourier coefficients as learnable complex-valued matrices $R_{\ell,k} \approx 2\pi\, \hat{\kappa}_\ell(k)$. Only the $|k| \le k_{\max}$ low-frequency modes are retained, with all others set to zero; $k_{\max}$ is a hyper-parameter.

Training Objective

The FNO is trained as a pure supervised learning problem. Given the dataset $\mathcal{S}$ (2.19), whose i-th element pairs the coefficients (α_i, β_i) with the reference solution (η_i, u_i) computed for them, the training objective minimizes the mean squared error between the predicted and reference space-time fields:

$$\mathcal{L}_{\mathrm{FNO}}(\theta) = \frac{1}{M}\sum_{i=1}^{M}\frac{1}{N_x N_t}\sum_{j=0}^{N_x-1}\sum_{n=0}^{N_t}\Big[(\eta_\theta^i(x_j,t_n)-\eta_i(x_j,t_n))^2 + (u_\theta^i(x_j,t_n)-u_i(x_j,t_n))^2\Big], \tag{2.55}$$

where G_θ is the neural operator (2.52) and $(\eta_\theta^i, u_\theta^i) \coloneqq G_\theta(\alpha_i, \beta_i)$ is its prediction for sample i. Minimizing (2.55) therefore drives $G_\theta(\alpha_i, \beta_i)$ towards (η_i, u_i) for all M samples of $\mathcal{S}$ at once, which is what lets a single trained operator answer the whole parameter family.

Application to Non-Time-Periodic Problems

When the solution is also assumed to be periodic in time with period T (so that $D = [0, 2\pi] \times [0, T]$), shift-invariance extends to the time direction as well: $\kappa_\ell(x, y, t, \tau) = \kappa_\ell(x - y,\, t - \tau)$. The convolution then generalizes to:

$$(\mathcal{K}_\ell v_\ell)(x,t) = \int_D \kappa_\ell(x-y,\, t-\tau)\, v_\ell(y,\tau)\, dy\, d\tau = \mathcal{F}^{-1}\Big(\mathcal{F}(\kappa_\ell)\cdot\mathcal{F}(v_\ell)\Big)(x,t),$$

and the FNO parametrizes the two-dimensional Fourier modes (k_x, k_t) with $|k_x| \le k_{x,\max}$ and $|k_t| \le k_{t,\max}$, both hyperparameters.

For problems that are not time-periodic, however, applying the Fourier transform along the temporal axis implicitly extends the signal periodically, potentially introducing a jump discontinuity at the temporal boundary. This can trigger the Gibbs phenomenon (GIBBS, 1899), producing oscillatory artifacts near the boundary that persist regardless of how many temporal modes are retained. Standard signal-processing techniques such as zero-padding and window functions (e.g. Hann and Hamming windows) can partially mitigate these effects, at the cost of altering the temporal structure of the signal. Li et al. (2021) acknowledge this limitation and provide an alternative formulation in which the Fourier layers operate only along the spatial axis while the operator advances forward in time, avoiding the periodicity assumption in the temporal direction entirely. In this work we deliberately retain the two-dimensional space-time formulation, in which the Fourier layers act on both the spatial and the temporal axes, so that a single operator emits the entire trajectory in one forward pass. This keeps the architecture simple and uniform across space and time, but it reintroduces the temporal-periodicity assumption for a Boussinesq solution that is not time-periodic, and therefore admits a Gibbs artifact

at the temporal boundary $t = T$.

2.4.3 Physics-Informed Neural Operators

Analogously to how PINNs add a physics-informed residual loss to the MLP training objective, the Physics-Informed Neural Operator (PINO) (LI et al., 2023a) incorporates PDE constraints into the FNO training. The FNO backbone remains unchanged; what differs is the loss function, which now combines a data term with residuals evaluated on the predicted space-time fields.

Unlike a PINN, which represents the solution as a single continuous function and differentiates it via automatic differentiation, the FNO outputs (η_θ, u_θ) as a discrete $N_x \times N_t$ array in a single forward pass over the full space-time domain. Evaluating the PDE residuals therefore reduces to numerically differentiating this output array.

Spatial derivatives are computed pseudospectrally using ξ_k (2.13): the DFT of η_θ along the spatial axis at each time t_n yields $\hat{\eta}_\theta(k, t_n)$, from which

$$(\eta_x)_j = \mathcal{F}^{-1}(i\,\xi_k\,\hat{\eta}_\theta(k,t_n))_j\,, \qquad (\eta_{xx})_j = \mathcal{F}^{-1}\big(-\xi_k^2\,\hat{\eta}_\theta(k,t_n)\big)_j\,,$$

and identically for u_θ. Time derivatives are computed by finite differences: second-order central in the interior and first-order one-sided at the boundaries,

$$(\eta_\theta)_t^n = \begin{cases} \dfrac{\eta_\theta^{n+1} - \eta_\theta^n}{\Delta t}, & n = 0, \\ \dfrac{\eta_\theta^{n+1} - \eta_\theta^{n-1}}{2\,\Delta t}, & 1 \le n \le N_t - 1, \\ \dfrac{\eta_\theta^n - \eta_\theta^{n-1}}{\Delta t}, & n = N_t, \end{cases}$$

and identically for $(u_\theta)_t^n$. The mixed term $(u_\theta)_{xxt}$ is obtained by first computing $(u_\theta)_{xx}$ spectrally and then applying the time-difference stencil to the result. This combination of spectral spatial derivatives and finite-difference temporal derivatives follows the reference implementation of Li et al. (2023a) (LI et al., 2023b).

With all derivatives available, the Boussinesq equations (2.1) define two differential operators acting on the predicted fields:

$$\mathcal{D}_1(\eta_\theta, u_\theta) \coloneqq (\eta_\theta)_t + (u_\theta)_x + \alpha\,(\eta_\theta\,u_\theta)_x, \tag{2.56}$$

$$\mathcal{D}_2(\eta_\theta, u_\theta) \coloneqq (u_\theta)_t - \tfrac{\beta}{3}(u_\theta)_{xxt} + (\eta_\theta)_x + \alpha\,(u_\theta(u_\theta)_x). \tag{2.57}$$

Here $(\eta_\theta^i, u_\theta^i) \coloneqq G_\theta(\alpha_i, \beta_i)$ denotes the network output for sample i, and the subscript $_{j,n}$ indicates evaluation at the grid point (x_j, t_n). The PINO loss is a weighted sum of three

terms:

$$\mathcal{L}_{\text{PINO}}(\theta) = \lambda_{\text{pde}}\,\mathcal{L}_{\text{pde}}(\theta) + \lambda_{\text{ic}}\,\mathcal{L}_{\text{ic}}(\theta) + \lambda_{\text{data}}\,\mathcal{L}_{\text{data}}(\theta), \tag{2.58}$$

where:

- $\mathcal{L}_{\text{pde}}(\theta) = \dfrac{1}{M}\displaystyle\sum_{i=1}^{M}\frac{1}{N_x N_t}\sum_{j=0}^{N_x-1}\sum_{n=0}^{N_t}\Big[\mathcal{D}_1(\eta_\theta^i, u_\theta^i)_{j,n}^2 + \mathcal{D}_2(\eta_\theta^i, u_\theta^i)_{j,n}^2\Big]$ penalizes violation of the Boussinesq equations at each grid point, with residuals evaluated using the parameters (α_i, β_i) of each sample;
- $\mathcal{L}_{\text{ic}}(\theta) = \dfrac{1}{M}\displaystyle\sum_{i=1}^{M}\frac{1}{N_x}\sum_{j=0}^{N_x-1}\Big[(\eta_\theta(x_j,0) - \eta_0(x_j))^2 + (u_\theta(x_j,0) - u_0(x_j))^2\Big]$ enforces the shared initial condition (2.2);
- $\mathcal{L}_{\text{data}}(\theta) = \dfrac{1}{M}\displaystyle\sum_{i=1}^{M}\frac{1}{N_x N_t}\sum_{j=0}^{N_x-1}\sum_{n=0}^{N_t}\Big[(\eta_\theta^i(x_j,t_n) - \eta_i(x_j,t_n))^2 + (u_\theta^i(x_j,t_n) - u_i(x_j,t_n))^2\Big]$ measures deviation from the reference solutions $(\eta_i, u_i) \in \mathcal{S}$ (2.19);
- λ_{pde}, λ_{ic}, $\lambda_{\text{data}} > 0$ are weighting hyperparameters.

The PINO therefore generalizes the FNO by adding physics supervision: it can be trained with fewer labeled samples by relying on the PDE residual for additional guidance.

3 METHODOLOGY

This chapter shows how the experiments were built, on what data, and under which rules, in enough detail to reproduce them. Three things are common to every experiment, and we settle them first: the computational setup (Section 3.1), the pseudospectral dataset (Section 3.2), and the error metrics that score every method on one scale (Section 3.3). Section 3.4 then defines the six trained methods. Section 3.5 sets up the experiment on spectral bias, which runs on the same Boussinesq solution and the same kind of network as the rest of the work, but with time and parameters held fixed, and Section 3.6 condenses the hyperparameters of this work in one table.

3.1 Computational Setup and Reproducibility

The software behind these experiments was all written for this work. It runs on Python 3.11 with PyTorch 2.12.0 (PASZKE et al., 2019) built against CUDA 13.0, on a single NVIDIA GeForce GTX 1660 SUPER GPU. The static figures were drawn with Plotly and the animations with Matplotlib. We used no pre-trained weights, no third-party operator-learning library, and no external dataset. The pseudospectral solver, the four architectures, and each optimization loop are our own code, which let us instrument them for evaluating and showing the spectral bias from Section 3.5 and the custom visualization used in upcoming chapters. The full implementation is available at `https://github.com/samuelkutz/TCC`.

Each learned architecture stays close to its original implementation. The Fourier Neural Operator follows Li et al. (2021); the Physics-Informed Neural Operator follows Li et al. (2023a), together with its released code (LI et al., 2023b); and the Physics-Informed Neural Network follows Raissi et al. (2019). All of them are written from scratch in PyTorch, and where an implementation differs from its reference we point that out.

Reproducibility rests on two conventions. A single global seed initializes Python, NumPy and PyTorch at the start of the pipeline and puts the cuDNN backend in deterministic mode. The stages then run in a fixed order, each inheriting the random state from its predecessors.

We now fix the experimental parameters shared by every method. Starting with

the physical model, the spatial and temporal domains of the Boussinesq system (2.1) are

$$x \in [-L, L], \quad L = 60, \qquad t \in [0, T], \quad T = 30. \tag{3.1}$$

Every experiment uses the initial condition (2.2) at unit amplitude $A = 1$, that is, $\eta(x, 0) = \operatorname{sech}^2(x)$ and $u(x, 0) = 0$. A unit crest sits at the center of a domain 120 units wide. That width is deliberate: the two counter-propagating pulses of the wave-equation limit (2.9)–(2.10) travel at unit speed, so over the horizon $T = 30$ they cover half the distance to the imposed periodic boundary and never reach it. Table 3.1 lists the constants that hold everywhere.[1]

Table 3.1 – Fixed simulation, dataset, and evaluation constants shared by every experiment. Unless a section says otherwise, these values hold throughout.

Parameter	**Value**
Domain and initial condition	
Spatial half-length L $(\Omega = [-L, L])$	60
Final time T $(t \in [0, T])$	30
Initial amplitude A	1
Initial condition	$\eta(x, 0) = \operatorname{sech}^2(x),\ u(x, 0) = 0$
Dataset solve	
Spatial grid points N_x	128
Time levels (127 RK4 steps)	128
Spatial spacing $\Delta x = 2L/N_x$	0,9375
Temporal spacing $\Delta t = T/127$	$\approx$ 0,2362
Parameter sampling	
Coupled coefficients	$\alpha = \beta$
Training samples M	20
Sampling grid	linspace(0,1; 4,0; 20)
Evaluation and reproducibility	
Evaluation values $\alpha = \beta$	{0,1; 3,21; 4,2}
Evaluation resolutions N_x	{128, 256, 512}
Spectral snapshot resolution	256
Global seed	37

Source: The author (2026).

3.2 The Parametric Dataset

The reference solutions are the supervised targets that the data-driven methods learn. This role asks for a high-fidelity baseline, and the pseudospectral Runge–Kutta solver of Section 2.2 supplies one since the domain is wide enough that the periodicity

[1] Some of these constants are from: Δx and Δt in (3.3), $\alpha_i = \beta_i$ in (3.2), $\alpha = \beta$ in (3.6), and N_x in (3.7). But these are seen throughout Chapter 2.

the solver assumes is not a problem.

We keep the parameter family one-dimensional on purpose. Following Section 2.2.4, the initial conditions are fixed and only the PDE coefficients vary, tied together as the pair $\{(\alpha_i = \beta_i), (\eta_i, u_i)\}$, sampled on the evenly spaced grid of $M = 20$ values

$$\alpha_i = \beta_i \in \text{linspace}(0{,}1;\ 4{,}0;\ 20), \qquad i = 1, \ldots, M. \tag{3.2}$$

For small values of the parameters, the dynamics stay close to linear. Each value in (3.2) is handed to the solver on $N_x = 128$ spatial points and marched through 127 Runge–Kutta steps over $[0, T]$, giving $N_t = 128$ time levels. The grid spacings are

$$\Delta x = \frac{2L}{N_x} = \frac{120}{128} = 0{,}9375, \qquad \Delta t = \frac{T}{127} = \frac{30}{127} \approx 0{,}2362. \tag{3.3}$$

A solve returns the two space-time solution matrices $\eta_i(x_j, t_n)$ and $u_i(x_j, t_n)$ on the $N_x \times N_t$ grid, and we collect them into the solution dataset $\mathcal{S}$ of Equation (2.19). Each sample is stored with four input channels and two output channels. The inputs are the initial conditions $\eta_i(x_j, 0)$ and $u_i(x_j, 0)$ at every spatial point x_j, together with the two coefficients α_i and β_i (summing four feature channels), each of the four held constant along time so that all of them fill the space-time grid:

$$X \in \mathbb{R}^{M \times 4 \times N_x \times N_t}, \qquad Y \in \mathbb{R}^{M \times 2 \times N_x \times N_t}. \tag{3.4}$$

The two output channels carry η_i and u_i themselves.

From now on the panels report only the surface elevation η, since it carries the wave structure under study and the velocity should be similar.

For this problem, we are interested in learning real functions η and u defined on $[-L, L] \times [0, T]$ and parametrized by α and β, which span $[0, 4]$ (for exact values, see Table 3.1). Following standard practice, we need a way to normalize the data. For pointwise networks the recommendation is to non-dimensionalize, scaling the variables so that they become dimensionless. For operators, it is common to implement a per-channel normalization by an affine map as a matter of course (KOSSAIFI et al., 2024; LI et al., 2021).

We use a single rule for all methods. Writing $[f_{\min}, f_{\max}]$ for the envelope of a scalar function f, every network input in this work is carried onto $[-1, 1]$ by

$$\tilde{f} = \frac{2\,(f - f_{\min})}{f_{\max} - f_{\min}} - 1, \qquad f = f_{\min} + \tfrac{1}{2}\,(\tilde{f} + 1)\,(f_{\max} - f_{\min}), \tag{3.5}$$

the second expression being the exact inverse of the first. The target $[-1, 1]$, not $[0, 1]$,

suits the hyperbolic tangent of the pointwise backbones, which is odd and centered at the origin.

Where each method uses it

The MLP applies (3.5) to its two coordinates and leaves its output in physical units. The PINN does the same, and applies it inside the forward pass; the residual is then differentiated with respect to the physical x and t; automatic differentiation carries the chain rule through the rescaling on its own, and what is enforced stays the residual of (2.1) and not of a rescaled equation. Applying the same map to both also leaves the two networks sharing an input scale as well as a backbone, so the comparison between them isolates the training signal.

The FNO and the PINO apply (3.5) to the four input channels and the two output channels of (3.4). To these four inputs both append two coordinate channels, x and t on the same $[-1, 1]$, so the lifting layer reads six channels instead of four; the spatial one follows the solver's periodic grid, whose right endpoint is dropped, so that the channel keeps a fixed physical meaning as the resolution changes. The two operators differ in the units each loss term is formed in. Both compare prediction against target in the normalized variables, which puts the pure-data and data-and-physics regimes on one objective scale. The PINO then returns its fields to physical units for the residual and initial-condition terms, so that the constraint it enforces is the balance of (2.1) at its true magnitudes. Every prediction is returned to physical units by the inverse in (3.5) before any reported metric is formed.

3.3 Error Metrics

A comparison is only fair if every method is measured against the same reference and on the same scales. Each method is scored against a freshly recomputed pseudospectral reference at three values of the parameter of (2.1),

$$\alpha = \beta \in \{0{,}1;\ 3{,}21;\ 4{,}2\}. \tag{3.6}$$

These three values span the sampling range: $\alpha = \beta = 0{,}1$ is a wave with weak nonlinearity and low dispersion, and is itself a sample of the training grid (3.2); $\alpha = \beta = 3{,}21$ is more nonlinear and falls between two training samples; $\alpha = \beta = 4{,}2$ is even more nonlinear still and lies past the upper end of the sampling range. The middle value 3,21 is the default wherever a single parameter is needed, as it is for the MLP and the two PINNs, which are trained at one parameter.

To test the discretization invariance that operators are supposed to enjoy, each

trained operator is queried without retraining on grids of resolution

$$N_x \in \{128,\ 256,\ 512\}, \tag{3.7}$$

the training resolution and its two- and fourfold refinements, all powers of two, the sizes on which the fast Fourier transform of Section 2.2.1 runs most efficiently (TREFETHEN, 2000). For each query the input channels of Section 3.2 are rebuilt on the finer grid from the same initial condition, and the reference is recomputed at the matching resolution, so prediction and reference are always compared at the same points. The pointwise methods are queried on the same grids, since a continuous map can be sampled anywhere.

Let η_{true} and η_{pred} be the reference and predicted elevation on a shared space-time grid. We report three complementary numbers.

Time-resolved relative error

At each time level t_n we take the spatial L^2 error and divide it by the norm of the reference at that instant,

$$E(t_n) = \frac{\big\|\eta_{\text{pred}}(\cdot, t_n) - \eta_{\text{true}}(\cdot, t_n)\big\|_2}{\big\|\eta_{\text{true}}(\cdot, t_n)\big\|_2}. \tag{3.8}$$

Absolute spectral error

To see which spatial frequencies are wrong, we take the spatial real DFT at a fixed time and compare amplitudes mode by mode. Writing $\hat{\eta}(k, t_n)$ for the DFT (2.11) of the elevation at wavenumber index k, the per-mode error is the absolute difference of amplitudes,

$$E_{\text{spec}}(k, t_n) = \frac{1}{N_x}\,\big|\,|\hat{\eta}_{\text{pred}}(k, t_n)| - |\hat{\eta}_{\text{true}}(k, t_n)|\,\big|. \tag{3.9}$$

The factor $1/N_x$ undoes the scaling the unnormalized transform (2.11) carries with the grid size, so the amplitudes, and with them the error, are read in the units of η and mean the same thing on each of the grids (3.7). Measuring the difference of amplitudes rather than their ratio is what keeps the metric honest across the spectrum.

Frequency-band error during training

To watch spectral bias form as the optimizer runs, we split the spatial spectrum into three bands and save each band error every 500 optimizer iterations. The per-mode quantity is exactly the absolute spectral error (3.9), and it is reduced to one number per band in two averages. First over the N_t time levels, giving a value for each mode, then

over the modes of the band,

$$\overline{E}_{\mathrm{spec}}(k) = \frac{1}{N_t}\sum_{n=1}^{N_t} E_{\mathrm{spec}}(k,t_n), \qquad E_{\mathrm{band}}(B) = \frac{1}{|B|}\sum_{k\in B}\overline{E}_{\mathrm{spec}}(k), \tag{3.10}$$

so the same metric drives the per-frequency spectra and the band-tracking curves. With $N_k = \lfloor N_x/2 \rfloor + 1$ retained real-DFT modes, and writing $q_1 = \lfloor N_k/4 \rfloor$ and $q_2 = \lfloor N_k/2 \rfloor$ for the two cut points, the three index sets B are

$$B_{\mathrm{low}} = \big\{\, k : 0 \le k < q_1 \,\big\}, \quad B_{\mathrm{mid}} = \big\{\, k : q_1 \le k < q_2 \,\big\}, \quad B_{\mathrm{high}} = \big\{\, k : q_2 \le k < N_k \,\big\}, \tag{3.11}$$

a quarter of the range each for the low and mid bands and the upper half for the high band. These curves feed the band-tracking comparison of Chapter 4. Every method is tracked at one common parameter, $\alpha = \beta = 3{,}21$: the pointwise MLP and PINN against their own training solution at that value, each operator against the dataset sample nearest it, so the bands are read at a single parameter across all six.

A second, simpler reduction of (3.9) averages over every retained mode at a fixed time instead of over a band across time, collapsing the spectrum to one curve against t,

$$\widetilde{E}_{\mathrm{spec}}(t_n) = \frac{1}{N_k}\sum_{k=0}^{N_k-1} E_{\mathrm{spec}}(k,t_n). \tag{3.12}$$

This is the curve the spectral panels of Chapter 4 plot beneath each pair of spectra, distinct from the band average (3.10) above, which reduces the two axes in the opposite order.

3.4 The Six Methods

Four architectures: MLP; PINN; FNO; and PINO, give rise to six trained methods: MLP and FNO (pure data); PINN and PINO (data and physics); and the PINN and PINO without data (pure physics), all described in Chapter 2.

We can organize the six experiments into three classes, divided for NNs and for NOs:

- Pure data fits the reference solutions with a supervised term alone and no equation. The data-only MLP and the FNO sit here;
- Data and physics keeps that supervised term and adds the Boussinesq residual beside it. The PINN and the PINO with data sit here, meaning they possess both data and physics-informed losses summed up;

- Pure physics switches the supervised term off, leaving the Boussinesq residual and the initial condition to drive the network. The PINN and the PINO without data sit here.

Every method is trained against the pseudospectral references of Section 3.2, on the grid of Table 3.1, and is scored by the same metrics of Section 3.3, so a difference in behavior traces to the family and the regime rather than to the setup. The operators read all $M = 20$ samples of (3.4) at once, while each pointwise method is tied to a single parameter and fits the one solve at $\alpha = \beta = 3{,}21$.

3.5 Spectral Bias Setting

In this section, we propose a numerical study to verify the plausibility of the spectral bias theory derived in Section 2.3.4 to confront estimates (2.40) and (2.47) with the actual training of a network.

To some degree every method in this work carries a version of the spectral bias derived in Section 2.3.4. We examine it on the simplest object this work can offer, and we keep it on the Boussinesq system with the Spectral Bias MLP, a network from $\mathbb{R}$ to $\mathbb{R}$ that learns the single elevation profile at a fixed time $t = 15$, taken from the same pseudospectral solver at the intermediate parameter $\alpha = \beta = 3{,}21$,

$$x \longmapsto \eta_\theta(x) \approx \eta(x,\, 15), \qquad x \in [-L, L]. \tag{3.13}$$

This creates the setting the theory of Section 2.3.4 was derived for, a target sampled on a regular periodic grid and fitted by gradient descent.

3.5.1 Setting Up the Experiment

We train three Spectral Bias MLPs that share one architecture and differ only in width, $N_{\mathrm{MLP}} \in \{8, 64, 512\}$, each with four hidden layers and hyperbolic-tangent activations, one input and one output. The normalization map (3.5) carries x onto $[-1, 1]$ as it does for the rest of the work, and every figure in this section undoes that scaling to show η in physical units.

The target lives on a single grid of $N_x = 128$ points, so the Jacobian $J \in \mathbb{R}^{N_x \times N_\theta}$ is cheap to compute in full, one backward pass per grid point, at every width including the widest. It sets the empirical kernel

$$K_0 = J_0 J_0^{\mathsf{T}} = V \Lambda V^{\mathsf{T}},$$

the empirical Neural Tangent Kernel of Jacot et al. (2018), only 128×128 and cheap

enough to eigendecompose once and to rebuild at later iterations, which is how Section 3.5.2 measures its drift.

Training is full-batch gradient descent on the sum-of-squares loss $\frac{1}{2}\|e\|^2$,

$$\theta \leftarrow \theta - \mu\, J^{\mathsf{T}} e, \qquad e = \eta_\theta - \eta_{\text{true}},$$

the same rule behind (2.37), with the error measured on the grid. Each width takes the same fraction of its own stability bound,

$$\mu = 0{,}2 \cdot \frac{2}{\lambda_{\max}} = \frac{0{,}4}{\lambda_{\max}}, \qquad \mu\lambda_{\max} = 0{,}4, \tag{3.14}$$

with $\lambda_{\max}$ the largest eigenvalue of K_0 at that width, the bound Section 2.3.4 derives. Because $\lambda_{\max}$ grows with width, the step size itself differs across the three networks, but the product $\mu\lambda_{\max}$ stays fixed, so every network sits at the same point inside the monotone regime $\mu\lambda_k < 1$. Scaling μ to each width's own bound this way keeps the three networks comparable, and we run every network for 500.000 iterations.

The MLP is nonlinear in its parameters, so J drifts away from J_0 as training proceeds, and the initial-kernel prediction (2.38) becomes an approximation once training starts. Whether it survives contact with a real network is what the four measurements below test, each one a quantity measured from the running network and set against the value the frozen kernel K_0 predicts.

Per-eigendirection error decay

Projecting the measured error onto each eigenvector v_k of K_0 gives the empirical coefficient

$$c_k^{(n)} = v_k^{\mathsf{T}} e^{(n)}, \tag{3.15}$$

tracked against the amplitude $|c_k(0)|\,|1-\mu\lambda_k|^n$ that (2.38) predicts.

Per-mode spectral decay

The real DFT of the error profile at each iteration gives one amplitude per wavenumber, read against the prediction of (2.43). Four fixed wavenumbers are tracked across the whole run, giving a direct, per-frequency reading of the panel.

Band-averaged absolute spectral error

The per-mode error (3.9) is averaged over the whole spectrum within each of the three bands of (3.11). A single time level is measured here, so the time average of (3.10)

does not apply; both the measured and the predicted fields are scored against the reference profile, on the same metric the six trained methods are tracked with in Chapter 4.

Per-mode iteration count

For each eigendirection, we record the iteration n_k at which the measured coefficient first drops below a precision ε,

$$n_k = \frac{\ln\big(|c_k(0)|/\varepsilon\big)}{-\ln|1-\mu\lambda_k|}, \tag{3.16}$$

the discrete crossing time (2.47) predicts, with the precision set as a fraction of the largest initial coefficient,

$$\varepsilon = 0{,}05 \max_k |c_k(0)|,$$

so the threshold scales with whatever amplitude η happens to carry.

The first three checks are drawn at the widest network, since lazy training is an assumption about wide networks, and the fourth collects all three widths.

3.5.2 Kernel Drift and the Eigenvalue Spectrum

The predictions above assume the kernel stays put, and we measure whether it does along the same three runs from Section 3.5.1. We log the relative parameter drift $\|\theta-\theta_0\|/\|\theta_0\|$ and the relative kernel drift $\|K-K_0\|/\|K_0\|$ at logarithmically spaced iterations, and we record the eigenvalue spectrum of K at initialization and again at the end of training.

The two drift curves ask whether the kernel stays put, as lazy training assumes, and whether it does so more convincingly as the network widens, as the initial-kernel approximation (2.31) predicts. The spectrum should expose the eigenvalue disparity that (2.40) names as the cause of spectral bias. Together with the four checks of Section 3.5.1, these curves open the results of Chapter 4, where they anchor the reading of the spectral bias curves measured on the full architectures.

3.6 Configuration Summary

Table 3.2 gathers the configuration of every experiment. The four learned architectures share a common budget, 15.000 iterations of Adam at learning rate 10^{-3}, so the comparison of Chapter 4 reflects the architecture and the training regime, not the effort spent on tuning. The last row reports the measured training wall-clock on the hardware of Section 3.1, which is indicative and depends on the GPU used.

Summing the weights and one bias per output unit across the input layer, the $L_{\mathrm{MLP}} - 2$ hidden-to-hidden layers, and the output layer of Section 2.3.1 gives the total number of trainable parameters in the MLP and the PINN. For the operators of Section 2.4.2, the lifting and projection operators contribute additional parameters.

Table 3.2 – Configuration of the four training architectures and of the Spectral Bias MLP. The FNO and PINO share the operator backbone and its parameter count; the MLP and PINN share the feed-forward backbone and are single-parameter methods. All Adam-trained methods use 15.000 iterations. The PINN draws a fresh set of 5.000 interior collocation points at every iteration and evaluates its initial-condition term on a further 500 points. The last column is a separate network, deliberately minimal, $\mathbb{R} \to \mathbb{R}$ on the single profile $\eta(x, 15)$, trained once per width.

Setting	**MLP**	**PINN**	**FNO**	**PINO**	**Spectral Bias MLPs**
Input → output	$\mathbb{R}^2 \to \mathbb{R}$	$\mathbb{R}^2 \to \mathbb{R}^2$	operator	operator	$\mathbb{R} \to \mathbb{R}$
Trainable parameters	198.401	198.658	2.106.146	2.106.146	241; 12.673; 789.505
Hidden layers	4	4	4 Fourier	4 Fourier	4
Width / channels	256	256	32	32	8; 64; 512
Fourier modes (per axis)	—	—	16	16	—
Optimizer	Adam	Adam	Adam	Adam	GD
Learning rate	10^{-3}	10^{-3}	10^{-3}	10^{-3}	$0{,}4/\lambda_{\max}$
Iterations	15.000	15.000	15.000	15.000	500.000
Parameters trained	single (3,21)	single (3,21)	all 20	all 20	single (3,21)
Collocation / batch	full batch	5.000 pts	batch 256	batch 256	128 pts (full)
Loss weights pde/ic/data	$-/-/1$	$1/1/\{1, 0\}$	$-/-/1$	$100/1/\{1, 0\}$	$-/-/1$
Physics-residual grid	—	5.000 pts	—	256×256	—
Target	$\eta(x, t)$	η, u	family	family	$\eta(x, 15)$
Data regimes	data only	with / without	data only	with / without	data only
Training wall-clock	$\approx 152\,\mathrm{s}$	$\approx 880\,\mathrm{s}$	$\approx 1178\,\mathrm{s}$	≈ 4440–$5460\,\mathrm{s}$	—

Source: The author (2026).

4 NUMERICAL RESULTS

This chapter reports what the experiments of Chapter 3 produced. Section 4.1 accounts for what each method costs to train. Section 4.2 puts the MLP and both PINN regimes side by side at their trained parameter, and Section 4.3 puts the FNO and the PINO side by side, sweeping the nonlinearity parameter at a fixed resolution (Section 4.3.1) and then querying both operators off their training resolution (Section 4.3.2). Section 4.4 fixes both and compares the spectra of all six methods mode by mode at three time instants. Section 4.5 tests the spectral-bias theory of Chapter 2 on a minimal network, and Section 4.6 follows the same bias through training in the six trained methods. The pseudospectral solver is the ground truth throughout.

4.1 Training Cost

Before asking how well the methods perform, we need to ask what they cost to train. Table 4.1 lists the size, training time, and final training loss of the six trained methods. It is important to notice that neural operator methods carry many more trainable parameters than the neural network methods. Specifically, PINO is slower because it assembles its physics residual on the finer 256×256 grid (Table 3.2) at every iteration.

Table 4.1 – Architecture size, training time on the GTX 1660 SUPER, and final training loss. The final-loss column is not comparable across rows. Accuracy is measured instead by the relative-error metrics of the following sections.

Method	**Regime**	**Parameters**	**Train time (s)**	**Final loss**
MLP	pure data	198.401	151,9	$4{,}3 \times 10^{-5}$
FNO	pure data	2.106.146	1177,8	$4{,}8 \times 10^{-7}$
PINO	data and physics	2.106.146	5459,4	$5{,}2 \times 10^{-5}$
PINO	pure physics	2.106.146	4441,2	$4{,}0 \times 10^{-5}$
PINN	data and physics	198.658	898,9	$7{,}1 \times 10^{-5}$
PINN	pure physics	198.658	868,4	$4{,}6 \times 10^{-5}$

Source: The author (2026).

4.2 Comparison Between Neural Networks

This section puts the MLP and both PINN regimes side by side. All three are trained at the single parameter $\alpha = \beta = 3{,}21$ and evaluated at resolution 256.

PINN with data wins, though it is still struggling with high-frequency content around the origin, while the pure-physics PINN's box plot spans nearly the whole height of the domain, which means its approximation completely misses the reference solution.

All three methods in Figure 4.1 still show the right wave shape. Data, not the PDE constraint, is what lowers the error down for the neural networks too, the same pattern we will see in the next Section 4.3.

4.3 Comparison Between Neural Operators

This section puts the FNO and the PINO side by side, with a sweep across the nonlinearity parameter, to test if they can readily predict the solution for a range of parameters, and a query off the training resolution, checking the zero-shot super-resolution.

4.3.1 Across the Parameter Range

The first comparison holds the resolution fixed at 256, 2x greater than the training resolution, and sweeps the parameters $\alpha = \beta$. It concerns the operators alone, since each pointwise MLP and PINN is trained at a single parameter and has nothing to say about the sweep.

Each figure below pairs the predicted field $\eta_\theta(x, t)$ against the time-resolved relative error $E(t_n)$ of (3.8), one column each, with the three parameters' distributions of $E(t_n)$ collected in the box plot at the foot, exposing its spread.

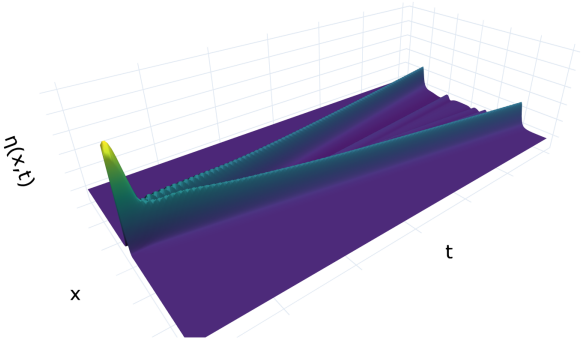
The box plots at the foot of Figures 4.2, 4.3, and 4.4 summarize the time-resolved error (3.8) over the whole trajectory at each parameter.

PINO with data beats both other operators at every parameter tested, which is what makes it the safer pick when the operating regime isn't known in advance. The physics-only PINO swings widest while FNO drifts less.

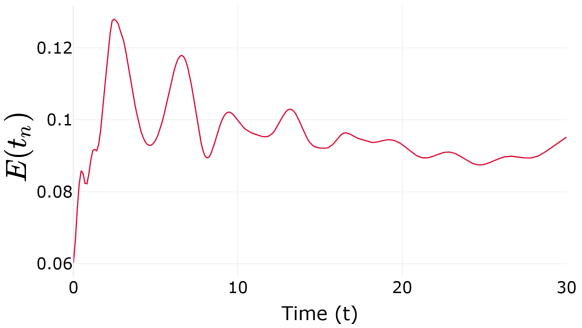
4.3.2 Across Resolutions

The next comparison queries each operator off its training resolution, probing the discretization invariance operators are meant to enjoy.

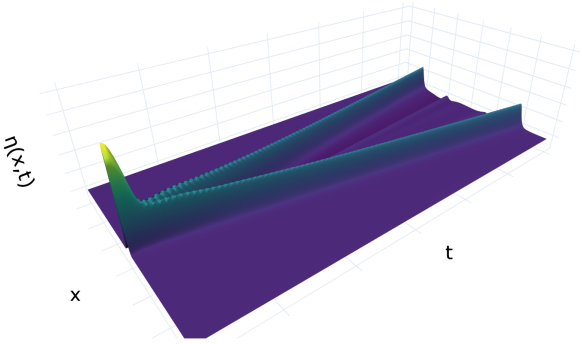
Figure 4.5 takes the plain FNO, trained only on data at resolution 128; Figures 4.6 and 4.7 take the two PINO regimes, whose physics residual was formed on a 256×256 grid (Table 3.2).



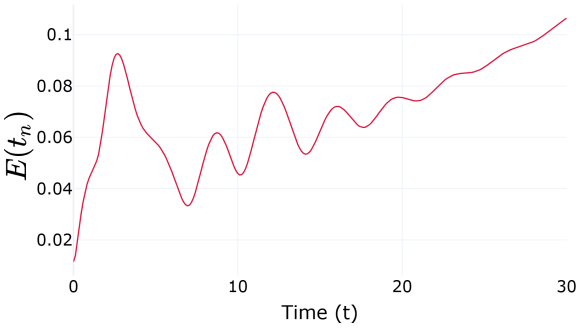
(a) Predicted $\eta(x,t)$, MLP.



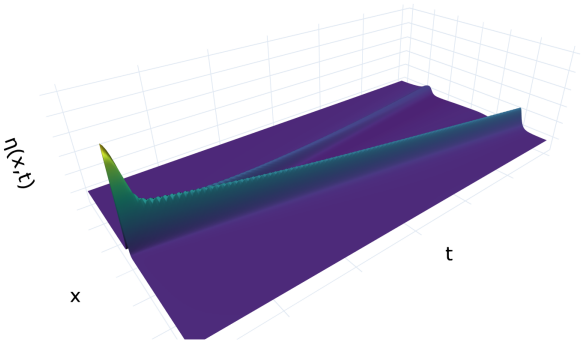
(b) Relative error, MLP.



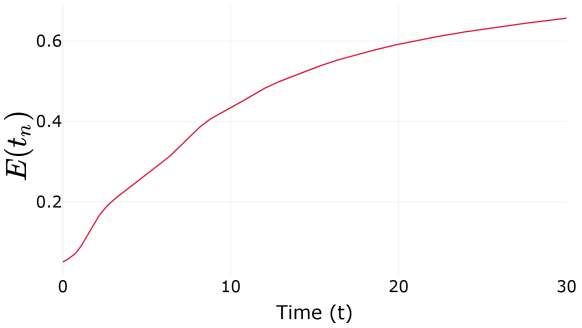
(c) Predicted $\eta(x,t)$, PINN (data and physics).



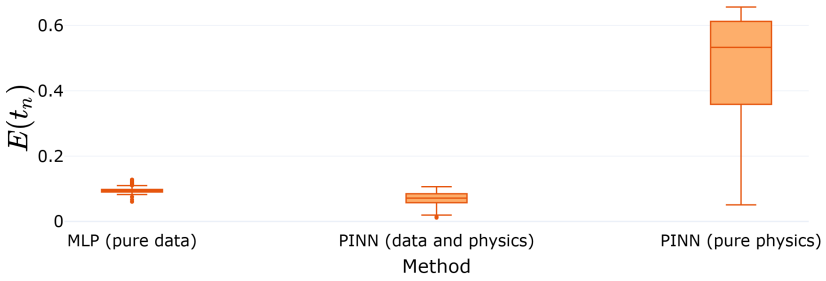
(d) Relative error, PINN (data and physics).



(e) Predicted $\eta(x,t)$, PINN (pure physics).



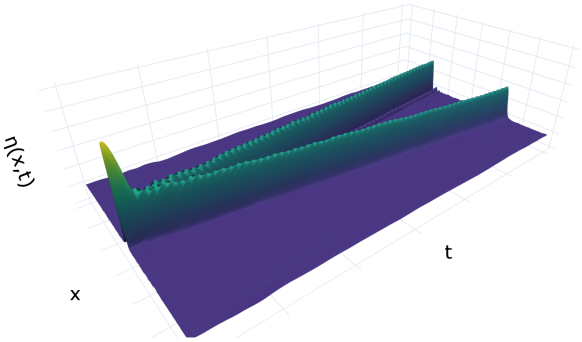
(f) Relative error, PINN (pure physics).



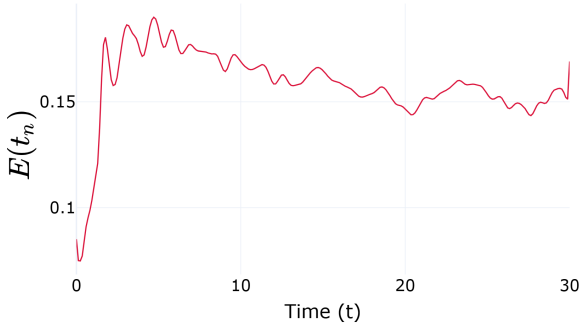
(g) Relative error distribution vs. method.

Figure 4.1 – MLP and both PINN regimes at $\alpha = \beta = 3{,}21$, resolution 256. Left: predicted $\eta(x,t)$. Right: time-resolved relative error $E(t_n)$ of (3.8).

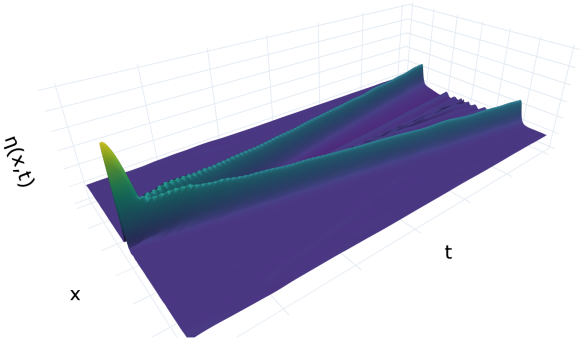
Source: The author (2026).



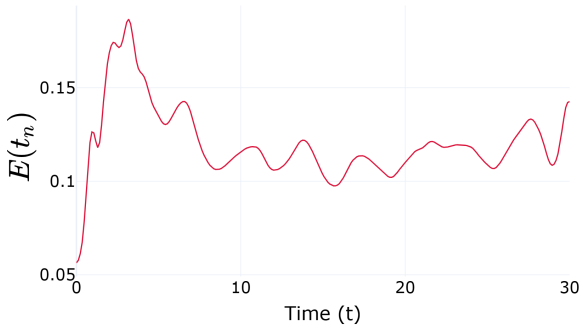
(a) Predicted $\eta(x,t)$, $\alpha = \beta = 0{,}1$.



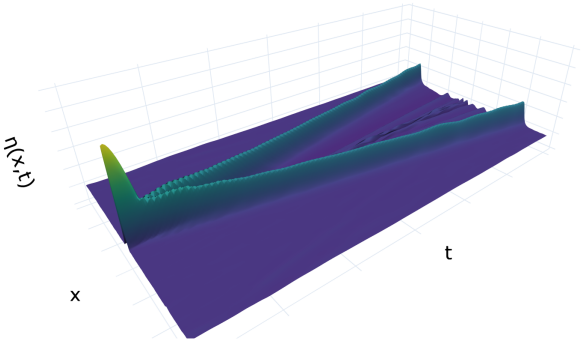
(b) Relative error, $\alpha = \beta = 0{,}1$.



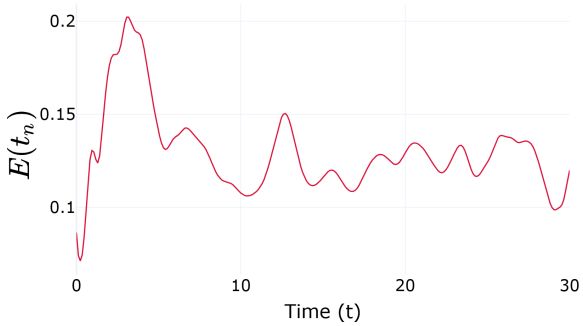
(c) Predicted $\eta(x,t)$, $\alpha = \beta = 3{,}21$.



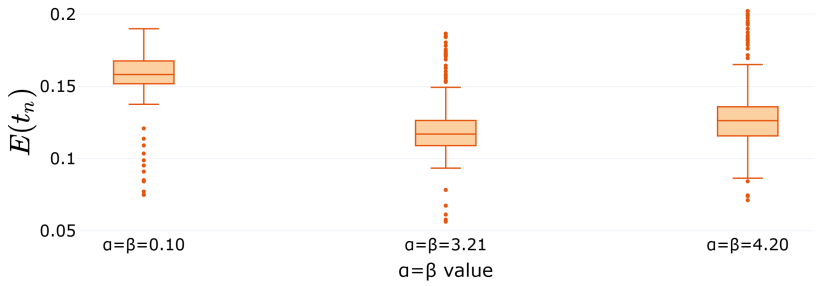
(d) Relative error, $\alpha = \beta = 3{,}21$.



(e) Predicted $\eta(x,t)$, $\alpha = \beta = 4{,}2$.



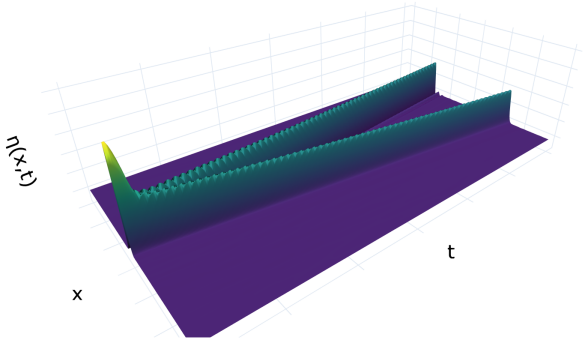
(f) Relative error, $\alpha = \beta = 4{,}2$.



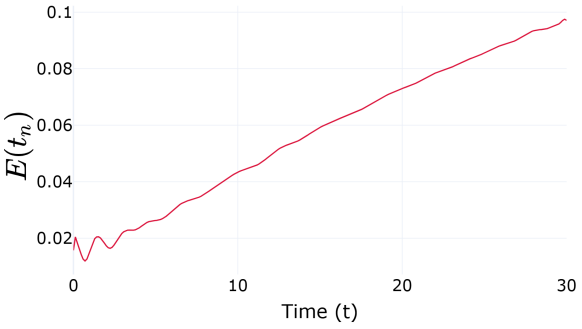
(g) Relative error distribution vs. $\alpha = \beta$.

Figure 4.2 – FNO (pure data) across the nonlinearity axis ($\alpha = \beta \in \{0{,}1;\ 3{,}21;\ 4{,}2\}$), evaluated at resolution 256. Left: predicted $\eta(x,t)$. Right: time-resolved relative error $E(t_n)$ of (3.8).

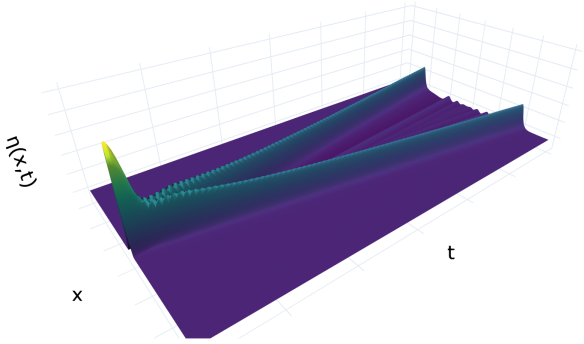
Source: The author (2026).



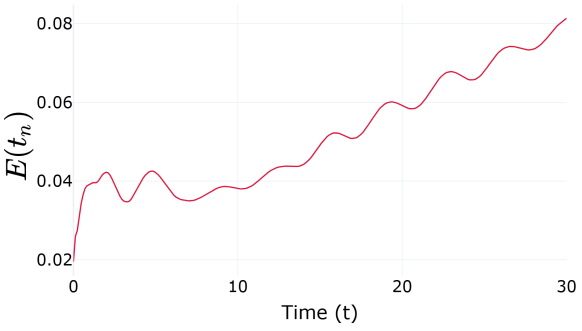
(a) Predicted $\eta(x,t)$, $\alpha = \beta = 0{,}1$.



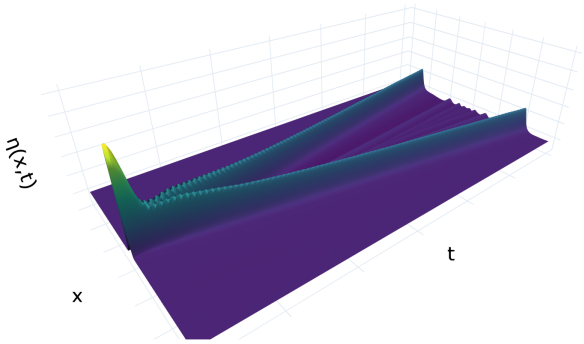
(b) Relative error, $\alpha = \beta = 0{,}1$.



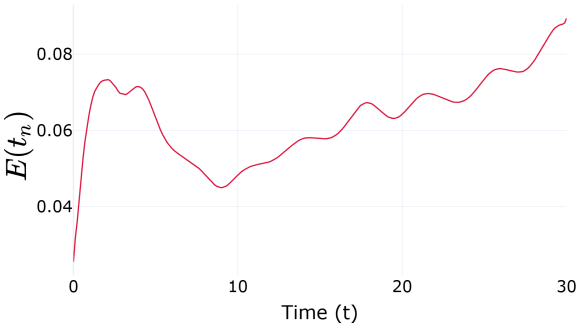
(c) Predicted $\eta(x,t)$, $\alpha = \beta = 3{,}21$.



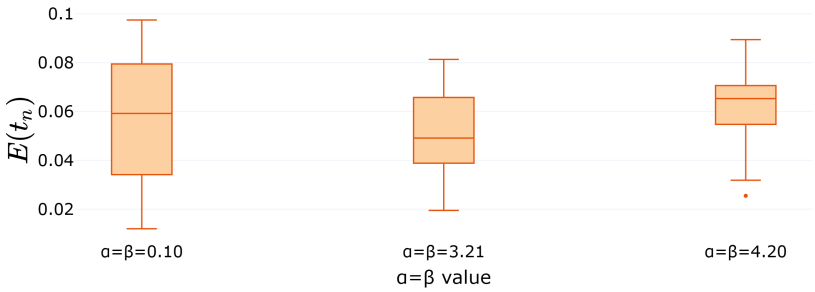
(d) Relative error, $\alpha = \beta = 3{,}21$.



(e) Predicted $\eta(x,t)$, $\alpha = \beta = 4{,}2$.



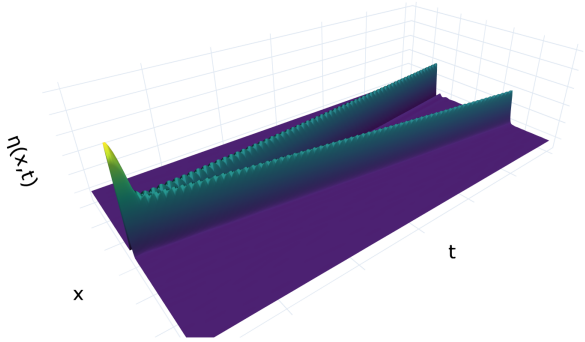
(f) Relative error, $\alpha = \beta = 4{,}2$.



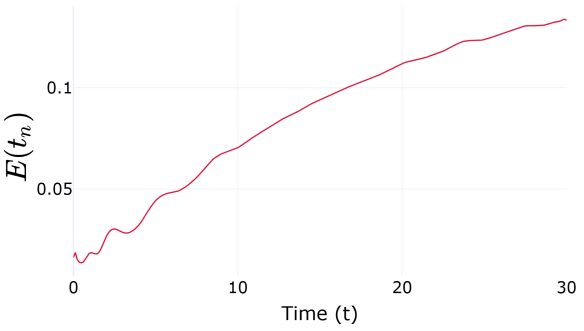
(g) Relative error distribution vs. $\alpha = \beta$.

Figure 4.3 – PINO (data and physics) across the same nonlinearity axis and resolution. Left: predicted $\eta(x,t)$. Right: time-resolved relative error $E(t_n)$ of (3.8).

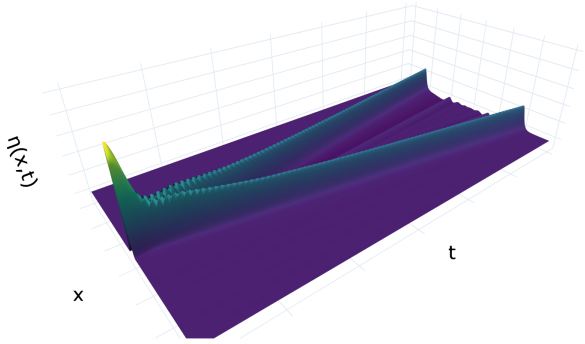
Source: The author (2026).



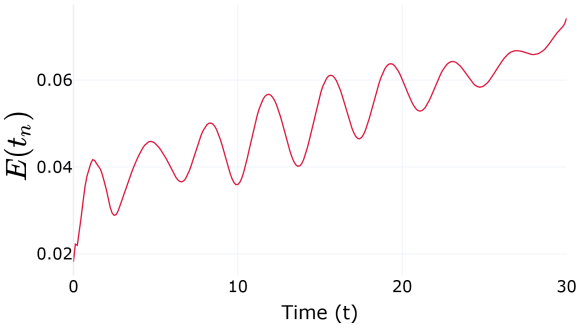
(a) Predicted $\eta(x,t)$, $\alpha = \beta = 0{,}1$.



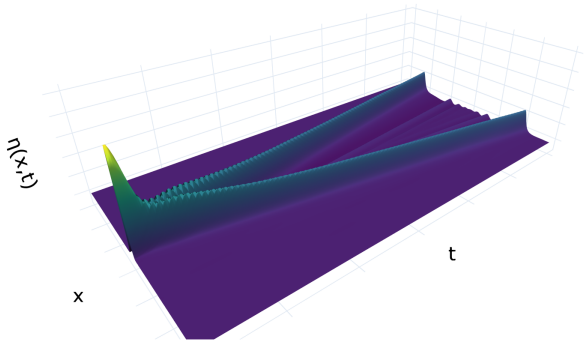
(b) Relative error, $\alpha = \beta = 0{,}1$.



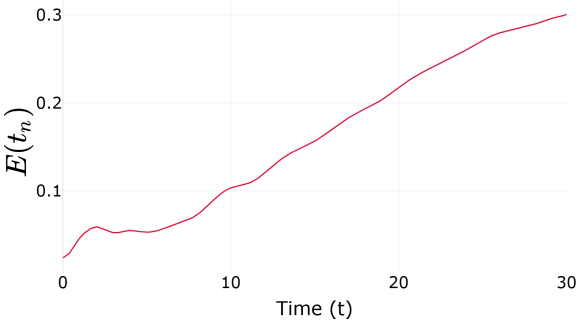
(c) Predicted $\eta(x,t)$, $\alpha = \beta = 3{,}21$.



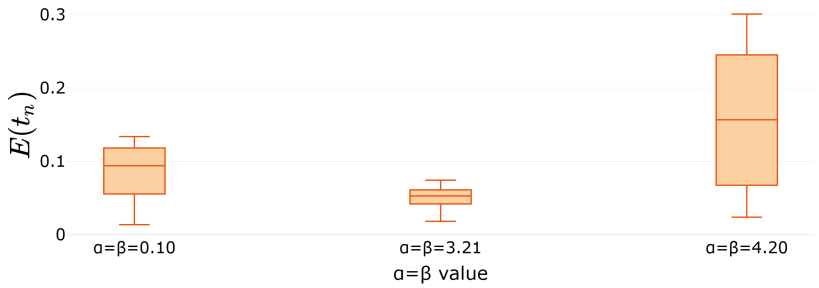
(d) Relative error, $\alpha = \beta = 3{,}21$.



(e) Predicted $\eta(x,t)$, $\alpha = \beta = 4{,}2$.



(f) Relative error, $\alpha = \beta = 4{,}2$.



(g) Relative error distribution vs. $\alpha = \beta$.

Figure 4.4 – PINO (pure physics) across the same nonlinearity axis and resolution. Left: predicted $\eta(x,t)$. Right: time-resolved relative error $E(t_n)$ of (3.8).

Source: The author (2026).

All three are queried at resolutions 128, 256, and 512 at the fixed parameter $\alpha = \beta = 3{,}21$ with no retraining nor fine-tuning.

As in the previous section, each panel pairs the predicted field $\eta_\theta(x,t)$ with the time-resolved relative error $E(t_n)$ of (3.8), and the box plot at the foot shows $E(t_n)$'s distribution at the three resolutions.

Every operator returns a smooth, physically plausible surface at every resolution tested, with no retraining, since the grid only changes how densely the same kernel gets sampled.

Both PINOs bottom out at resolution 256, the grid their physics residual was trained on, while FNO was trained fitting the dataset at 128, yielding the lowest error there. Again, PINO with data was the best performer.

4.4 Spectral Comparison of the Six Methods

To compare the families on equal terms we fix $\alpha = \beta = 3{,}21$ and inspect the full spatial spectrum of the prediction at three times, $t = 0$, $t = T/2 = 15$, and $t = T = 30$. Figures 4.8–4.13 report the six methods. In every panel the black curve is the reference amplitude $|\hat{\eta}_{\text{true}}(k, t_n)|$ and the dashed orange curve is the prediction $|\hat{\eta}_{\text{pred}}(k, t_n)|$, both read off the DFT (2.11).

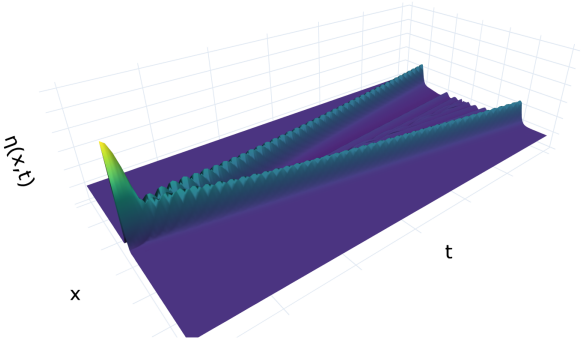
Beside each spectrum is also the per-mode absolute error $E_{\text{spec}}(k, t_n)$ of (3.9), and at the foot of each figure is that same error averaged over every mode, $\widetilde{E}_{\text{spec}}(t_n)$ of (3.12). One reading note applies throughout. The per-mode error (3.9) is a difference of amplitudes in the units of η, not a ratio, so it is read against the reference amplitude at the same wavenumber.

PINO with data wins and the worst performer is the physics-only PINN followed by PINO without data; the FNO, the plain MLP, and the data-trained PINN land in between, but all of them show good results.

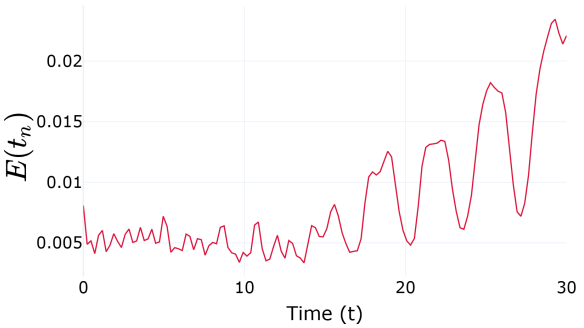
4.5 Benchmark for Spectral Bias MLP

Here, we aim to take a turn into investigating the spectral bias of basic MLPs. Derived in Section 2.3.4, this effect predicts that gradient training resolves low frequencies before high ones at a rate set by the eigenvalues of the Neural Tangent Kernel. Our experiment of Section 3.5 puts that prediction on the Boussinesq system in its barest form with the Spectral Bias MLP learning the single profile $\eta(x, 15)$ at $\alpha = \beta = 3{,}21$, at three widths, by just gradient descent at the per-width step $\mu = 0{,}4/\lambda_{\max}$, the largest initial NTK eigenvalue, run for half a million iterations. Figure 4.14 reports the four checks.

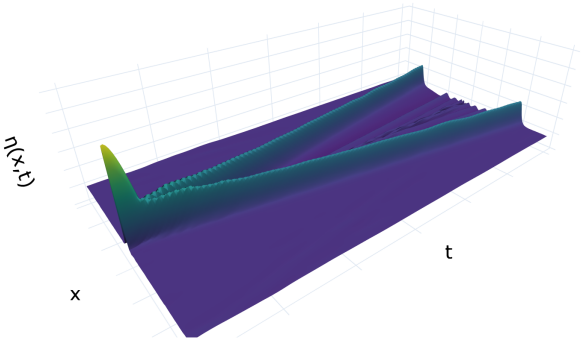
In Figure 4.14a the amplitude of the error along each eigendirection follows the



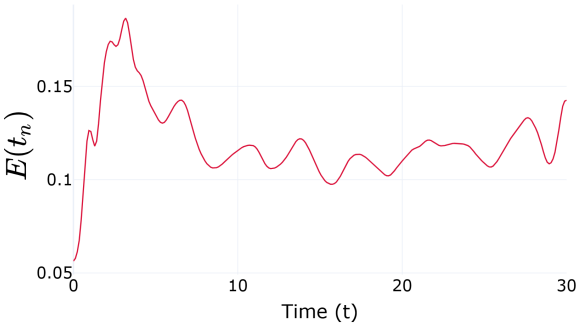
(a) Predicted $\eta(x,t)$, res. 128.



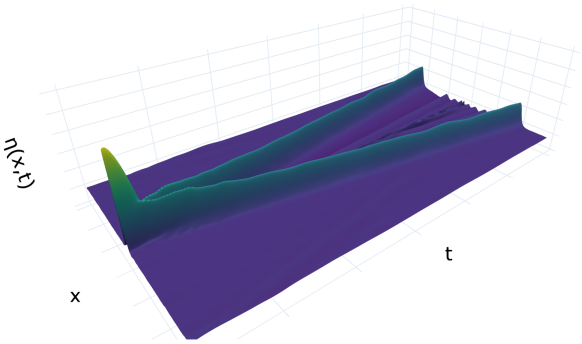
(b) Relative error, res. 128.



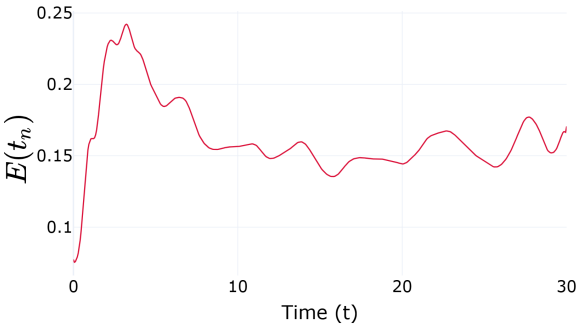
(c) Predicted $\eta(x,t)$, res. 256.



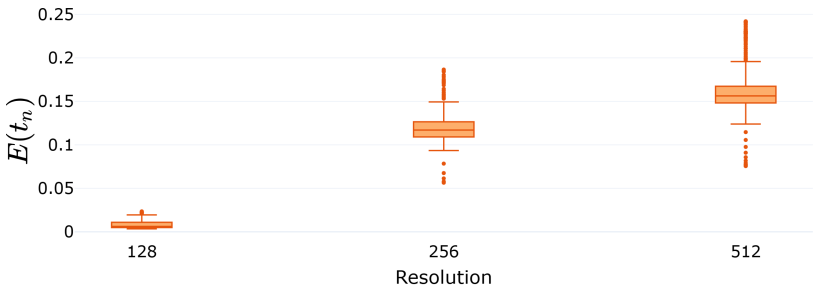
(d) Relative error, res. 256.



(e) Predicted $\eta(x,t)$, res. 512.



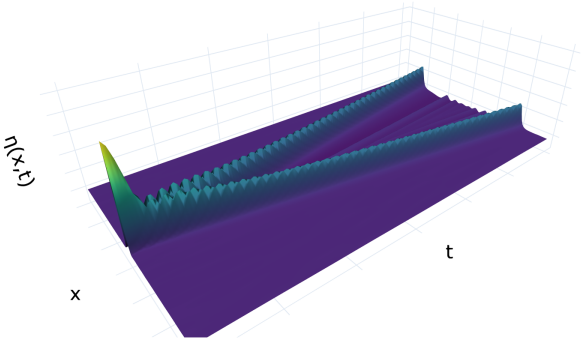
(f) Relative error, res. 512.



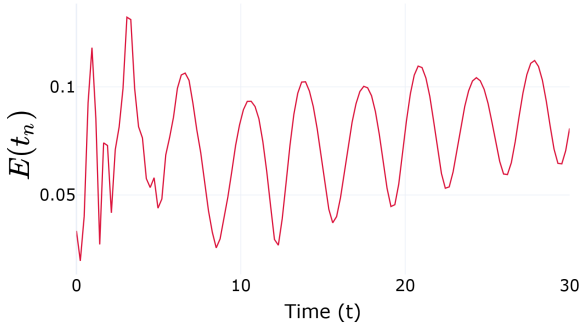
(g) Relative error distribution vs. resolution.

Figure 4.5 – Plain FNO queried zero-shot at resolutions 128, 256 and 512 at $\alpha = \beta = 3{,}21$. Predicted $\eta(x,t)$ and time-resolved relative error $E(t_n)$ of (3.8) at each resolution.

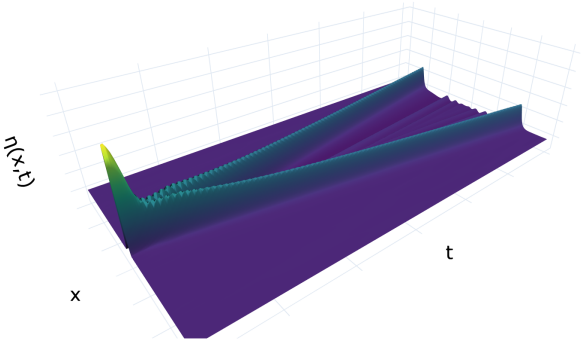
Source: The author (2026).



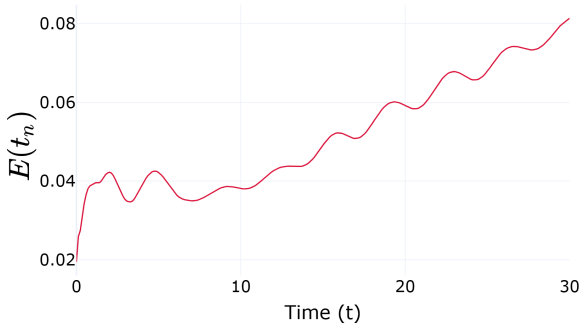
(a) Predicted $\eta(x,t)$, res. 128.



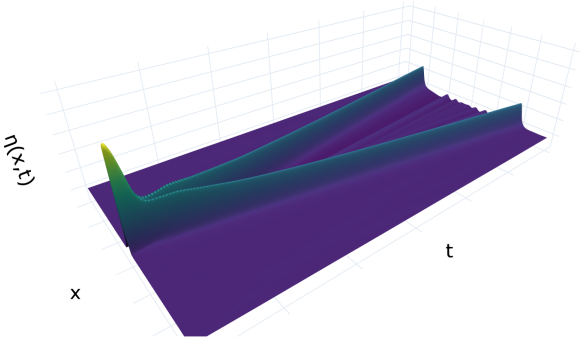
(b) Relative error, res. 128.



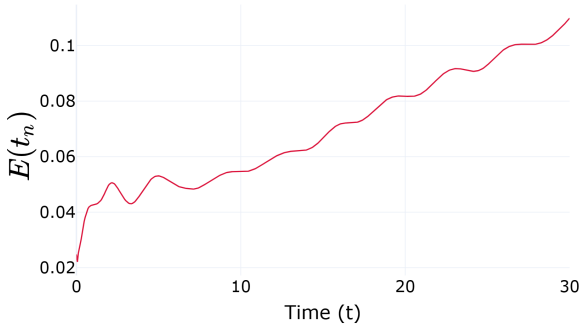
(c) Predicted $\eta(x,t)$, res. 256.



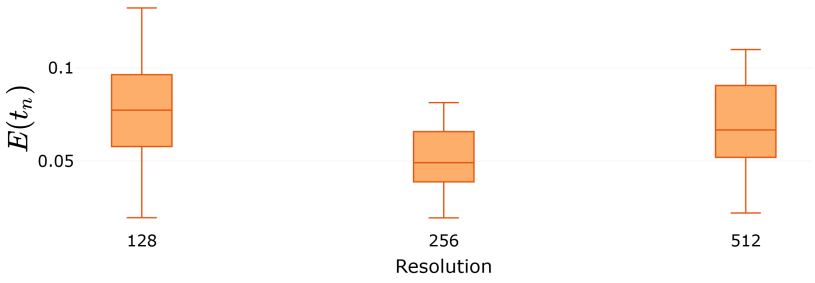
(d) Relative error, res. 256.



(e) Predicted $\eta(x,t)$, res. 512.



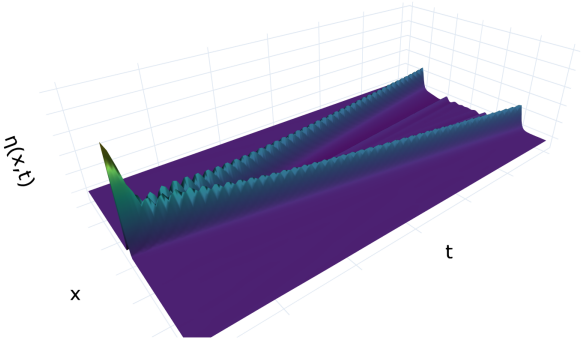
(f) Relative error, res. 512.



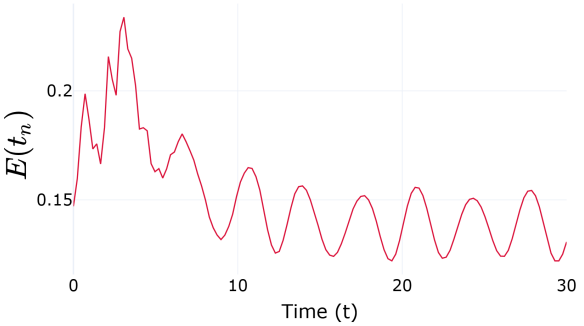
(g) Relative error distribution vs. resolution.

Figure 4.6 – PINO (data and physics) queried zero-shot at resolutions 128, 256 and 512 at $\alpha = \beta = 3{,}21$. Predicted $\eta(x,t)$ and time-resolved relative error $E(t_n)$ of (3.8) at each resolution.

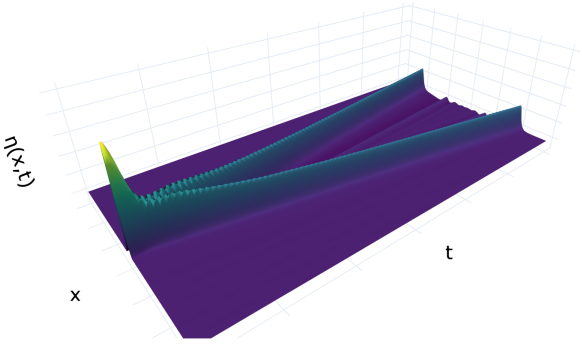
Source: The author (2026).



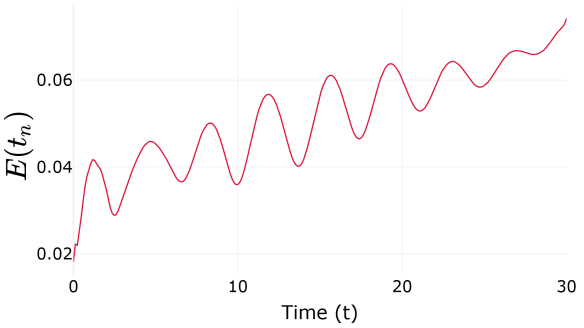
(a) Predicted $\eta(x,t)$, res. 128.



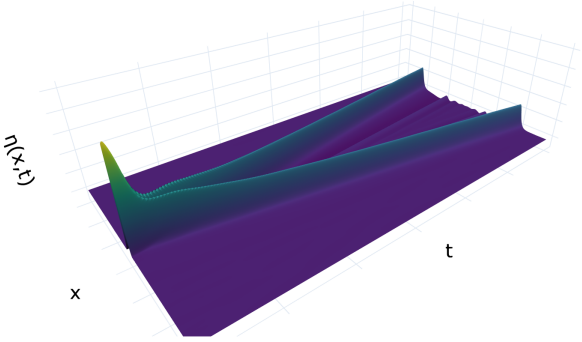
(b) Relative error, res. 128.



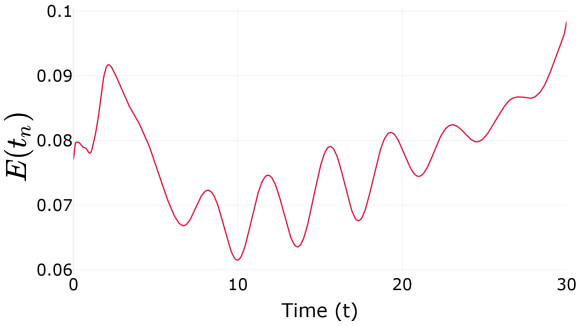
(c) Predicted $\eta(x,t)$, res. 256.



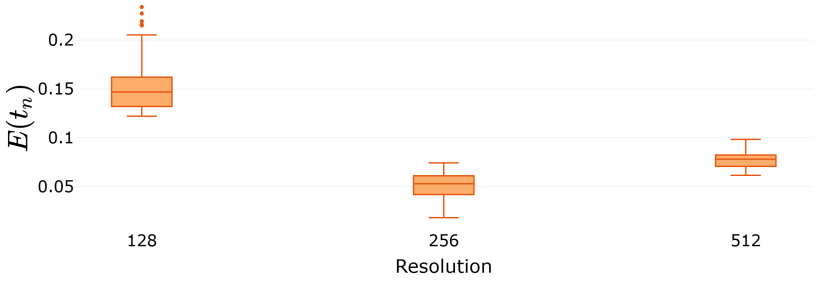
(d) Relative error, res. 256.



(e) Predicted $\eta(x,t)$, res. 512.



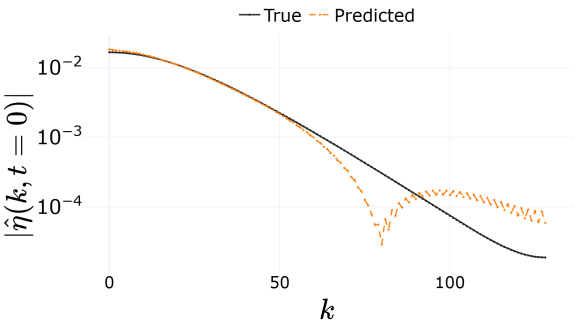
(f) Relative error, res. 512.



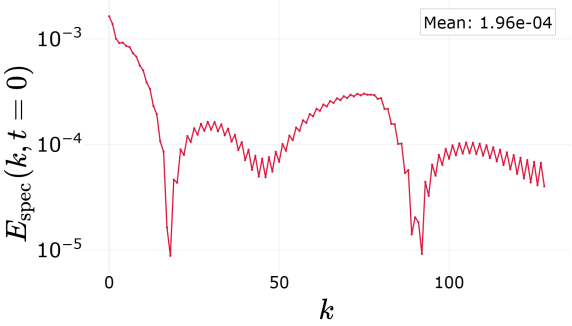
(g) Relative error distribution vs. resolution.

Figure 4.7 – PINO (pure physics) queried zero-shot at resolutions 128, 256 and 512 at $\alpha = \beta = 3{,}21$. Predicted $\eta(x,t)$ and time-resolved relative error $E(t_n)$ of (3.8) at each resolution.

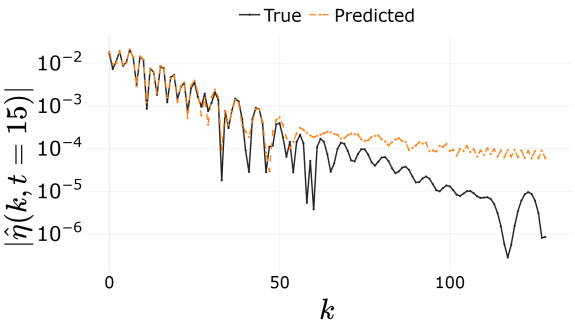
Source: The author (2026).



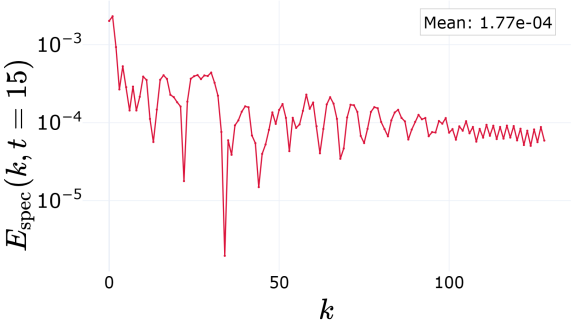
(a) Spectrum, $t = 0$.



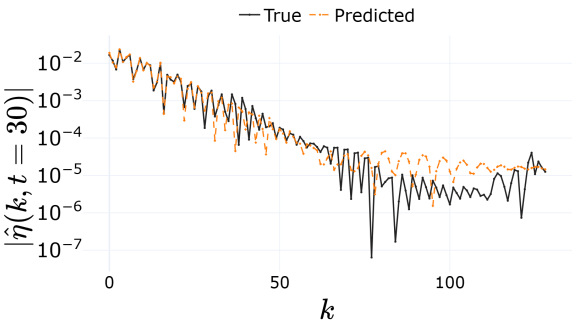
(b) Relative error, $t = 0$.



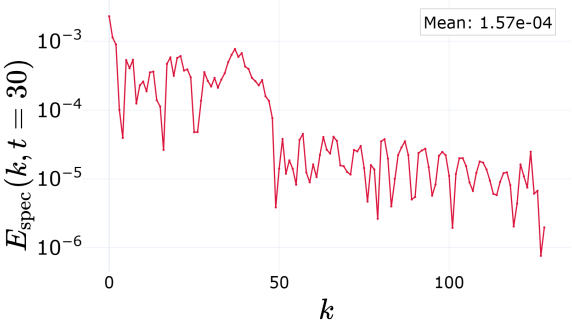
(c) Spectrum, $t = 15$.



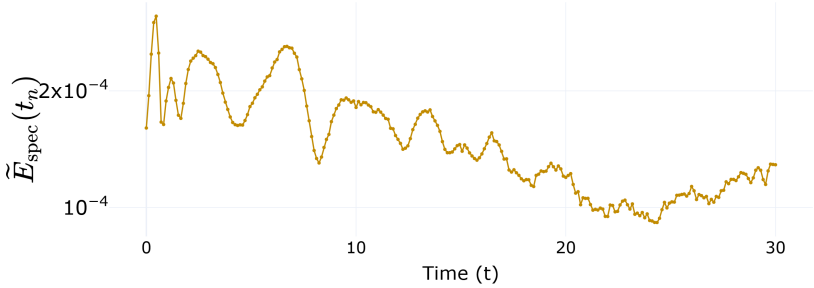
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



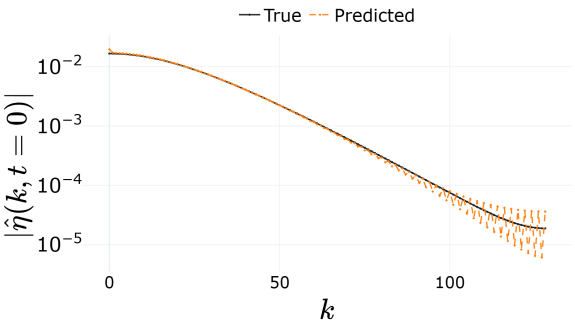
(f) Relative error, $t = 30$.



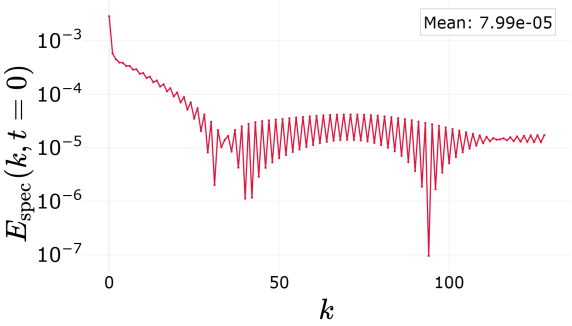
(g) Mean absolute spectral error over time.

Figure 4.8 – MLP (pure data) spatial spectrum at $\alpha = \beta = 3{,}21$. Amplitude $|\hat{\eta}(k, t_n)|$ from the DFT (2.11), absolute error $E_{\text{spec}}(k, t_n)$ of (3.9), and mean error $\widetilde{E}_{\text{spec}}(t_n)$ of (3.12).

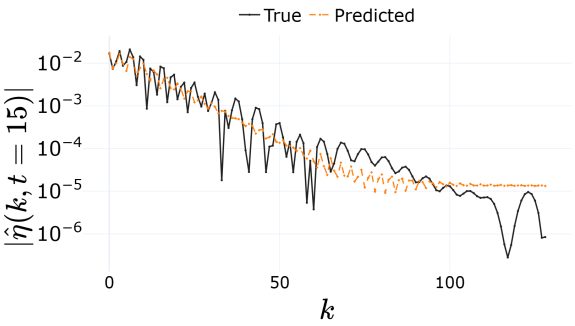
Source: The author (2026).



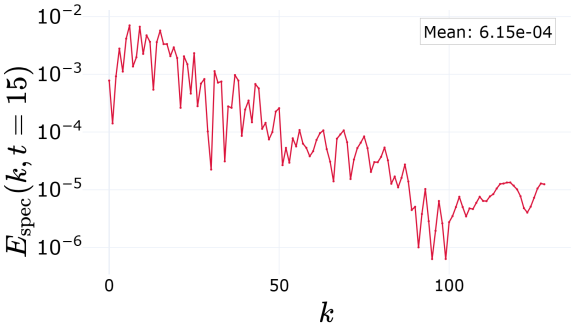
(a) Spectrum, $t = 0$.



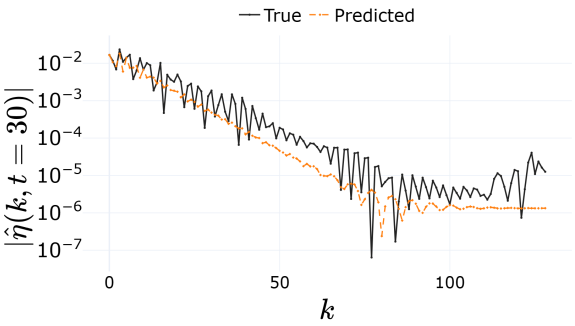
(b) Relative error, $t = 0$.



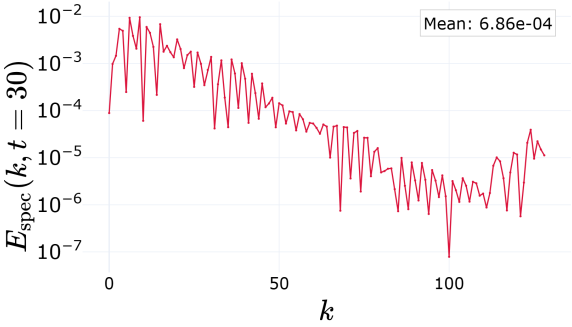
(c) Spectrum, $t = 15$.



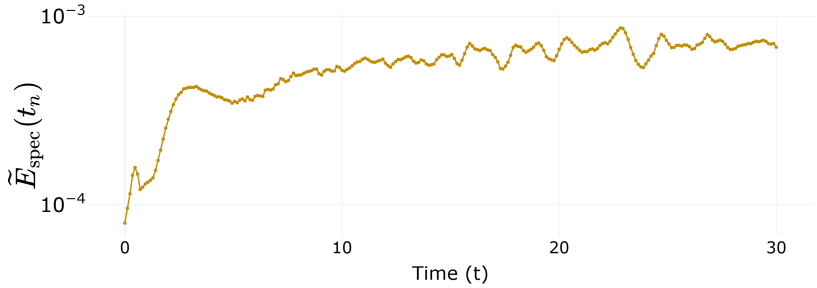
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



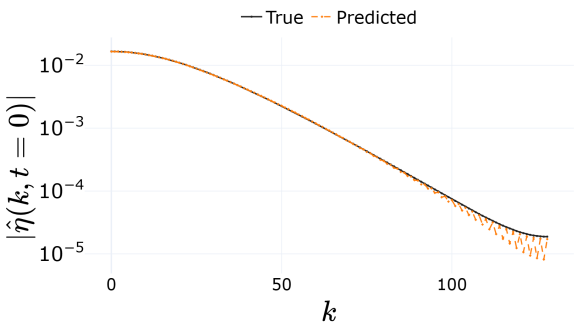
(f) Relative error, $t = 30$.



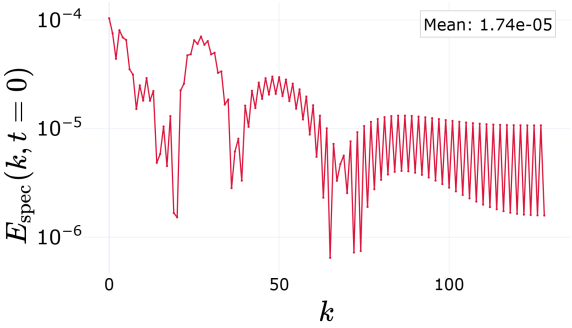
(g) Mean absolute spectral error over time.

Figure 4.9 – PINN (pure physics) spatial spectrum at $\alpha = \beta = 3{,}21$. Amplitude $|\hat{\eta}(k, t_n)|$ from the DFT (2.11), absolute error $E_{\text{spec}}(k, t_n)$ of (3.9), and mean error $\widetilde{E}_{\text{spec}}(t_n)$ of (3.12).

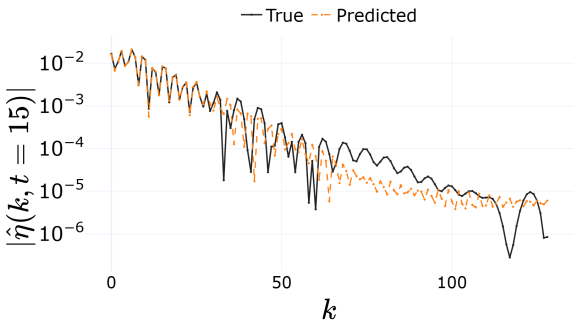
Source: The author (2026).



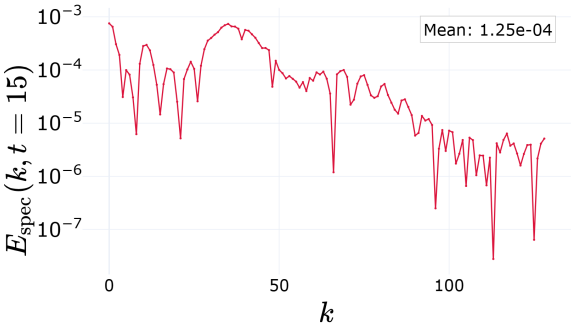
(a) Spectrum, $t = 0$.



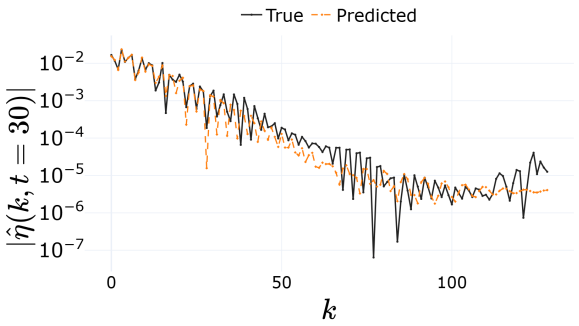
(b) Relative error, $t = 0$.



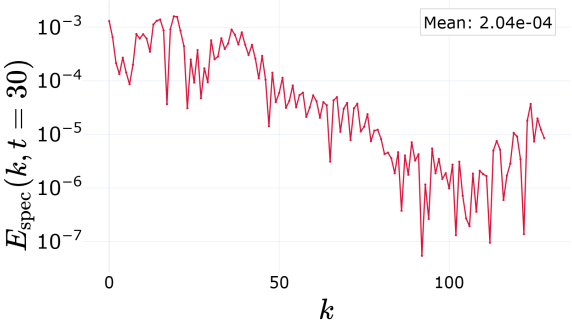
(c) Spectrum, $t = 15$.



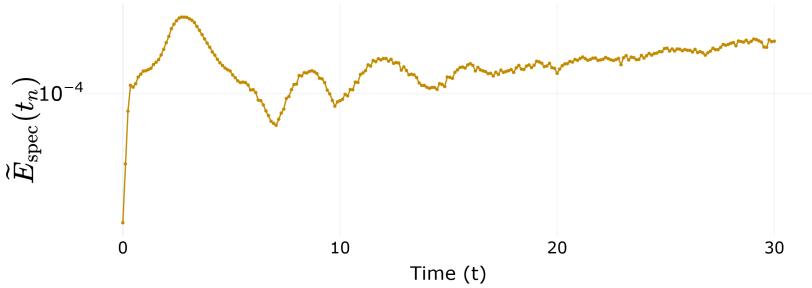
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



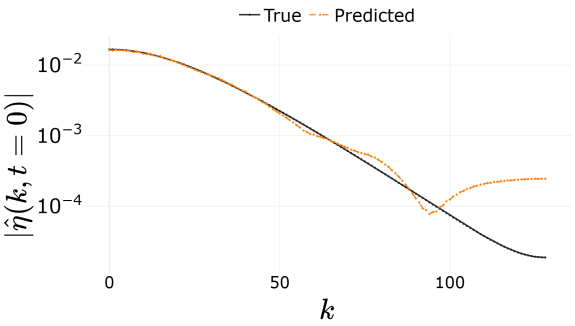
(f) Relative error, $t = 30$.



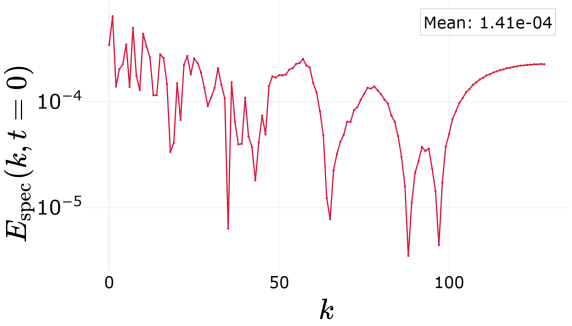
(g) Mean absolute spectral error over time.

Figure 4.10 – PINN (data and physics) spatial spectrum at $\alpha = \beta = 3{,}21$. Amplitude $|\hat{\eta}(k, t_n)|$ from the DFT (2.11), absolute error $E_{\text{spec}}(k, t_n)$ of (3.9), and mean error $\widetilde{E}_{\text{spec}}(t_n)$ of (3.12).

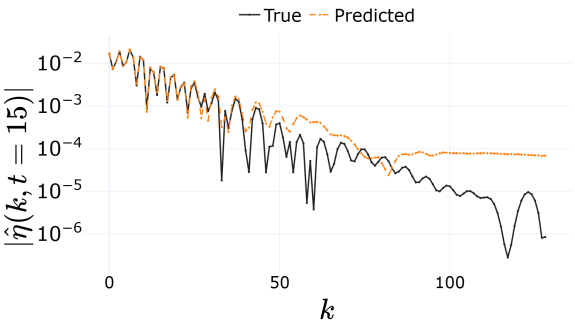
Source: The author (2026).



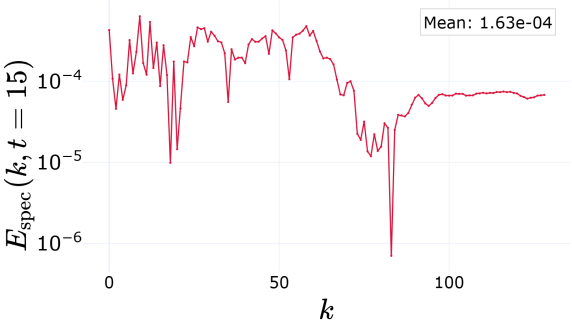
(a) Spectrum, $t = 0$.



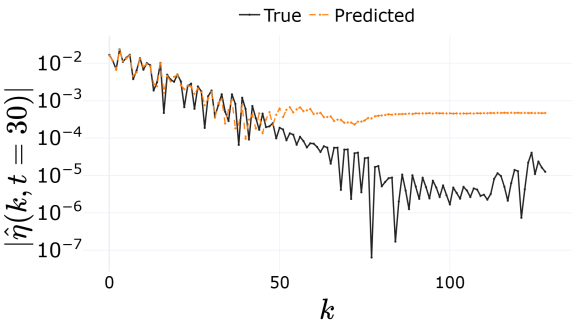
(b) Relative error, $t = 0$.



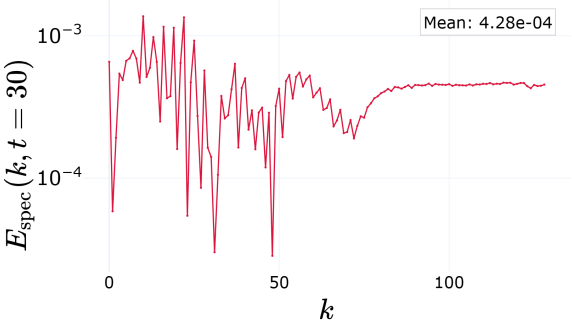
(c) Spectrum, $t = 15$.



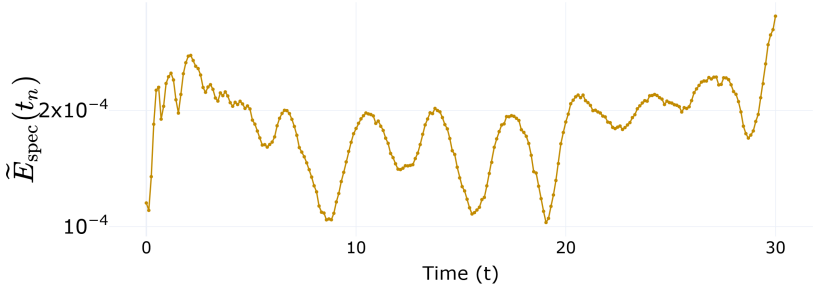
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



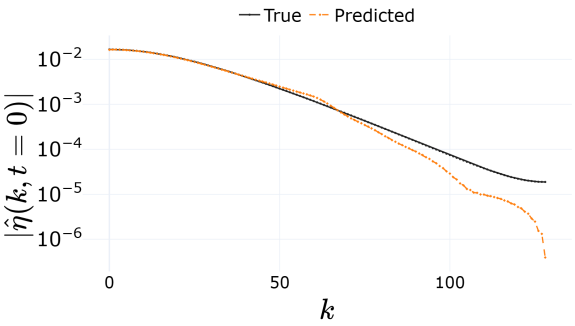
(f) Relative error, $t = 30$.



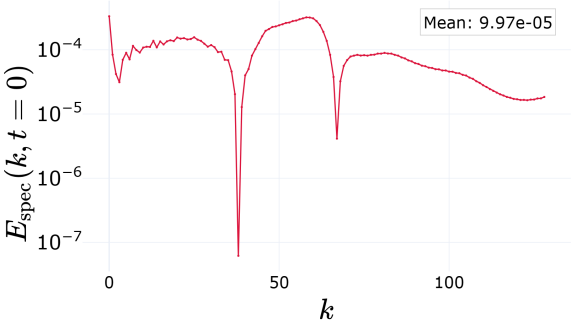
(g) Mean absolute spectral error over time.

Figure 4.11 – FNO (pure data) spatial spectrum at $\alpha = \beta = 3{,}21$. Amplitude $|\hat{\eta}(k, t_n)|$ from the DFT (2.11), absolute error $E_{\text{spec}}(k, t_n)$ of (3.9), and mean error $\widetilde{E}_{\text{spec}}(t_n)$ of (3.12).

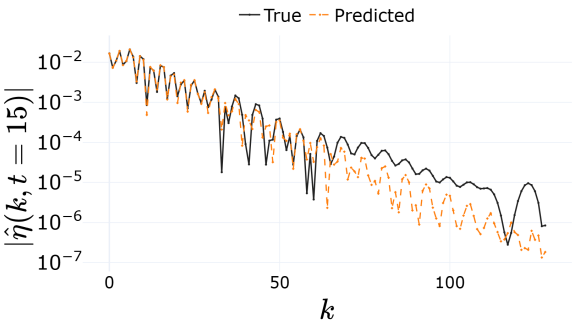
Source: The author (2026).



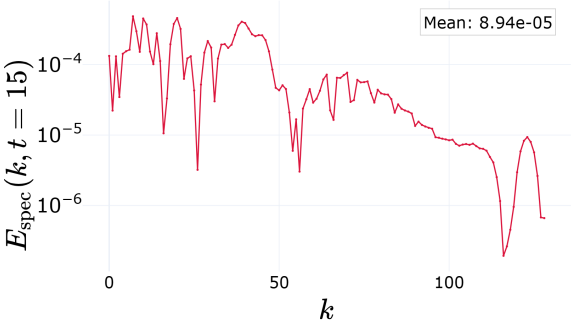
(a) Spectrum, $t = 0$.



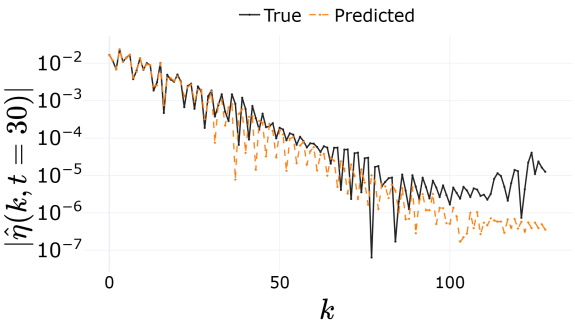
(b) Relative error, $t = 0$.



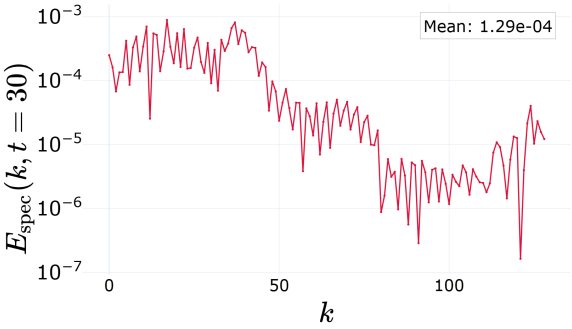
(c) Spectrum, $t = 15$.



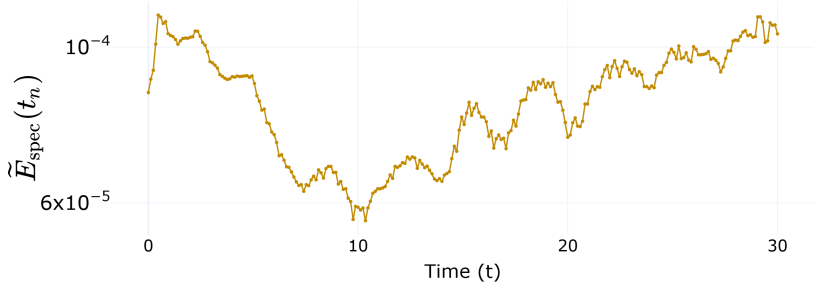
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



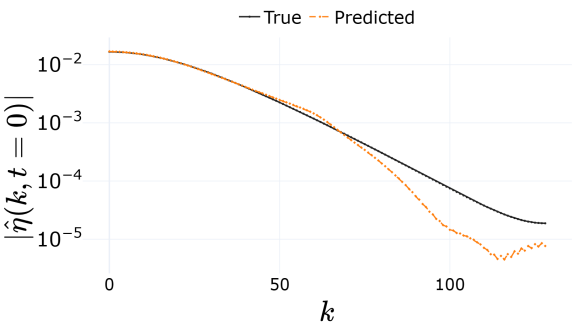
(f) Relative error, $t = 30$.



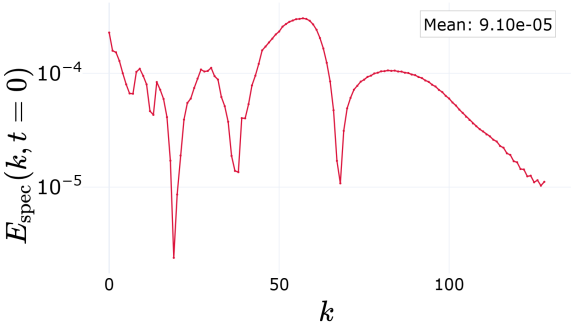
(g) Mean absolute spectral error over time.

Figure 4.12 – PINO (data and physics) spatial spectrum at $\alpha = \beta = 3{,}21$. Amplitude $|\hat{\eta}(k, t_n)|$ from the DFT (2.11), absolute error $E_{\text{spec}}(k, t_n)$ of (3.9), and mean error $\widetilde{E}_{\text{spec}}(t_n)$ of (3.12).

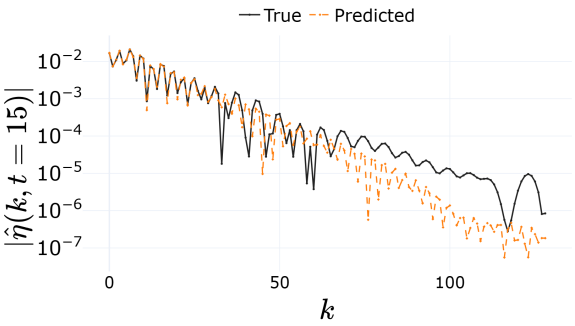
Source: The author (2026).



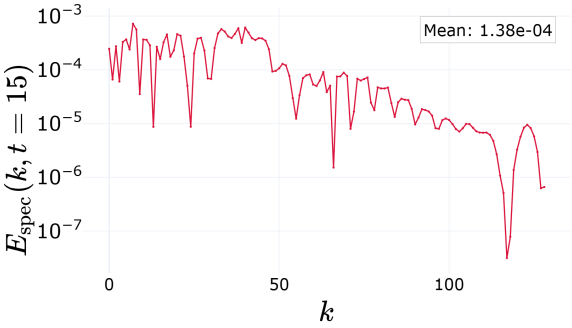
(a) Spectrum, $t = 0$.



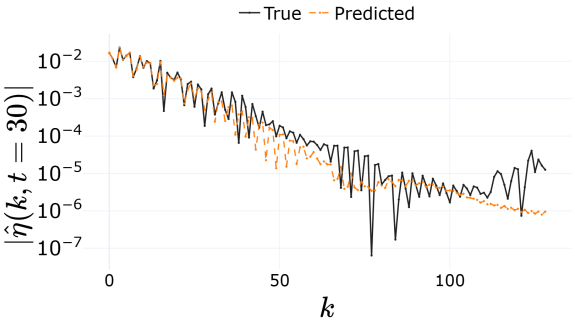
(b) Relative error, $t = 0$.



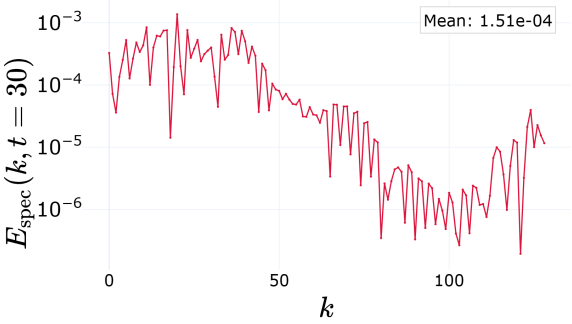
(c) Spectrum, $t = 15$.



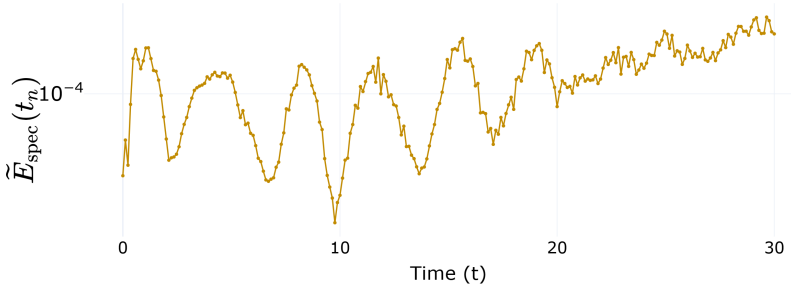
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



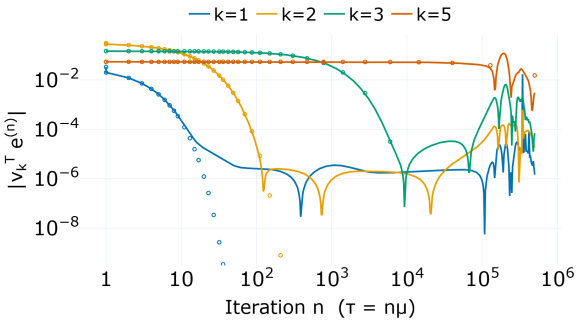
(f) Relative error, $t = 30$.



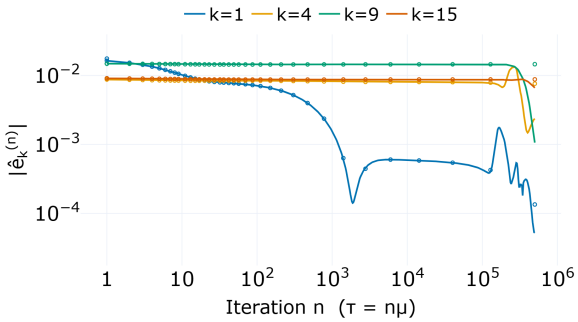
(g) Mean absolute spectral error over time.

Figure 4.13 – PINO (pure physics) spatial spectrum at $\alpha = \beta = 3{,}21$. Amplitude $|\hat{\eta}(k, t_n)|$ from the DFT (2.11), absolute error $E_{\text{spec}}(k, t_n)$ of (3.9), and mean error $\widetilde{E}_{\text{spec}}(t_n)$ of (3.12).

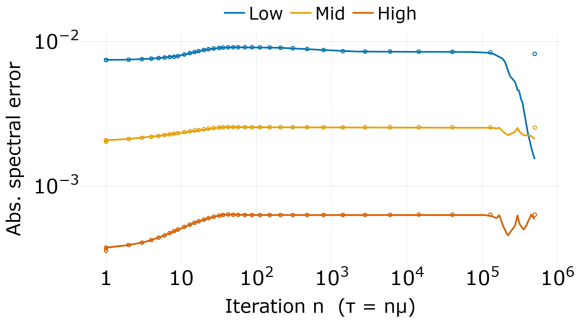
Source: The author (2026).



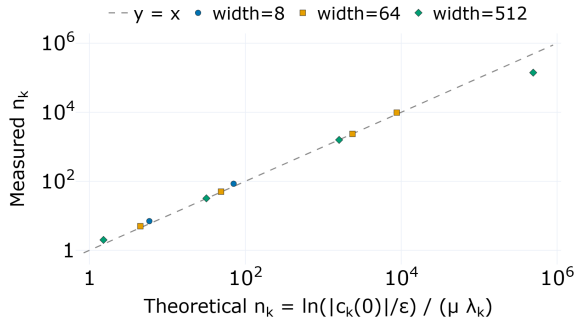
(a) Per-eigendirection decay, $c_k^{(n)}$ of (3.15) against (2.38).



(b) Per-mode spectral decay against (2.43).



(c) Band-averaged absolute spectral error $E_{\text{band}}(B)$ of (3.10).



(d) Iteration count n_k against (3.16).

Figure 4.14 – Spectral bias predictions measured in the minimal experiment of Section 3.5, the Spectral Bias MLP fitting $\eta(x, 15)$ at $\alpha = \beta = 3{,}21$ by full-batch gradient descent at the per-width step (3.14), run for 500.000 iterations. Panels (a)–(c) are the widest network, $N_{\text{MLP}} = 512$; panel (d) collects all three widths. Lines are measured, open markers are the initial-kernel prediction.

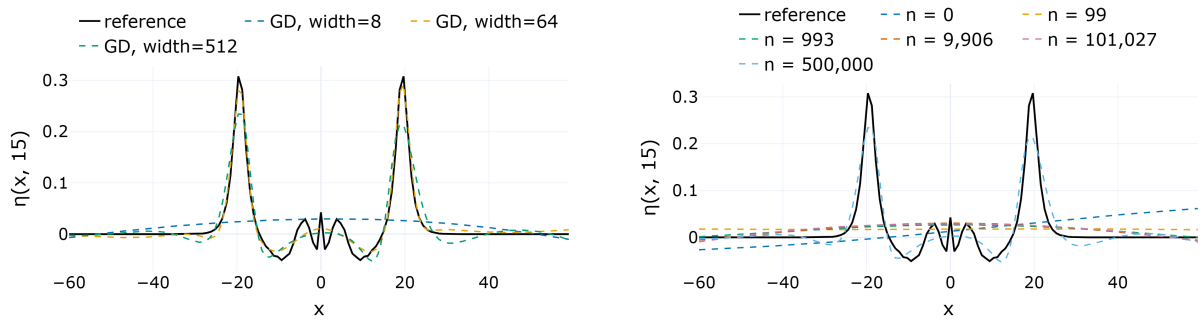
Source: The author (2026).

predicted $|1 - \mu\lambda_k|^n$ of (2.38) nicely.

Figure 4.14b reads the error in Fourier space, where the lowest frequency goes down while the higher ones lie flat across the run, showing the clear tendency of the spectral bias. Figure 4.14c takes the whole spectrum instead of single wavenumbers, averaging the absolute error over the low, mid and high bands defined in (3.11), also showing the tendency of first learning low-frequency content.

Figure 4.14d shows the discrete crossing count of (2.47) for the three trained networks of different sizes, as described in Section 3.5; we see a good fit until the kernel drifts away and our prediction falls apart.

Figure 4.15 sets what the three runs produced against the data coming from the solution of (3.13); we see that, although the 64 neuron width MLP is the best, our goal does not care if the prediction is good, only to show how the wide MLPs follow the expected behavior coming from an almost constant kernel throughout the training.



(a) Final fit at each width.

(b) Widest network across the run.

Figure 4.15 – The Spectral Bias MLPs against the reference profile $\eta(x, 15)$ at $\alpha = \beta = 3{,}21$. Panel (a) shows the final fit at each width; panel (b) follows the widest network across logarithmically spaced checkpoints.

Source: The author (2026).

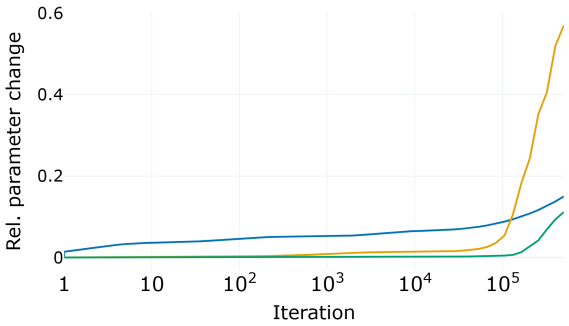
Everything above assumes the kernel stays put. Figure 4.16 reports whether it actually does, measured along the same three runs.

Figures 4.16a and 4.16b show that the wider the network, the fewer parameters change in training, implicating less kernel change. This is what inspires us to use the widest MLP possible in the Spectral Bias MLP experiment defined in Section 3.5.

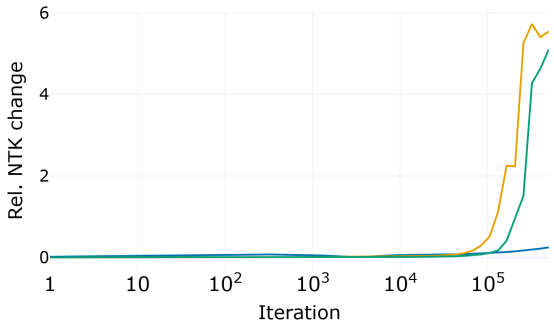
Figure 4.16c shows that eigenvalues do change a lot when compared with the MLP width.

4.6 Spectral Bias Throughout All Six Methods

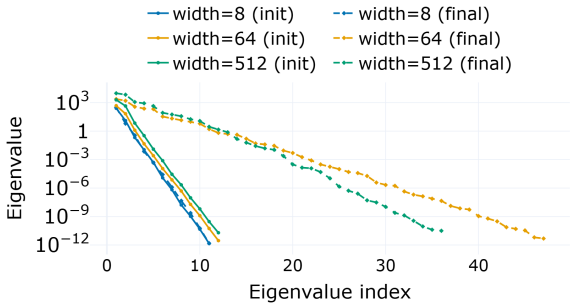
We now watch spectral bias develop as the optimizer runs. Figure 4.17 tracks the band-averaged absolute spectral error $E_{\text{band}}(B)$ of (3.10), for the low, mid, and high



(a) Relative parameter drift $\|\theta - \theta_0\|/\|\theta_0\|$.



(b) Relative kernel drift $\|K - K_0\|/\|K_0\|$.

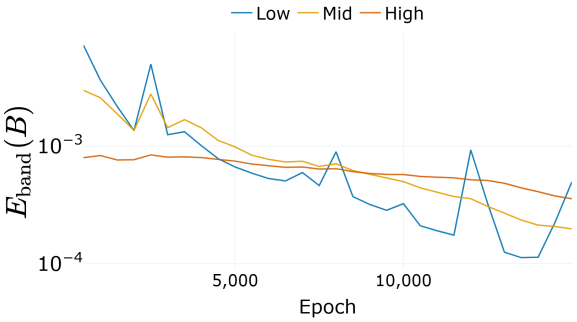


(c) Eigenvalue spectrum of K: initialization (solid) versus end of training (dashed), on log–log axes.

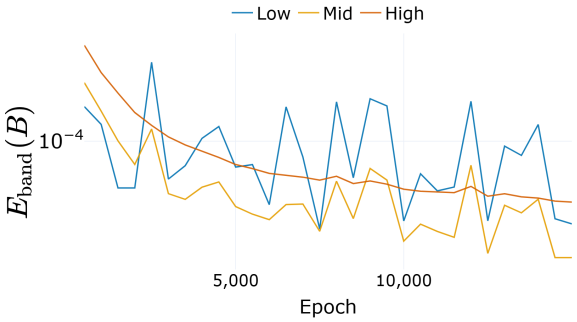
Figure 4.16 – Neural Tangent Kernel diagnostic of Section 3.5.2 for the minimal experiment at $\alpha = \beta = 3{,}21$, logged along the same gradient-descent runs as Figure 4.14, for widths 8, 64 and 512. This figure was made inspired by Wang, Yu, et al. (2022) and Jacot et al. (2018).

Source: The author (2026).

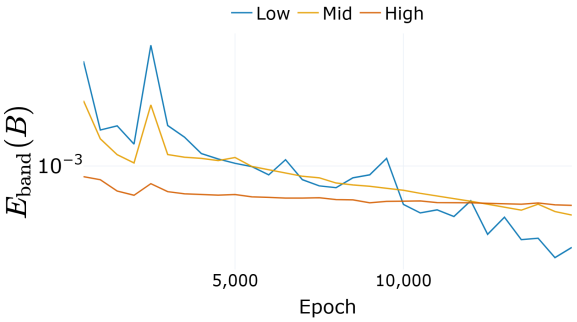
bands of (3.11), across the training for all six methods defined in Section 3.4 at parameter $\alpha = \beta = 3{,}21$ and resolution 256.



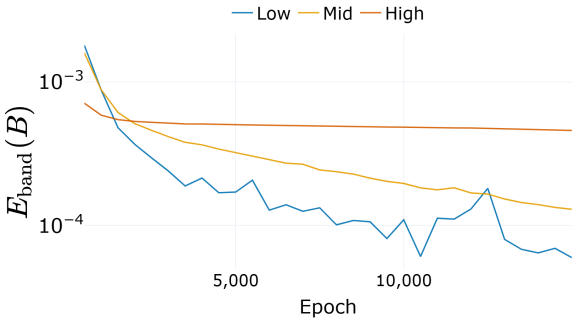
(a) MLP (pure data), at $\alpha = \beta = 3{,}21$.



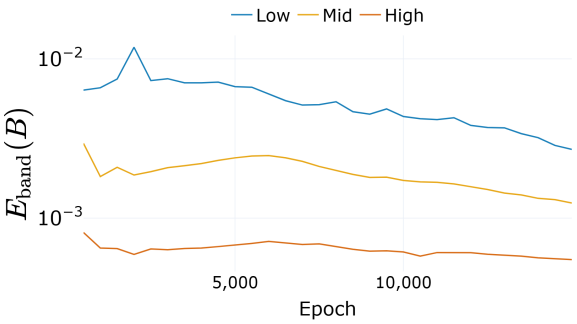
(b) FNO (pure data), at $\alpha = \beta = 3{,}21$.



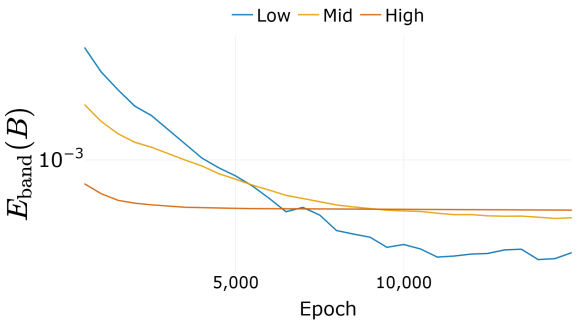
(c) PINN (data and physics), at $\alpha = \beta = 3{,}21$.



(d) PINO (data and physics), at $\alpha = \beta = 3{,}21$.



(e) PINN (pure physics), at $\alpha = \beta = 3{,}21$.



(f) PINO (pure physics), at $\alpha = \beta = 3{,}21$.

Figure 4.17 – Band-averaged absolute spectral error $E_{\text{band}}(B)$ of (3.10) during training, every method tracked at the common parameter $\alpha = \beta = 3{,}21$. The low band (blue), mid band (yellow) and high band (red) follow (3.11).

Source: The author (2026).

As one can see, the behavior of the low band is similar across all methods, with the error dropping quickly and leveling off. The mid band shows a more gradual decrease, while the high band remains relatively flat, indicating that these methods struggle to capture high-frequency components effectively. The choice of normalization technique here mattered empirically for us; we saw that the low band fall off was much faster and

consistent across all methods without our normalization (3.5). After implementation of our presented normalization, the low band fall off was less consistent and clear.

This picture matches the one anticipated in Chapter 1 that these methods complement the pseudospectral solver that sets the accuracy ceiling here rather than replacing it. Their value is in amortizing the parameter sweep and in the temporal regularization physics supplies, and it is smallest at the high wavenumbers, where the reference solver is unchallenged. Overall, this shows that physics-informed interpolation can be a somewhat reliable way of approximating the continuous solution, but fitting physics alone must be taken with a grain of salt.

5 CONCLUSION

After an extensive set of tests, we can finally draw some general and specific conclusions. First, spectral bias shows up in every architecture tested here, growing during training much as Chapter 2 predicts (Figure 4.17). Data supervision fits the low and mid frequency bands (3.11) better than a pure physics loss does, in both the network and the operator architectures. The rest of this chapter reads that result method by method, then marks the limits of the study and the paths still open.

5.1 Assessment of Each Method Family

We classify the six implemented methods as NN-focused and NO-focused, following the distinction made in Section 1.3, and directly answer what Chapter 1 set out to ask.

Neural Networks: MLP and PINN

These methods fit one Boussinesq parameter at a time. The MLP, trained on data alone, shows the spectral bias pattern the theory predicts. Figure 4.17a shows it fitting the low band fast and the high band slowly.

The pure-physics PINN struggles the most. Without data to anchor it, the band error (3.10) barely moves in the mid and high bands over the whole run (Figure 4.17), so the wave's fine structure, its high-frequency content, is the part it never really learns.

The PINN trained on both data and physics keeps the lowest time-resolved error $E(t_n)$ (3.8), the plain MLP follows, and the pure-physics PINN trails with a much wider spread. Figure 4.1 puts them side by side for the same problem.

Here, data, not the physics residual, is what pins the solution down. The residual works better as a regularizer on top of data than as a replacement for it, at least for the Boussinesq system (2.1) and the parameter range tested here.

Neural Operators: FNO and PINO

An operator learns the whole parameter range at once, so it can predict at values close to the ones it trained on, not just the one it saw.

Figures 4.2–4.4 sweep the parameters for all three operators. The FNO fits the data well but still carries spectral bias, and its main weakness is the Gibbs artifact discussed in Section 2.4.2, since it treats time as periodic, error spikes appear near the edges of the time window.

The pure-physics PINO repeats the pure-physics PINN's problem, missing the high-frequency ripples of the solution (Figure 4.17).

The PINO trained on data and physics together does best across the parameter sweep, combining a close fit with the physics constraint.

Figures 4.5–4.7 then query the operators off their training resolution. All three keep working as expected, so discretization invariance holds, though accuracy drops a little as resolution grows. Even with that drop, an operator remains a practical way to cover a range of parameters that would otherwise need a new network trained for each one.

Spectral Bias MLP Study

Section 3.5 runs the same test directly on the Spectral Bias MLP, and the same picture holds.

Figure 4.15 shows the network struggling to fit the target profile (3.13) at every width tested. As long as its Neural Tangent Kernel stays close to its value at initialization, the theory of Chapter 2 predicts the loss along each kernel eigendirection well, as seen in Figure 4.14a, and gives a good a priori estimate (2.47) of how many iterations each frequency needs (Figure 4.14d) for a given error.

5.2 What This Study Cannot Claim and Directions Forward

We have many options to branch out from the results above, and they should be weighed before the conclusions are carried anywhere else.

Every experiment used one initial condition, the unit solitary profile of Table 3.1, and swept only the single value $\alpha = \beta$ (3.2). The operators therefore learned a one-parameter restriction of the Boussinesq system (2.1) and not the four-parameter family classified by Bona et al. (2002) and Martins (2023). Decoupling α from β, or varying the initial data, is a different challenge left open and needs further investigation.

The reference solution is itself a numerical solution, accurate to the order of the pseudospectral scheme it was solved with and not exact. This matters most at the high wavenumbers, where the reference carries the largest share of its own error, and Figures 4.8–4.13 show that this is also where every trained method is weakest. One possible approach is to design forcing terms in the PDE (2.1) that produce a known analytical

solution, so the reference is exact and the error can be measured directly, building a fair comparison between ML methods and classical methods.

The Spectral Bias MLP (Section 3.5.1) has time fixed, the target is a single profile, training is plain gradient descent, and the kernel is small enough to eigendecompose at every checkpoint. All six methods compared in Chapter 4 run under none of these simplifications, so the exact predictions of Figure 4.14 do not carry over to them directly. Only the general trend, low frequencies learned before high ones, does.

Mitigating this spectral bias is a natural next step, and the literature already offers many candidates such as adaptive weighting of the loss terms (WANG; TENG, et al., 2021), the Neural Tangent Kernel weighting that Wang, Yu, et al. (2022) derived for PINNs, and the second-order, spectrum-aware optimizers of Khodakarami et al. (2026).

5.3 Final Considerations

Spectral bias is a real behavior in these architectures, and someone willing to implement these architectures will need a workaround like the mitigations discussed or at least consider that the approximation cannot be fully trusted at high-frequency details.

None of this argues for replacing classical numerical solvers with ML ones. Industrial fluid dynamics will keep relying on classical solvers, and the two go well together. Once trained, an operator answers a whole parameter sweep in one forward pass, at a fraction of what repeated solver calls would cost. The classical solver still supplies the reference data ML methods train on and check against, and it carries an abundant, mature theory and formalism with it. This is what is missing from ML methods: error bounds and convergence guarantees that come standard with a finite-difference or spectral scheme are not yet natural to this way of approximating solutions to PDEs. Of course, the field is still moving, with new mitigations for problems being developed every year, including for spectral bias.

The comparison in this work, spectral bias, training cost, and generalization across one parameter, covers four architectures on one PDE. Other equations, other parameter families, and other architectures would need their own version of the same test.

All of this considered, we finish this work noting that this is a novel field with many open questions but great potential, as ML tends to gain more and more importance in the field of numerical solutions of PDEs and scientific computing. We hope this is a good entry point for someone willing to implement these architectures and test them on their own problems, and that the results here help them understand what to expect from each method.

REFERENCES

ARORA, S. et al. Fine-Grained Analysis of Optimization and Generalization for Overparameterized Two-Layer Neural Networks. In: PROCEEDINGS of the 36th International Conference on Machine Learning (ICML). [S.l.: s.n.], 2019. v. 97, p. 322–332.

BAKER, N. et al. **Workshop Report on Basic Research Needs for Scientific Machine Learning: Core Technologies for Artificial Intelligence**. Washington, DC, 2019. DOI: 10.2172/1478744.

BIHLO, A.; POPOVYCH, R. O. Physics-informed neural networks for the shallow-water equations on the sphere. **Journal of Computational Physics**, 2022. Available from: <https://arxiv.org/abs/2104.00615>. Visited on: 2 Sept. 2026.

BONA, J. L.; CHEN, M.; SAUT, J.-C. Boussinesq equations and other systems for small-amplitude long waves in nonlinear dispersive media: I. Derivation and linear theory. **Journal of Nonlinear Science**, v. 12, p. 283–318, 2002. DOI: 10.1007/s00332-002-0466-4.

BONEV, B. et al. Spherical Fourier Neural Operators: Learning Stable Dynamics on the Sphere. In: INTERNATIONAL Conference on Machine Learning (ICML). [S.l.: s.n.], 2023. Available from: <https://arxiv.org/abs/2306.03838>. Visited on: 2 Sept. 2026.

BORDELON, B.; CANATAR, A.; PEHLEVAN, C. Spectrum Dependent Learning Curves in Kernel Regression and Wide Neural Networks. In: PROCEEDINGS of the 37th International Conference on Machine Learning (ICML). [S.l.: s.n.], 2020. Available from: <https://arxiv.org/abs/2002.02561>. Visited on: 2 Sept. 2026.

BOUSSINESQ, J. Théorie des ondes et des remous qui se propagent le long d'un canal rectangulaire horizontal. **Journal de Mathématiques Pures et Appliquées**, v. 17, p. 55–108, 1872.

BOYCE, W. E.; DIPRIMA, R. C. **Elementary Differential Equations and Boundary Value Problems**. 10. ed. Hoboken, NJ: John Wiley & Sons, 2012.

BRUNTON, S. L.; PROCTOR, J. L.; KUTZ, J. N. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. **Proceedings of the National Academy of Sciences**, v. 113, n. 15, p. 3932–3937, 2016. DOI: 10.1073/pnas.1517384113.

BURDEN, R. L.; FAIRES, J. D. **Numerical Analysis**. 9. ed. Boston: Brooks/Cole, 2011.

CAO, Y. et al. Towards Understanding the Spectral Bias of Deep Learning. In: PROCEEDINGS of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI). [S.l.: s.n.], 2021. Available from: <https://arxiv.org/abs/1912.01198>. Visited on: 2 Sept. 2026.

CARROLL, S. **The Particle at the End of the Universe: How the Hunt for the Higgs Boson Leads Us to the Edge of a New World**. New York: Dutton, 2012.

CHEN, R. T. Q. et al. Neural Ordinary Differential Equations. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2018. v. 31. Available from: <https://arxiv.org/abs/1806.07366>. Visited on: 2 Sept. 2026.

CHEN, T.; CHEN, H. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. **IEEE Transactions on Neural Networks**, v. 6, n. 4, p. 911–917, 1995.

CHIZAT, L.; OYALLON, E.; BACH, F. On Lazy Training in Differentiable Programming. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2019. v. 32. Available from: <https://arxiv.org/abs/1812.07956>. Visited on: 2 Sept. 2026.

CUOMO, S. et al. Physics-informed neural networks for data-driven simulation: Advantages, limitations, and opportunities. **Physica A: Statistical Mechanics and its Applications**, 2023. DOI: 10.1016/j.physa.2022.128415.

CYBENKO, G. Approximation by superpositions of a sigmoidal function. **Mathematics of Control, Signals and Systems**, v. 2, n. 4, p. 303–314, 1989.

DE PINA, I. et al. Investigating Steady Unconfined Groundwater Flow using Physics Informed Neural Networks. **Advances in Water Resources**, 2023. DOI: 10.1016/j.advwatres.2023.104445.

DEBNATH, L. **Nonlinear Partial Differential Equations for Scientists and Engineers**. 3. ed. New York: Birkhäuser, 2012.

GIBBS, J. Willard. Fourier's Series. **Nature**, v. 59, n. 1539, p. 606, 1899. DOI: 10.1038/059606a0.

GOLUB, G. H.; VAN LOAN, C. F. **Matrix Computations**. 4. ed. Baltimore: Johns Hopkins University Press, 2013.

GUPTA, G. et al. Multiwavelet-based Operator Learning for Differential Equations. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2021. Available from: <https://arxiv.org/abs/2109.13459>. Visited on: 2 Sept. 2026.

HORNIK, K.; STINCHCOMBE, M.; WHITE, H. Multilayer feedforward networks are universal approximators. **Neural Networks**, v. 2, n. 5, p. 359–366, 1989.

JACOT, A.; GABRIEL, F.; HONGLER, C. Neural Tangent Kernel: Convergence and Generalization in Neural Networks. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2018. v. 31. Available from: <https://arxiv.org/abs/1806.07572>. Visited on: 2 Sept. 2026.

JAGTAP, A. D. et al. Deep Learning of Inverse Water Waves Problems Using Multi-Fidelity Data: Application to Serre–Green–Naghdi Equations. **arXiv preprint arXiv:2202.02899**, 2022. Available from: <https://arxiv.org/abs/2202.02899>. Visited on: 2 Sept. 2026.

KARNIADAKIS, G. E. et al. Physics-informed machine learning. **Nature Reviews Physics**, v. 3, n. 6, p. 422–440, 2021. DOI: 10.1038/s42254-021-00314-5.

KHODAKARAMI, S. et al. Spectral bias in physics-informed and operator learning: analysis and mitigation guidelines. **arXiv preprint**, 2026. Available from: <https://arxiv.org/abs/2602.19265>. Visited on: 2 Sept. 2026.

KINGMA, D. P.; BA, J. Adam: A method for stochastic optimization. **arXiv preprint arXiv:1412.6980**, 2014.

KOSSAIFI, J. et al. A Library for Learning Neural Operators. **arXiv preprint arXiv:2412.10354**, 2024. Available from: <https://arxiv.org/abs/2412.10354>. Visited on: 2 Sept. 2026.

KREYSZIG, E. **Introductory functional analysis with applications**. New York: John Wiley & Sons, 1978.

KRISHNAPRIYAN, A. S. et al. Characterizing Possible Failure Modes in Physics-Informed Neural Networks. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2021. v. 34, p. 26548–26560.

KUMAR, S.; DHAMA, S. Physics-informed neural networks for exploring soliton collisions in the non-integrable Schamel equation in plasmas. **Physics of Fluids**, v. 37, n. 1, 2025. DOI: 10.1063/5.0301334.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. **Nature**, v. 521, n. 7553, p. 436–444, 2015. DOI: 10.1038/nature14539.

LEVEQUE, R. J. **Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems**. Philadelphia: Society for Industrial and Applied Mathematics, 2007. DOI: 10.1137/1.9780898717839.

LI, Z. et al. Physics-Informed Neural Operator for Learning Partial Differential Equations. **ACM/IMS Transactions on Data Science**, 2023. Available from: <https://arxiv.org/abs/2111.03794>. Visited on: 2 Sept. 2026.

LI, Z. et al. **PINO: Physics-Informed Neural Operator – Reference Implementation**. [S.l.: s.n.], 2023. GitHub repository. Available from: <https://github.com/neuraloperator/PINO>. Visited on: 2 Sept. 2026.

LI, Z. et al. Fourier Neural Operator for parametric partial differential equations. In: INTERNATIONAL Conference on Learning Representations (ICLR). [S.l.: s.n.], 2021. Available from: <https://arxiv.org/abs/2010.08895>. Visited on: 2 Sept. 2026.

LINNAINMAA, A. Taylor expansion of the accumulated rounding error. **BIT Numerical Mathematics**, v. 10, n. 1, p. 47–55, 1970.

LIU, H. et al. Prediction of phase transition and time-varying dynamics of the (2+1)-dimensional Boussinesq equation by parameter-integrated PINNs with phase domain decomposition. **Physical Review E**, v. 108, n. 4, p. 045303, 2023. DOI: 10.1103/PhysRevE.108.045303.

LKHAGVASUREN, B.; CHOI, B.-J.; JIN, H. S. Applications of the Fourier neural operator in a regional ocean modeling and prediction. **Frontiers in Marine Science**, v. 11, 2024. DOI: 10.3389/fmars.2024.1383997.

LU, L. et al. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. **Nature Machine Intelligence**, v. 3, p. 218–229, 2021. DOI: 10.1038/s42256-021-00302-5.

MARTINS, L. G. **Estudo Numérico do Modelo Boussinesq com Topografia e Pressão Variável no Espaço e no Tempo**. 2023. MA thesis – Universidade Federal do Paraná, Curitiba.

MARTINS, L. G.; FLAMARION, M. V.; RIBEIRO-JR, R. A forced Boussinesq model with a sponge layer. **Partial Differential Equations in Applied Mathematics**, v. 10, p. 100661, 2024. DOI: 10.1016/j.padiff.2024.100661.

MCCULLOCH, W. S.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. **Bulletin of Mathematical Biophysics**, v. 5, n. 4, p. 115–133, 1943.

NASCIMENTO, Maria Francisca do; NEVES, Claudio Freitas; MACIEL, Geraldo de Freitas. Modelo Numérico de Boussinesq Adaptado para Ondas de

Embarcação. **Revista Brasileira de Recursos Hídricos**, v. 15, n. 4, p. 5–15, 2010. DOI: 10.21168/rbrh.v15n4.p5-15.

NOCEDAL, J. Updating quasi-Newton matrices with limited storage. **Mathematics of Computation**, v. 35, n. 151, p. 773–782, 1980.

PARIKH, N.; BOYD, S. Proximal Algorithms. **Foundations and Trends in Optimization**, v. 1, n. 3, p. 123–231, 2013. DOI: 10.1561/2400000003.

PASZKE, A. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: ANNUAL Conference on Neural Information Processing Systems (NeurIPS). Vancouver: [s.n.], 2019. p. 8024–8035. Available from: <https://arxiv.org/abs/1912.01703>. Visited on: 2 Sept. 2026.

PATEL, A. et al. A Neural Operator-Based Emulator for Regional Shallow Water Dynamics. **arXiv preprint**, 2025. Available from: <https://arxiv.org/abs/2502.14782>. Visited on: 2 Sept. 2026.

PEREGRINE, D. H. Long waves on a beach. **Journal of Fluid Mechanics**, v. 27, n. 4, p. 815–827, 1967. DOI: 10.1017/S0022112067002605.

RACKAUCKAS, C. et al. Universal Differential Equations for Scientific Machine Learning. **arXiv preprint arXiv:2001.04385**, 2020.

RAHAMAN, N. et al. On the spectral bias of neural networks. In: PMLR. INTERNATIONAL Conference on Machine Learning (ICML). Long Beach: [s.n.], 2019. p. 5301–5310.

RAISSI, M.; PERDIKARIS, P.; KARNIADAKIS, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. **Journal of Computational Physics**, v. 378, p. 686–707, 2019.

ROSENBLATT, F. The perceptron: a probabilistic model for information storage and organization in the brain. **Psychological Review**, v. 65, n. 6, p. 386–408, 1958.

ROSOFSKY, S. et al. Applications of Physics-Informed Neural Operators (PINOs): Shallow Water Equations and Beyond. **arXiv preprint**, 2023. Available from: <https://github.com/shawnrosofsky/PINO_Applications>. Visited on: 2 Sept. 2026.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. **Nature**, v. 323, n. 6088, p. 533–536, 1986.

SHARMA, Pushan et al. A Review of Physics-Informed Machine Learning in Fluid Mechanics. **Energies**, v. 16, n. 5, 2023. DOI: 10.3390/en16052343.

STEINMOELLER, D. T.; STASTNA, M.; LAMB, K. G. Fourier pseudospectral methods for 2D Boussinesq-type equations. **Ocean Modelling**, v. 52–53, p. 76–89, 2012. DOI: 10.1016/j.ocemod.2012.05.003.

STRANG, G. **Linear Algebra and Its Applications**. 4. ed. Belmont: Thomson Brooks/Cole, 2006.

SÜLI, E.; MAYERS, D. F. **An Introduction to Numerical Analysis**. Cambridge: Cambridge University Press, 2003.

TANCIK, M. et al. Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2020. v. 33.

TREFETHEN, L. N. **Spectral Methods in MATLAB**. Philadelphia: Society for Industrial and Applied Mathematics, 2000. DOI: 10.1137/1.9780898719598.

UCHIDA, T. et al. Ocean Emulation With Fourier Neural Operators: Double Gyre. **Journal of Advances in Modeling Earth Systems**, v. 17, n. 1, 2025. DOI: 10.1029/2023MS004137.

WANG, L. et al. Explorations of nonlinear waves of the Boussinesq and Camassa-Holm equations using PINNs. **Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences**, v. 480, n. 2286, 2024. DOI: 10.1098/rspa.2023.0580.

WANG, S.; TENG, Y.; PERDIKARIS, P. Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks. **SIAM Journal on Scientific Computing**, v. 43, n. 5, a3055–a3081, 2021. DOI: 10.1137/20M1318043.

WANG, S.; YU, X.; PERDIKARIS, P. When and Why PINNs Fail to Train: A Neural Tangent Kernel Perspective. **Journal of Computational Physics**, v. 449, p. 110768, 2022. DOI: 10.1016/j.jcp.2021.110768.

WHITHAM, G. B. **Linear and Nonlinear Waves**. New York: Wiley-Interscience, 1974.

XU, Z.-Q. J. et al. Frequency Principle: Fourier Analysis Sheds Light on Deep Neural Networks. **Communications in Computational Physics**, v. 28, n. 5, p. 1746–1767, 2020. DOI: 10.4208/cicp.OA-2020-0085.

YU, J. et al. Spherical multigrid neural operator for improving autoregressive global weather forecasting. **Scientific Reports**, v. 15, n. 1, 2025. DOI: 10.1038/s41598-025-96208-y.

ZHANG, Y. et al. Influence of layer and neuron configurations on Boussinesq equation solutions via a bilinear neural network. **Nonlinear Dynamics**, v. 112, p. 12315–12333, 2024. DOI: 10.1007/s11071-024-09708-3.